

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SANTA CATARINA
CÂMPUS FLORIANÓPOLIS
DEPARTAMENTO ACADÊMICO DE ELETRÔNICA

GUILHERME DA COSTA FRANCO

**DESENVOLVIMENTO DE UMA PLATAFORMA VESTÍVEL MODULAR
MULTISSENSOR BASEADA NO ESP32-C6**

FLORIANÓPOLIS
2026

Guilherme da Costa Franco

Desenvolvimento de uma plataforma vestível modular multissensor baseada no
ESP32-C6

Monografia apresentada ao Curso de Engenharia Eletrônica do Instituto Federal de Santa Catarina, Câmpus Florianópolis, para obtenção do título de Bacharel em Engenharia Eletrônica.

Orientador: Prof. Me. Leandro Schwarz.

Florianópolis
2026

Ficha de identificação da obra elaborada pelo autor.

Franco, Guilherme

DESENVOLVIMENTO DE UMA PLATAFORMA VESTÍVEL MODULAR

MULTISSENSOR BASEADA NO ESP32-C6 / Guilherme Franco; orientação de **Leandro Schwarz**. - Florianópolis, SC, 2026.

102 p.

Trabalho de Conclusão de Curso (TCC) - Instituto Federal de Santa Catarina, Câmpus Florianópolis. Bacharelado em Engenharia Eletrônica. Departamento Acadêmico de Eletrônica.

Inclui Referências.

1. Sistemas embarcados. 2. Smartwatch. 3. Sensores. 4. ESP32-C6. 5. Internet das Coisas. I. Schwarz, Leandro. II. Instituto Federal de Santa Catarina. III. DESENVOLVIMENTO DE UMA PLATAFORMA VESTÍVEL MODULAR MULTISSENSOR BASEADA NO ESP32-C6.

Guilherme da Costa Franco

Desenvolvimento de uma plataforma vestível modular multissensor baseada no
ESP32-C6

Monografia apresentada ao Curso de Engenharia Eletrônica do Instituto Federal de Santa Catarina, Câmpus Florianópolis, para obtenção do título de Bacharel em Engenharia Eletrônica, aprovada em sua forma final pela banca examinadora abaixo.

Florianópolis, 6 de agosto de 2026.

Prof. Me. Leandro Schwarz
Orientador
Instituto Federal de Santa Catarina

Prof. Dr. Fernando Pedro Henriques de Miranda
Instituto Federal de Santa Catarina

Prof. Dr. Renan Augusto Starke
Instituto Federal de Santa Catarina

Dedico este trabalho aos meus pais, pelo incentivo e apoio ao longo de toda a minha formação acadêmica. Seguimos fazendo o melhor que podemos, sem olhar para trás com pesar, sabendo que sempre há tempo de escolher novos caminhos.

AGRADECIMENTOS

Agradeço, com todo o pudor da ironia, às minhas incontáveis tentativas fracassadas de desistir da graduação, que tantas vezes circundaram minha mente em momentos de fragilidade. Vivenciei na pele que o fracasso, por mais dolorosa que essa experiência tenha sido, também pode servir de alicerce para grandes fundações, principalmente quando deixamos de nos guiar por uma ótica pessimista e pela tendência incontrolável de subestimar a própria capacidade, sem que isso nos leve a diminuir o outro ou a confundir confiança com soberba.

À minha família, seja ela definida por laços diretos ou por conexões que, em um grafo afetivo, teriam peso infinitamente maior do que qualquer grau de parentesco poderia sugerir. Em especial, agradeço à minha tia Zaira Sinara, pelo acolhimento, pela orientação e pelo apoio nos momentos em que precisei de firmeza e cuidado.

Agradeço também à minha prima Denise, que carinhosamente sempre chamei de tia, por ter exercido esse mesmo papel com tanto amor, companheirismo e risadas soltas. Vocês foram vértices fundamentais nessa trajetória, conectando afeto, escuta e incentivo quando eu mais precisei, sem que fosse necessário nenhum algoritmo de Dijkstra rodando ao fundo para me mostrar o caminho mais curto.

Sou grato também aos meus amigos e colegas de profissão Marcelo Zampieri, Caio Meira e Rafael Aquino, que foram protagonistas essenciais em todos os altos e baixos da graduação, com sua camaradagem, parceria e irreverência nos momentos mais desesperadores, como apresentações de trabalhos feitos de última hora e torres de chope de vinho que chegavam ao fim rápido demais.

Agradeço também aos meus amigos Juliana Andrade e Kevin Trovão, por terem sido verdadeiros guardiões deste trabalho. O destino nos uniu em um completo acaso e fez com que vocês também se tornassem parte importante da minha jornada acadêmica. Entre conversas, apoio e companheirismo, vocês me lembraram que nenhuma missão difícil precisa ser enfrentada completamente sozinho quando se tem um bom esquadrão ao lado.

Ao meu orientador, professor Leandro Schwarz, pelos conhecimentos transmitidos, pela confiança depositada em minhas habilidades e por ter enxergado em mim a capacidade de desenvolver um projeto do qual eu pudesse me orgulhar. Agradeço imensamente pela orientação, pelo incentivo e por ter contribuído para que este trabalho se tornasse não apenas a conclusão de uma etapa acadêmica, mas também uma importante realização pessoal.

E, por fim, agradeço ao Instituto Federal de Santa Catarina, instituição que foi parte fundamental da minha formação acadêmica, profissional e pessoal. Agradeço também aos professores do Departamento Acadêmico de Eletrônica, pela excelência didática, pela dedicação ao ensino e pela forma como conduziram, ao longo do curso,

uma formação fortemente baseada na experimentação, na prática e na construção concreta do conhecimento.

Essa abordagem foi essencial para transformar conceitos teóricos em aprendizado real, aproximando a engenharia dos desafios concretos da prática profissional e despertando em mim maior interesse pela criação, pela investigação e pela solução de problemas. Levo comigo, além dos conteúdos aprendidos, a influência de uma formação que valoriza o pensamento crítico, a autonomia e a capacidade de aprender fazendo.

“Kept you waiting, huh?”

— Snake, *Metal Gear*

RESUMO

Este trabalho apresenta o desenvolvimento e a avaliação de uma plataforma vestível modular multissensor baseada no ESP32-C6. A versão final integrou interface gráfica circular sensível ao toque, relógio de tempo real, fotopletismografia, temperatura, iluminância, radiação ultravioleta, bússola e pedômetro em uma mesma aplicação embarcada. O método combinou testes isolados dos componentes, separação entre acesso ao dispositivo, processamento e apresentação, integração incremental e regressão das funções existentes. A arquitetura resultante emprega contratos de ciclo de vida, tarefas independentes e serviços compartilhados para concorrência, persistência e recuperação do barramento I²C. Os ensaios confirmaram a navegação entre as telas, a aquisição e o tratamento dos canais sensoriais, a continuidade da inicialização nos ensaios com sensores ausentes e o funcionamento da leitura de bateria. Os resultados demonstram o atendimento dos objetivos de integração e modularidade e confirmam a plataforma como base técnica funcional para novas extensões. O alcance da avaliação é funcional e arquitetural, sem pretensão de certificação clínica ou metrológica.

Palavras-chave: sistemas embarcados; dispositivo vestível; arquitetura modular; ESP32-C6; sensores; smartwatch; integração multissensor; Internet das Coisas.

ABSTRACT

This work presents the development and evaluation of a modular multisensor wearable platform based on the ESP32-C6. The final version integrated a circular touchscreen interface, a real-time clock, photoplethysmography, temperature, illuminance, ultraviolet radiation, compass and pedometer functions into a single embedded application. The method combined standalone component tests, separation among driver, processing and presentation, incremental integration and regression of existing functions. The resulting architecture employs lifecycle contracts, independent tasks and shared services for concurrency, persistence and I²C bus recovery. The tests confirmed navigation among screens, acquisition and processing of sensor channels, continued initialization in tests with missing sensors, and operation of the battery measurement. The results demonstrate that the integration and modularity objectives were achieved and confirm the platform as a functional technical foundation for further extensions. The evaluation scope is functional and architectural and does not claim clinical or metrological certification.

Keywords: embedded systems; wearable device; modular architecture; ESP32-C6; sensors; smartwatch; multisensor integration; Internet of Things.

LISTA DE FIGURAS

1	Contrato uniforme e detalhes encapsulados em módulos conceituais . .	21
2	Diferença conceitual entre atrasos relativos e referência temporal periódica	24
3	Inversão de prioridade e efeito da herança sobre o bloqueio	26
4	Linha I ² C em dreno aberto e subida determinada pela rede de <i>pull-up</i> e pela capacitância equivalente	28
5	Geometria de uma área circular inscrita em uma matriz lógica quadrada	32
6	Configurações transmissiva e reflexiva da fotopletismografia	37
7	Decomposição conceitual do sinal PPG em componentes DC e AC . . .	38
8	Efeitos de <i>hard iron</i> e <i>soft iron</i> e alcance da correção diagonal	44
9	Detecção conceitual de passos por limiar e intervalo refratário	45
10	Método incremental de desenvolvimento e integração	54
11	Mapa de ligações da montagem final	55
12	Instrumentação empregada nos ensaios do protótipo	56
13	Relação entre a plataforma modular e a aplicação demonstradora . . .	58
14	Arquitetura de componentes da plataforma implementada	59
15	Divisor resistivo empregado na leitura da tensão da bateria	61
16	Ciclo de vida dos módulos funcionais	63
17	Fluxo para integração de um novo sensor ao <i>firmware</i>	64
18	Tarefas FreeRTOS e recursos compartilhados no núcleo único	66
19	Transação I ² C e recuperação após falha	68
20	Mapa de navegação da interface do protótipo	70
21	Aquisição e processamento implementados no módulo MAX30102 . . .	72
22	Máquina de estados implementada para o LTR390	75
23	Calibração magnética executada no protótipo	77
24	Arquitetura do módulo MPU-9250	80
25	Placa protótipo com display e módulos durante a operação	84
26	Renderizações das faces da placa desenvolvida	85
27	Dependências identificadas por análise estática do <i>firmware</i> integrado .	89
28	Linha de base do canal infravermelho e comparação espectral antes e depois do passa-baixa	89
29	Corrente total medida nos cenários funcionais da plataforma	93

LISTA DE QUADROS

1	Componentes empregados e sua função na plataforma	36
2	Comparação entre plataformas vestíveis relacionadas	46
3	Mapa consolidado de interfaces e ligações	50
4	Pseudocódigo da inicialização não fatal e do despacho da tela ativa . .	65
5	Pseudocódigo de uma transação I ² C protegida e de sua recuperação .	68
6	Pseudocódigo da aquisição e da detecção de passos	79
7	Responsabilidades na aquisição e no tratamento dos dados	81
8	Atendimento aos requisitos da plataforma	86
9	Comportamento observado durante a inicialização com sensores ausentes	88
10	Mapa dos principais artefatos publicados	97

LISTA DE TABELAS

1	Procedimentos e alcance das evidências produzidas	51
2	Comparação funcional das indicações do MAX30102 e do oxímetro . .	90
3	Contagens e erros observados no percurso de 100 m	91
4	Síntese das verificações funcionais complementares	91

SUMÁRIO

1	Introdução	14
1.1	Contextualização e problema de pesquisa	14
1.2	Objetivos	15
1.3	Justificativa, contribuições e delimitação	15
1.4	Organização do documento	17
2	Fundamentação Teórica	18
2.1	Sistemas embarcados vestíveis	18
2.2	Modularidade e sistemas de tempo real	19
2.3	Interfaces de comunicação	27
2.4	Interface gráfica, temporização, alimentação e persistência	31
2.5	Módulos empregados na plataforma	35
2.6	Processamento dos sinais e algoritmos	37
2.7	Trabalhos relacionados e síntese teórica	45
3	Metodologia	48
3.1	Materiais e infraestrutura experimental	48
3.2	Método incremental de desenvolvimento	49
3.3	Construção e integração do protótipo	49
3.4	Procedimentos de avaliação e rastreabilidade	50
4	Desenvolvimento e Implementação	57
4.1	Visão geral e plataforma física	57
4.2	Arquitetura modular e contrato de integração	61
4.3	Serviços compartilhados, concorrência e recuperação	67
4.4	Implementação dos módulos	71
4.5	Síntese da integração	80
5	Resultados e Discussão	83
5.1	Protótipo integrado e atendimento aos requisitos	83
5.2	Modularidade e contenção da ausência de sensores	86
5.3	Comportamento dos algoritmos implementados	88
5.4	Verificações funcionais dos módulos	91
5.5	Discussão integrada e limitações	93
6	Conclusões	95
A	Repositório e mapa dos artefatos	97
	Referências	98

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E PROBLEMA DE PESQUISA

Os dispositivos vestíveis combinam aquisição de dados, processamento embarcado e interação com o usuário em plataformas sujeitas a restrições de dimensões, memória, capacidade computacional e consumo de energia. A disponibilidade de microcontroladores com múltiplos periféricos, sensores miniaturizados e interfaces gráficas tornou viável a construção de sistemas capazes de reunir funções biométricas, ambientais e inerciais em um único dispositivo. Essa integração, entretanto, não depende apenas da conexão elétrica dos componentes: exige uma organização de *software* que coordene diferentes protocolos de comunicação, taxas de aquisição, requisitos de processamento e mecanismos de apresentação dos dados (SENEVIRATNE et al., 2017; HESTER et al., 2016).

À medida que novos sensores são incorporados, uma aplicação organizada de forma monolítica tende a concentrar dependências no núcleo do *firmware*. Nesse cenário, alterações em um subsistema podem repercutir nos demais, dispositivos que compartilham um barramento podem interferir entre si e a falha de um componente pode impedir a inicialização de toda a aplicação. Em um sistema embarcado com sistema operacional de tempo real, a integração também envolve concorrência entre tarefas, sincronização de recursos e definição de prioridades, aspectos que precisam ser considerados para preservar a responsividade da interface e o acesso ordenado ao *hardware* (FREERTOS, 2026c; SHA; RAJKUMAR; LEHOCZKY, 1990).

Esses desafios são particularmente relevantes em uma plataforma vestível modular multissensor. Além de acomodar dispositivos com transportes distintos, a plataforma deve permitir a evolução do *firmware* sem exigir a reestruturação do conjunto a cada nova funcionalidade. Também é desejável que funções independentes permaneçam disponíveis quando um sensor estiver ausente ou apresentar falha.

O problema abordado consiste em organizar e integrar sensores heterogêneos, interface gráfica e serviços compartilhados em um dispositivo vestível com recursos limitados, de modo que a inclusão ou a falha de um módulo não exija alterações disseminadas pelo *firmware* nem comprometa o funcionamento das funções independentes. A plataforma deve, adicionalmente, coordenar o acesso concorrente ao barramento I²C compartilhado e recuperar sua operação após falhas de comunicação.

Diante desse contexto, formula-se a seguinte questão de projeto:

Como projetar uma plataforma de *software* modular para um dispositivo vestível baseado no ESP32-C6, capaz de integrar sensores heterogêneos, compartilhar recursos de comunicação e permanecer operacional diante da ausência ou falha de módulos?

1.2 OBJETIVOS

Projetar e implementar uma plataforma vestível modular multissensor baseada no XIAO ESP32-C6, capaz de integrar sensores heterogêneos, interface gráfica e serviços compartilhados, verificando estruturalmente sua extensibilidade e observando seu comportamento diante da ausência de módulos.

Para alcançar o objetivo geral, são estabelecidos os seguintes objetivos específicos:

- definir uma arquitetura modular para sensores e telas;
- estabelecer um contrato uniforme para inicialização dos módulos, criação das telas e acesso às funcionalidades;
- implementar serviços compartilhados de comunicação, interface gráfica e concorrência;
- integrar módulos biométricos, ambientais e inerciais à aplicação demonstradora;
- implementar inicialização não fatal, de modo que a plataforma possa operar com um subconjunto dos sensores;
- coordenar o acesso ao barramento I²C compartilhado e implementar sua recuperação após falhas de comunicação;
- demonstrar a inclusão de módulos com impacto restrito sobre o núcleo e sobre os módulos existentes;
- avaliar o acoplamento, a extensibilidade, o uso estático de memória e, dentro do escopo experimental, a estabilidade e os mecanismos de robustez da plataforma;
- realizar testes funcionais dos módulos integrados; e
- produzir documentação técnica que permita compreender e reproduzir as decisões de arquitetura e integração.

1.3 JUSTIFICATIVA, CONTRIBUIÇÕES E DELIMITAÇÃO

A relevância técnica do trabalho decorre do tratamento conjunto de problemas que surgem na integração de um sistema embarcado multissensor. A coexistência de dispositivos no mesmo barramento, a execução concorrente de tarefas de aquisição e interface, as diferenças entre protocolos e a necessidade de preservar funções independentes diante de falhas exigem decisões que ultrapassam a implementação

isolada de *drivers*. Uma arquitetura com contrato uniforme, serviços compartilhados e separação entre núcleo, serviços, *drivers* e módulos oferece uma base para controlar essas dependências e tornar explícitas as responsabilidades de cada parte (PARNAS, 1972; BASS; CLEMENTS; KAZMAN, 2021).

No âmbito acadêmico, o trabalho permite discutir a aplicação de conceitos de modularidade, extensibilidade, concorrência e tolerância a falhas em uma plataforma embarcada concreta. A escolha do ESP-IDF e do FreeRTOS fornece acesso aos mecanismos de tarefas, prioridades, sincronização, periféricos e diagnóstico empregados na implementação.

Do ponto de vista prático, a plataforma reúne módulos com características distintas: o MAX30102 para fotopletismografia e estimativas de frequência cardíaca e SpO₂; o DS18B20 para temperatura; o LTR390 para iluminância e índice UV; e o conjunto MPU-9250/AK8963 para bússola e pedometria. O relógio em tempo real PCF8563 e a interface gráfica no display circular completam a aplicação demonstradora. A diversidade desses subsistemas permite observar problemas de integração que não aparecem quando cada sensor é operado isoladamente.

A contribuição principal é a arquitetura da plataforma, estruturada em módulos autocontidos e em um contrato uniforme de integração com o núcleo e a interface. A implementação inclui tarefas independentes de aquisição, ativação da aquisição conforme a tela exibida, registro de telas e funções de atualização, inicialização não fatal e operação com os módulos disponíveis.

Também integram a contribuição o tratamento do barramento I²C compartilhado por meio de exclusão mútua com herança de prioridade, a recuperação pós-NACK e o scanner de dispositivos executado na inicialização. Complementam a plataforma o uso pragmático de *drivers* próprios e componentes oficiais, bem como a persistência da calibração da bússola em memória não volátil.

O escopo compreende o projeto da arquitetura, sua implementação em C com ESP-IDF 5.5.1 e FreeRTOS em configuração unicore, a integração dos sensores e da interface gráfica e a avaliação funcional e arquitetural da plataforma. A avaliação arquitetural abrangeu extensibilidade e acoplamento por análise estática, memória global da configuração integrada e ensaios qualitativos de inicialização e operação com sensores ausentes. CPU e *heap* por cenário, memória incremental, falha I²C induzida, série de *boot* e ensaio prolongado completo foram delimitados fora do escopo experimental desta versão.

O dispositivo não possui finalidade médica. Os valores de frequência cardíaca e a estimativa de SpO₂ destinam-se a testes funcionais, sem validação clínica ou certificação para diagnóstico. Também não fazem parte do estado implementado a compensação de inclinação da bússola, a persistência do pedômetro, o gerenciamento avançado de energia e a conectividade sem fio. A placa protótipo autoral e o invólucro

impresso em 3D foram realizados, embora sua caracterização mecânica e ambiental não tenha integrado os ensaios.

1.4 ORGANIZAÇÃO DO DOCUMENTO

Além desta Introdução, o trabalho está organizado em cinco partes principais. A Fundamentação Teórica apresenta os conceitos necessários à compreensão dos dispositivos vestíveis, da arquitetura modular, do sistema operacional de tempo real, dos protocolos de comunicação, da interface gráfica e dos princípios de operação dos sensores. A Metodologia descreve os recursos, o método incremental, a construção e os procedimentos de avaliação.

Desenvolvimento e Implementação detalha a arquitetura de *hardware* e *software*, o contrato dos módulos, o modelo de concorrência, os mecanismos de robustez e a implementação dos subsistemas. Resultados e Discussão reúne as evidências obtidas nos ensaios, interpreta o comportamento da plataforma e explicita suas limitações. Por fim, as Conclusões retomam o problema e os objetivos, sintetizam as contribuições sustentadas pelos resultados e indicam possibilidades de continuidade. Referências, apêndices e eventuais anexos complementam o documento.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 SISTEMAS EMBARCADOS VESTÍVEIS

Sistemas embarcados vestíveis são dispositivos computacionais concebidos para uso junto ao corpo e para operação durante as atividades do usuário. Eles combinam processamento, sensores, comunicação, interface e alimentação em um volume limitado, o que impõe restrições simultâneas de área, memória, capacidade computacional e energia. Diferentemente de instrumentos de bancada, o posicionamento do dispositivo e os movimentos do usuário também passam a integrar as condições de medição.

Uma plataforma vestível multissensor reúne funções com requisitos distintos. A interface gráfica demanda resposta perceptível ao usuário e transferências frequentes para o display; sensores podem exigir períodos de aquisição, tempos de conversão e tratamentos de dados diferentes; serviços compartilhados coordenam barramentos, memória e armazenamento. A concorrência entre essas atividades precisa ser organizada sem perder de vista a capacidade limitada da plataforma.

A proximidade com o corpo favorece a aquisição contínua ou recorrente de sinais fisiológicos e de movimento, mas não elimina as limitações dos métodos empregados. Contato variável, movimento, luz externa, posição no punho e diferenças entre usuários podem alterar as medições. Por isso, a disponibilidade de um sensor em um dispositivo vestível não é suficiente para atribuir validade clínica aos valores produzidos, que depende de procedimento de validação compatível com a finalidade declarada (SENEVIRATNE et al., 2017).

Dentro desse cenário, a seleção de uma plataforma computacional para um vestível envolve compromissos entre dimensões, consumo, interfaces disponíveis, memória, capacidade de processamento, ecossistema de desenvolvimento e possibilidade de integrar os periféricos previstos. Esses critérios devem ser relacionados à carga da aplicação e às interfaces necessárias; a simples disponibilidade de maior frequência de clock ou conectividade não demonstra adequação ao sistema.

O ESP32-C6 integra um processador principal RISC-V de 32 bits, com frequência de até 160 MHz, 512 KiB de SRAM de alto desempenho e 16 KiB de SRAM de baixo consumo, além de interfaces como I²C, SPI, RMT, USB Serial/JTAG e conversor analógico-digital SAR de 12 bits (ESPRESSIF SYSTEMS, 2025a). Embora o circuito também contenha um processador de baixo consumo, a configuração usual do FreeRTOS para a aplicação no processador principal é de núcleo único. Assim, tarefas concorrentes compartilham o mesmo tempo de CPU e não executam em paralelo nesse núcleo.

O conjunto de instruções anunciado para o processador principal inclui extensões inteiras, de multiplicação e divisão, atômicas e comprimidas, mas não implementa as extensões RISC-V de ponto flutuante de precisão simples ou dupla (ESPRESSIF SYSTEMS, 2025b). Operações em ponto flutuante permanecem disponíveis pela linguagem e pela cadeia de compilação, porém podem recorrer a rotinas de *software*. Essa característica recomenda avaliar a necessidade e a frequência dessas operações, sem permitir inferir um custo quantitativo sem medição.

A XIAO ESP32-C6 expõe o circuito em uma placa de 21 mm por 17,8 mm, com USB, gerenciamento para bateria, 4 MiB de *flash* e um conjunto reduzido de pinos (SEEED STUDIO, 2024). Essa combinação favorece um protótipo compacto e o uso do ESP-IDF, mas também impõe limites: memória e pinos permanecem finitos, periféricos podem ser compartilhados e a capacidade de processamento precisa atender simultaneamente à interface gráfica e aos sensores. Plataformas com mais pinos, memória externa ou mais núcleos podem ser preferíveis para outras cargas; a escolha é uma decisão de engenharia situada, e não uma superioridade universal.

2.2 MODULARIDADE E SISTEMAS DE TEMPO REAL

Uma arquitetura de *software* descreve a organização de um sistema em elementos, as relações entre eles e as decisões que condicionam seu comportamento e sua evolução. No caso de sistemas embarcados, essa organização precisa considerar não apenas a distribuição das funções no código, mas também as dependências de *hardware*, os serviços compartilhados e as formas de comunicação entre partes executadas concorrentemente. A decomposição em módulos procura limitar o conhecimento que cada parte precisa manter sobre as demais.

Parnas (1972) propõe que a divisão de um sistema seja orientada pelas decisões de projeto suscetíveis a mudança. Cada módulo deve ocultar dos outros uma decisão, representação ou mecanismo que possa variar, expondo somente o necessário para sua utilização. Esse critério difere de uma separação baseada apenas na sequência de etapas do processamento: arquivos distintos ou funções agrupadas por conveniência não constituem, por si sós, uma arquitetura modular.

A modularidade pode ser examinada pela relação entre coesão e acoplamento. A coesão expressa o grau em que os elementos internos de um módulo contribuem para uma responsabilidade bem definida. Um módulo coeso reúne dados e operações relacionados ao mesmo propósito, o que facilita sua compreensão e reduz a necessidade de coordenar alterações internas sem relação entre si. O acoplamento, por sua vez, caracteriza as dependências entre módulos, como o compartilhamento de dados, o conhecimento de representações internas ou a dependência de uma sequência par-

ricular de chamadas. No projeto estruturado, busca-se elevar a coesão e reduzir o acoplamento para limitar a propagação de alterações pelo sistema (STEVENS; MYERS; CONSTANTINE, 1974).

Baixo acoplamento não significa ausência de interação. Os módulos precisam cooperar para que o sistema realize suas funções, mas essa cooperação pode ocorrer por relações explícitas e restritas. Quando um módulo depende apenas de uma interface estável, alterações em sua implementação interna tendem a permanecer localizadas. Em sentido oposto, o acesso direto ao estado interno, a duplicação de conhecimento e as dependências implícitas aumentam a quantidade de partes que precisam ser examinadas ou modificadas em conjunto.

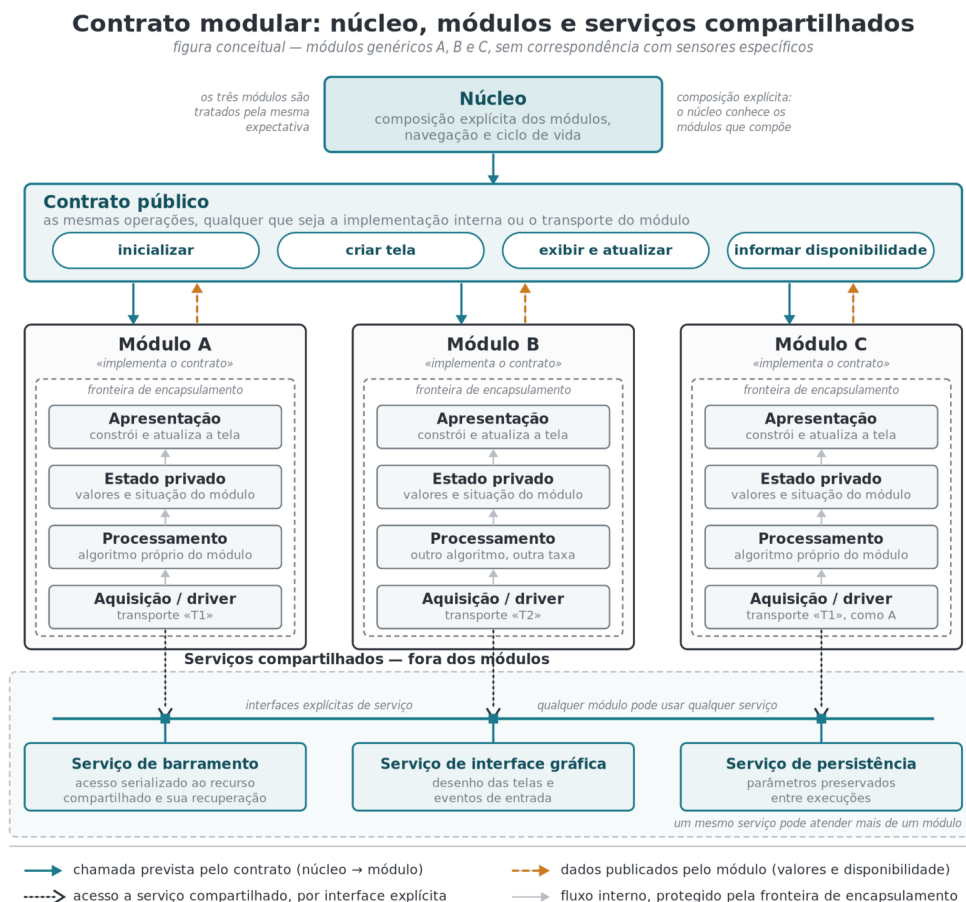
Em plataformas embarcadas, o acoplamento também pode decorrer do compartilhamento de recursos físicos. Dois módulos logicamente separados continuam relacionados se utilizam o mesmo barramento, periférico ou região de memória. A arquitetura deve tornar essa dependência visível e atribuir a um serviço ou mecanismo definido a responsabilidade de coordená-la. A separação do código não elimina conflitos de acesso ao *hardware*.

Para controlar esse acoplamento, a interface de um módulo estabelece os elementos pelos quais ele pode ser utilizado, enquanto oculta os detalhes que não devem ser conhecidos externamente. Ela pode especificar operações disponíveis, tipos de dados, estados possíveis, condições de chamada e formas de comunicar sucesso ou falha. Para preservar o isolamento, uma interface deve oferecer informação suficiente ao uso correto do módulo sem expor decisões internas que possam mudar (PARNAS, 1972).

O conceito de contrato acrescenta às assinaturas das operações as obrigações assumidas pelas partes envolvidas. Em sua formulação clássica, um contrato explicita pré-condições que o chamador deve satisfazer, pós-condições garantidas pela operação e invariantes que permanecem válidas durante o ciclo de vida do componente (MEYER, 1992). Em *firmware* escrito em C, um contrato pode ser expresso por tipos, valores de retorno, documentação, convenções de estado e verificações executadas pelo código, mesmo quando não há suporte nativo da linguagem para contratos formais.

Um contrato uniforme permite que o núcleo trate módulos distintos por um mesmo conjunto de expectativas. Isso não obriga todos os módulos a possuírem implementações ou meios de transporte idênticos. Diferenças de *hardware* podem permanecer encapsuladas, desde que as operações necessárias à integração e o comportamento observável diante de sucesso ou falha estejam definidos. A Figura 1 sintetiza essa relação entre o contrato público, o estado interno dos módulos e os serviços compartilhados.

Figura 1 – Contrato uniforme e detalhes encapsulados em módulos conceituais



Fonte: Elaborado pelo autor com base em Parnas (1972) e Meyer (1992).

O ciclo de vida de um módulo pode compreender inicialização, ativação, operação, desativação e liberação de recursos. O contrato precisa distinguir um módulo ainda não inicializado, disponível, temporariamente indisponível ou em falha, de modo que chamadas fora do estado permitido possam ser rejeitadas de forma previsível. A inicialização não fatal é uma política arquitetural em que a indisponibilidade de uma função opcional é registrada e isolada, permitindo prosseguir com funções sem dependência daquele recurso.

Na implementação desses princípios, uma decomposição recorrente de *firmware* separa a adaptação ao *hardware*, o *driver* do dispositivo, o processamento do sinal, a tarefa de coordenação e a apresentação. A camada de abstração de *hardware* (Hardware Abstraction Layer — HAL) representa capacidades do periférico do microcontrolador; o *driver* traduz registradores e transações do componente; o processamento transforma amostras em grandezas ou eventos; a tarefa define quando adquirir e publicar; e a apresentação transforma o estado em elementos da interface.

Essa separação reduz a propagação de detalhes, mas não cria independência absoluta. Um algoritmo continua condicionado pela taxa de amostragem e pela faixa

configuradas no *hardware*, e uma tela continua condicionada à disponibilidade e à validade dos dados. Em sistemas restritos, interfaces adicionais, cópias de dados e estados intermediários também consomem memória e tempo. A abstração deve preservar dependências relevantes e ocultar apenas as decisões que não precisam atravessar a fronteira do módulo.

Além de localizar responsabilidades, essa organização busca extensibilidade, entendida como a capacidade de incorporar novas funções ou ampliar funções existentes com impacto controlado sobre a arquitetura. Ela não pode ser inferida apenas da existência de diretórios ou funções separados. Sua avaliação requer um cenário de mudança que identifique o estímulo, a parte modificada, as condições da alteração e uma medida de resposta, como número de módulos afetados, interfaces modificadas ou esforço necessário (BASS; CLEMENTS; KAZMAN, 2021).

A extensibilidade depende das decisões tomadas antecipadamente sobre pontos de variação. Interfaces estáveis, responsabilidades localizadas e serviços comuns reduzem a necessidade de repetir código ou alterar o núcleo a cada inclusão. Entretanto, mecanismos genéricos também possuem custo de memória, processamento e complexidade. Em sistemas com recursos restritos, a arquitetura deve oferecer os pontos de extensão necessários ao problema sem introduzir abstrações cujo custo não seja justificado.

A demonstração de extensibilidade deve observar uma modificação concreta. A adição de um novo módulo, por exemplo, permite registrar quais arquivos e interfaces precisaram mudar e se módulos existentes permaneceram inalterados. Esse procedimento produz evidência mais direta do que a classificação da arquitetura com base apenas em sua estrutura declarada.

A modularidade também delimita o alcance de falhas. Na terminologia de dependabilidade, uma falha interna pode produzir um estado incorreto e, caso esse estado alcance a interface do sistema, resultar na interrupção ou na prestação incorreta de um serviço. A tolerância a falhas compreende os meios empregados para evitar que a presença de uma falha provoque a falha do serviço oferecido (AVIZIENIS et al., 2004). Essa propriedade não equivale à impossibilidade de falhas nem garante, isoladamente, confiabilidade absoluta.

Em uma plataforma composta por funções independentes, a falha ou ausência de um sensor não precisa impedir a inicialização dos demais módulos. Para isso, a arquitetura deve conter o erro no subsistema afetado, comunicar sua indisponibilidade e preservar os serviços que não dependem dele. A continuidade obtida está limitada pelas dependências reais: a falha de um recurso compartilhado pode afetar vários módulos e exige mecanismos específicos de coordenação, detecção e recuperação.

Esse comportamento deve ser avaliado pela introdução controlada de falhas e pela observação dos serviços preservados. Ensaaios com sensores ausentes, erros

de comunicação e recuperação de recursos compartilhados permitem distinguir uma decisão arquitetural implementada de uma propriedade efetivamente demonstrada.

A coordenação de módulos executados concorrentemente pode ser realizada por um sistema operacional de tempo real (Real-Time Operating System — RTOS) organiza a execução de atividades concorrentes de acordo com requisitos temporais. Seu objetivo não é apenas executar operações rapidamente, mas oferecer comportamento temporal suficientemente previsível para que eventos sejam tratados dentro dos limites definidos pela aplicação. Esses limites podem assumir a forma de períodos de ativação, prazos, latências máximas ou tempos de resposta.

Em um sistema embarcado multissensor, o uso de um sistema operacional de tempo real permite representar aquisições, processamento e interface como tarefas separadas. Essa decomposição facilita a atribuição de períodos e prioridades, mas não garante por si só que os requisitos temporais sejam atendidos. O tempo de execução das tarefas, as relações de precedência, o compartilhamento de recursos e os intervalos em que interrupções ou regiões críticas impedem o escalonamento também afetam o comportamento do sistema.

No nível de execução, uma tarefa representa um fluxo com contexto e pilha próprios. Durante sua existência, ela transita entre estados controlados pelo núcleo: pode estar em execução, pronta para executar, bloqueada à espera de tempo ou evento, ou suspensa explicitamente. Uma tarefa bloqueada não disputa o processador até que sua condição de desbloqueio seja satisfeita, o que permite aguardar períodos, filas, notificações ou recursos sem executar espera ocupada (FREERTOS, 2026a).

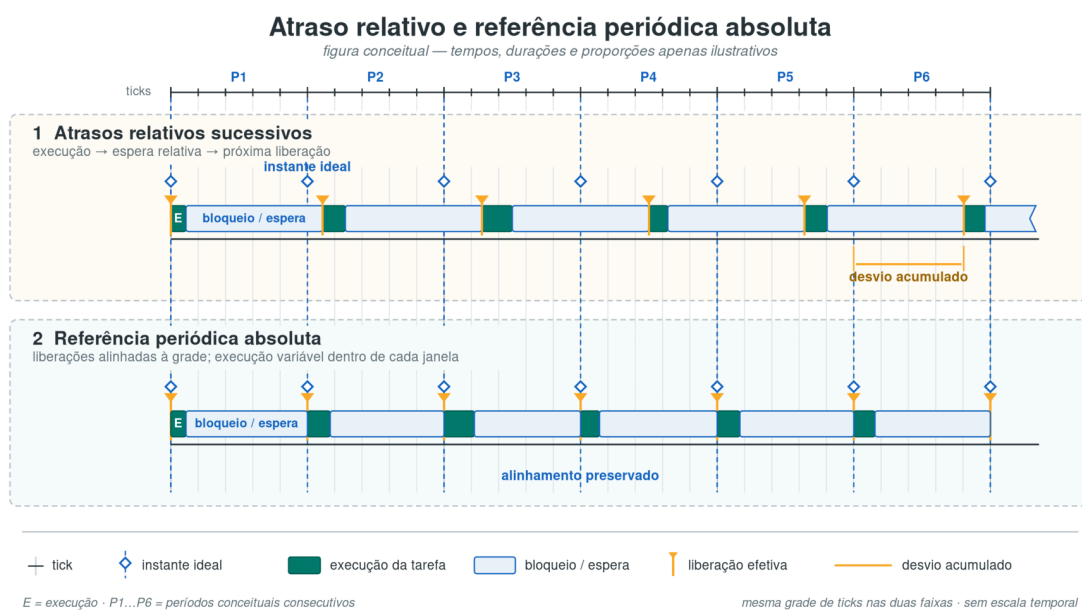
Em um escalonador preemptivo baseado em prioridades, a tarefa pronta de maior prioridade recebe o processador. Quando uma tarefa de prioridade superior se torna pronta, ela pode interromper a execução de uma tarefa menos prioritária. Tarefas de mesma prioridade podem compartilhar o processador por alternância temporal quando essa opção está habilitada. A escolha das prioridades deve refletir requisitos de resposta e dependências, pois atribuir prioridade elevada a uma atividade que executa continuamente pode impedir o progresso das demais.

O escalonamento de tarefas periódicas é um problema clássico de sistemas de tempo real. Liu e Layland (1973) demonstraram condições de escalonabilidade para conjuntos de tarefas periódicas independentes sob algoritmos de prioridade fixa e dinâmica. As hipóteses desses modelos, como independência entre tarefas e conhecimento de seus piores tempos de execução, não são automaticamente satisfeitas em um *firmware* que compartilha barramentos e mutexes. Por isso, prioridades e períodos precisam ser analisados em conjunto com os bloqueios introduzidos pela sincronização.

Em muitos sistemas operacionais de tempo real, um temporizador gera interrupções periódicas denominadas *ticks*. A cada ocorrência, o núcleo atualiza sua referência de tempo, verifica atrasos e limites de espera e pode liberar tarefas bloqueadas.

A frequência do *tick* determina a resolução temporal dessas operações: uma frequência mais alta oferece granularidade mais fina, mas aumenta a quantidade de interrupções executadas pelo núcleo. Uma frequência mais baixa reduz essa sobrecarga, porém limita a resolução dos atrasos expressos em múltiplos de *ticks*. Para tarefas periódicas, atrasos relativos sucessivos podem acumular variação de fase; mecanismos que utilizam como referência o instante de liberação anterior permitem manter um período mais regular. Essa diferença temporal é representada na Figura 2.

Figura 2 – Diferença conceitual entre atrasos relativos e referência temporal periódica



Fonte: Elaborado pelo autor com base na documentação do FreeRTOS (2026a).

As tarefas concorrentes podem acessar dados ou recursos cuja utilização precisa ser coordenada. Sem sincronização, o resultado pode depender da ordem das operações, produzindo condições de corrida, estados parcialmente atualizados ou transações sobrepostas. A necessidade de proteção depende tanto da atomicidade oferecida pelo processador quanto da sequência lógica que deve permanecer indivisível.

A exclusão mútua permite que somente uma tarefa por vez execute uma região associada a um recurso compartilhado. O objeto de exclusão mútua (*mutex*) possui relação de propriedade: a tarefa que o obtém deve liberá-lo depois de concluir o acesso protegido. Essa característica o diferencia de mecanismos usados apenas para sinalização de eventos. No FreeRTOS, mutexes incluem herança de prioridade, enquanto semáforos binários não oferecem esse mecanismo (FREERTOS, 2026b).

O tempo durante o qual um *mutex* permanece retido afeta a latência das tarefas concorrentes. A região protegida deve conter somente as operações que precisam ser serializadas, sem atrasos deliberados ou processamento alheio ao recurso. Quando uma operação de entrada e saída requer várias etapas para preservar a integridade

de uma transação, liberar a proteção entre essas etapas pode permitir interferência de outra tarefa, ainda que reduza aparentemente o tempo de bloqueio.

O uso de vários recursos protegidos também pode criar espera circular. Se duas tarefas obtêm mutexes em ordens diferentes, cada uma pode reter um recurso enquanto aguarda o outro. Uma ordem global de aquisição, a redução do aninhamento e o tratamento explícito de limites de espera são medidas de projeto para evitar ou detectar esse bloqueio.

Nesse contexto, o qualificador `volatile` não constitui um mecanismo de sincronização. Na linguagem C, ele obriga o compilador a preservar acessos observáveis ao objeto segundo as regras da linguagem, sendo pertinente quando o valor pode mudar por uma interrupção ou por *hardware* mapeado em memória. Ele não torna uma sequência de leitura, modificação e escrita indivisível, não estabelece propriedade de um recurso e não substitui mutexes, regiões críticas ou primitivas atômicas.

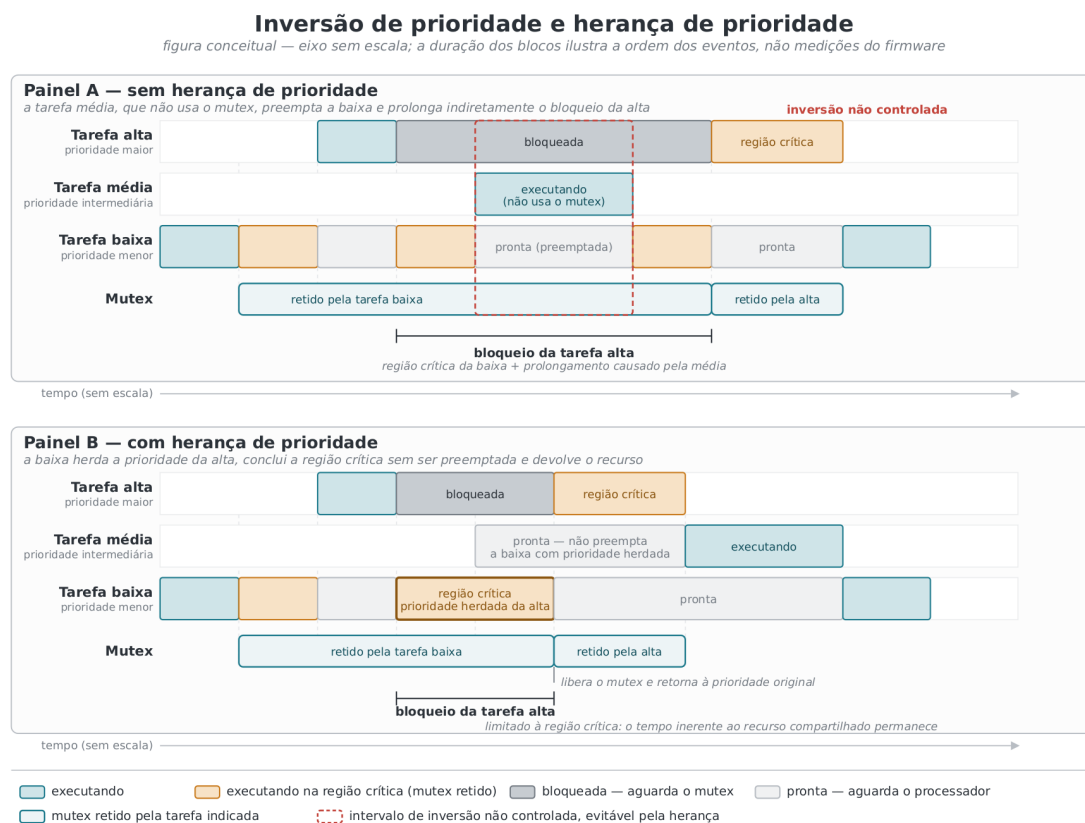
Mesmo quando uma leitura ou escrita alinhada cabe em uma instrução do processador, uma operação composta pode sofrer interferência entre suas etapas. Além disso, a correção depende da ordenação e da visibilidade exigidas entre tarefas e interrupções. Portanto, a escolha da primitiva deve decorrer do invariante a proteger, e não apenas do tamanho da variável compartilhada.

Entre os efeitos desse compartilhamento, a inversão de prioridade ocorre quando uma tarefa de alta prioridade precisa aguardar um recurso retido por uma tarefa de prioridade inferior. O bloqueio pelo tempo necessário à conclusão da região crítica é inerente ao compartilhamento do recurso. O problema torna-se mais grave quando tarefas de prioridade intermediária, que não utilizam esse recurso, preemptam a tarefa que o retém e prolongam indiretamente a espera da tarefa prioritária.

No protocolo de herança de prioridade, a tarefa que retém o recurso recebe temporariamente a maior prioridade entre as tarefas bloqueadas por ela. Isso permite que conclua a região crítica e libere o recurso sem ser sucessivamente preemptada por tarefas intermediárias. Após a liberação, sua prioridade retorna ao valor anterior. Sha, Rajkumar e Lehoczky (1990) formalizaram protocolos destinados a limitar a inversão de prioridade em sistemas de tempo real.

A herança reduz a inversão não controlada, mas não elimina a necessidade de limitar regiões críticas, definir uma ordem de aquisição e analisar bloqueios encadeados. Também não deve ser presumida para qualquer primitiva de sincronização: sua presença depende do tipo de objeto e da implementação do sistema operacional. A Figura 3 compara o encadeamento sem herança com a elevação temporária da prioridade da tarefa que detém o *mutex*.

Figura 3 – Inversão de prioridade e efeito da herança sobre o bloqueio



Fonte: Elaborado pelo autor com base em Sha, Rajkumar e Lehoczky (1990).

Além da sincronização, o progresso das tarefas pode ser supervisionado. Um temporizador de supervisão, ou *watchdog*, detecta situações em que uma parte do sistema deixa de progredir dentro de um intervalo estabelecido. No ESP-IDF, o *watchdog* de interrupções verifica períodos prolongados em que as interrupções e a troca de tarefas não conseguem ocorrer, enquanto o *watchdog* de tarefas pode monitorar tarefas inscritas e a execução da tarefa ociosa. A ausência prolongada de execução da tarefa ociosa é um indício de que alguma tarefa não está cedendo o processador (ESPRESSIF SYSTEMS, 2025d).

O disparo de um *watchdog* identifica uma violação temporal, mas não determina sozinho sua causa. Regiões críticas extensas, interrupções desabilitadas, espera ocupada, processamento excessivo e operações longas sobre memória não volátil podem produzir sintomas semelhantes. O limite de supervisão deve ser compatível com o maior intervalo legítimo de execução, sem ser ampliado apenas para ocultar tarefas que deixam de bloquear ou ceder o processador.

A carga da unidade central de processamento (Central Processing Unit — CPU) pode ser estimada pelo tempo durante o qual as tarefas permanecem em execução. O FreeRTOS pode acumular estatísticas de tempo de execução por tarefa e apresentá-las como valores absolutos e percentuais, desde que seja configurada uma base de tempo

com resolução adequada (FREERTOS, 2026c). Em uma plataforma de núcleo único, o tempo atribuído à tarefa ociosa também permite estimar a parcela não utilizada do processador.

Essas métricas dependem da janela de observação e do estado funcional da aplicação. Uma medição realizada com telas estáticas ou sensores inativos não representa necessariamente o pior caso. A coleta de estatísticas também introduz custo de instrumentação, e o percentual de CPU não substitui a verificação de latências e prazos das tarefas críticas.

Uma estimativa baseada na execução da tarefa ociosa pode indicar a folga global do processador em uma janela, desde que exista uma referência obtida sob condições equivalentes. Ela não atribui custo a cada tarefa, não revela diretamente bloqueios em barramentos e pode variar com interrupções, modos funcionais e instrumentação. O *idle hook* deve executar rapidamente e não pode conter espera bloqueante, pois é chamado no contexto da tarefa ociosa (FREERTOS, 2026c).

2.3 INTERFACES DE COMUNICAÇÃO

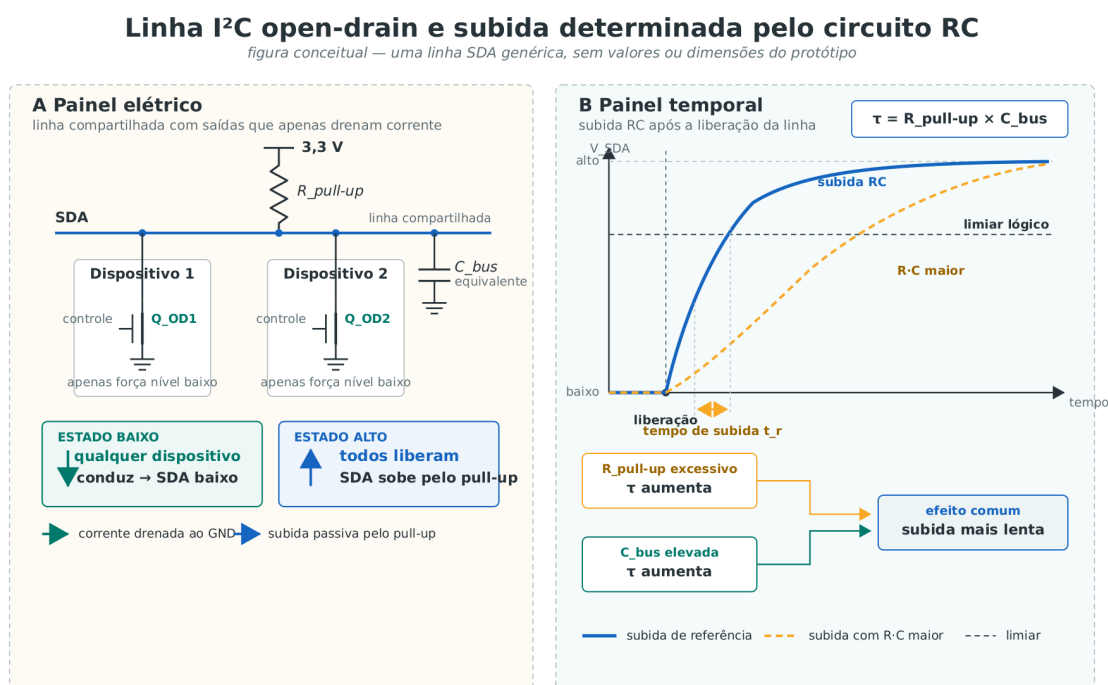
Interfaces seriais transferem dados por um número reduzido de condutores, em contraste com interfaces paralelas que transportam vários bits simultaneamente. A quantidade de sinais, o método de endereçamento, a forma de sincronização e os mecanismos de confirmação variam entre os protocolos. Em uma plataforma com sensores e interface gráfica, a escolha de cada interface afeta a ocupação de pinos, a taxa de transferência, a complexidade elétrica e a coordenação do acesso por *software*.

Entre as interfaces compartilhadas da plataforma, o barramento Inter-Integrated Circuit (I²C) utiliza duas linhas: dados seriais (Serial Data — SDA) e clock serial (Serial Clock — SCL). As linhas empregam saídas de dreno aberto ou coletor aberto e resistores de *pull-up*. Os dispositivos conectados podem forçar uma linha ao nível baixo, mas o nível alto resulta da liberação coletiva da linha e da ação dos resistores. Essa característica permite compartilhar os mesmos condutores sem que duas saídas ativas imponham níveis lógicos opostos (NXP SEMICONDUCTORS, 2021).

A comunicação é organizada em transações iniciadas por uma condição de START e encerradas por uma condição de STOP ou por um START repetido. Após o endereço e a indicação de leitura ou escrita, os dados são transmitidos em grupos de oito bits. Cada byte é seguido por um bit de reconhecimento: ACK indica que o receptor reconheceu a transferência, enquanto NACK pode sinalizar ausência do dispositivo, impossibilidade de receber mais dados ou término de uma leitura, conforme a etapa da transação.

Vários dispositivos podem compartilhar SDA e SCL, desde que seus endereços não entrem em conflito e que as características elétricas permaneçam dentro dos limites do barramento. O tempo de subida depende da resistência de *pull-up* e da capacitância total do barramento, formada por dispositivos, trilhas, fios e conectores. O aumento da capacitância torna as transições para o nível alto mais lentas e pode limitar a frequência utilizável, sendo necessário dimensionar o resistor de *pull-up* de modo a atender simultaneamente ao tempo máximo de subida e à corrente que os dispositivos devem drenar quando a linha está em nível baixo. A Figura 4 relaciona o comportamento de dreno aberto à subida determinada pela rede resistivo-capacitiva.

Figura 4 – Linha I²C em dreno aberto e subida determinada pela rede de *pull-up* e pela capacitância equivalente



Fonte: Elaborado pelo autor com base em NXP Semiconductors (2021).

O protocolo também prevê arbitragem para situações em que mais de um controlador pode iniciar transferências. Alguns dispositivos podem prolongar o nível baixo de SCL para adiar a continuação da transação, recurso denominado *clock stretching*. O suporte efetivo a esses recursos depende dos controladores e dispositivos empregados.

O compartilhamento físico não fornece exclusão mútua entre tarefas de *software*. Quando diferentes tarefas iniciam transações pelo mesmo controlador, o *driver* ou a aplicação deve preservar a atomicidade de cada operação e impedir que sequências pertencentes a dispositivos distintos sejam intercaladas. Erros como NACK, limite de espera e barramento ocupado precisam ser tratados de acordo com o estado reportado pelo controlador.

A especificação define, entre outros, o modo padrão de até 100 kbit/s e o modo rápido de até 400 kbit/s (NXP SEMICONDUCTORS, 2021). Operar a 100 kbit/s em uma integração e a 400 kbit/s em um teste isolado constitui uma decisão de configuração condicionada pelos dispositivos, pela interligação e pelas margens temporais; não implica que uma das frequências seja universalmente correta. Após falhas, a recuperação pode exigir encerrar ou reinicializar a transação e o controlador, ou liberar uma linha retida segundo as capacidades do *hardware*. A estratégia deve evitar que uma tentativa local de recuperação interfira em outra transação válida no mesmo barramento.

Para as transferências do display, a Serial Peripheral Interface (SPI) é uma interface serial síncrona na qual um controlador fornece o sinal de clock e seleciona o dispositivo participante da transação. Na configuração convencional, são utilizados o clock serial (Serial Clock — SCLK), a linha de dados do controlador para o periférico (Master Out, Slave In — MOSI), a linha de retorno (Master In, Slave Out — MISO) e um sinal de seleção, usualmente denominado Chip Select (CS), para cada dispositivo.

Como MOSI e MISO são linhas distintas, a transferência pode ocorrer simultaneamente nas duas direções. Aplicações que necessitam apenas de escrita ou apenas de leitura podem omitir uma das linhas. Diferentemente do I²C, a forma convencional de SPI não inclui endereçamento no barramento nem um bit padronizado de reconhecimento a cada byte. O significado dos bits transferidos, a organização de comandos e a indicação de erros são definidos pelo protocolo do dispositivo.

A relação entre a polaridade do clock e a borda usada para lançar ou amostrar os dados determina o modo SPI. Controlador e periférico precisam utilizar combinações compatíveis de polaridade e fase; uma configuração incorreta pode deslocar a amostragem e corromper os dados. Também devem ser respeitados os tempos de ativação do CS, propagação e preparação especificados para o periférico.

Dispositivos podem compartilhar SCLK, MOSI e MISO, permanecendo inativos enquanto seus sinais CS não estiverem selecionados. Cada dispositivo adicional normalmente exige uma linha CS própria. O controlador precisa impedir a sobreposição de transações e aplicar a configuração de clock e modo correspondente ao dispositivo selecionado. No *driver* mestre do ESP-IDF, uma transação compreende a seleção por CS, a transferência e a liberação do dispositivo; o *driver* também pode utilizar acesso direto à memória para transferir blocos sem movimentar cada byte diretamente pela CPU (ESPRESSIF SYSTEMS, 2025e).

Já o 1-Wire é um protocolo serial bidirecional que utiliza uma linha de dados, além da referência de terra, para comunicação entre um controlador e um ou mais dispositivos. A linha é compartilhada e mantida em nível alto por um resistor de *pull-up*; controlador e dispositivos sinalizam ao forçá-la temporariamente ao nível baixo. Alguns dispositivos podem obter energia da própria linha quando ela permanece em nível alto,

modalidade denominada alimentação parasita, enquanto outros admitem alimentação externa (ANALOG DEVICES, 2008).

Uma comunicação começa com um pulso de reinicialização produzido pelo controlador. Os dispositivos presentes respondem com um pulso de presença, após o qual são transmitidos comandos de endereçamento e de função. Cada dispositivo possui um identificador de 64 bits, que permite selecionar uma unidade específica, selecionar todas as unidades ou executar um procedimento de busca no barramento (ANALOG DEVICES, 2019).

Os bits são representados por intervalos temporais durante os quais o controlador e o dispositivo alteram ou amostram o nível da linha. Há intervalos distintos para escrita de zero, escrita de um e leitura, além de tempos mínimos de recuperação. Como a interpretação depende da duração dos pulsos e do instante de amostragem, atrasos introduzidos pelo *software* ou por interrupções podem violar os limites do protocolo. No DS18B20, por exemplo, cada intervalo transporta um bit, e o controlador inicia todos os sinais, exceto o pulso de presença (ANALOG DEVICES, 2019).

A redução do número de condutores ocorre em troca de menor taxa de transferência e de requisitos temporais estritos. A capacitância da linha, o valor do resistor de *pull-up*, a topologia da conexão e a quantidade de dispositivos influenciam os tempos de subida e a margem disponível para amostragem.

O RMT complementa esse protocolo sem substituí-lo. O Remote Control Transceiver (RMT) é um periférico para transmissão e recepção de sinais definidos por níveis lógicos e durações. Embora tenha sido concebido para sinais de controle remoto por infravermelho, seu formato permite representar outros protocolos baseados em pulsos. Cada símbolo associa dois níveis a seus respectivos intervalos de duração, e canais independentes podem ser configurados para transmissão ou recepção (ESPRESSIF SYSTEMS, 2025f).

O RMT não define endereços, comandos ou regras de um protocolo de comunicação. Ele fornece os recursos temporais usados para produzir ou medir a forma de onda. Um codificador ou *driver* de nível superior continua responsável por converter comandos e dados nos símbolos correspondentes e por interpretar os pulsos recebidos.

Ao transferir a geração e a medição dos pulsos para um periférico, reduz-se a dependência de laços de *software* executados com temporização precisa. Essa separação é útil quando o processador também atende tarefas e interrupções que poderiam introduzir variação nos instantes de alteração ou amostragem de uma entrada ou saída digital.

2.4 INTERFACE GRÁFICA, TEMPORIZAÇÃO, ALIMENTAÇÃO E PERSISTÊNCIA

Na apresentação gráfica, displays de transistores de filme fino (Thin-Film Transistor — TFT) empregam uma matriz ativa para controlar os elementos da imagem. Em módulos embarcados, um controlador integrado recebe comandos e dados de pixels, mantém a informação na memória do display e atualiza a matriz. A aplicação não precisa transmitir continuamente o quadro completo para preservar uma imagem estática, mas deve enviar os pixels correspondentes às regiões que pretende modificar.

No formato RGB565, cada pixel é representado por 16 bits: cinco para vermelho, seis para verde e cinco para azul. Essa codificação oferece 2^{16} , ou 65.536, combinações possíveis e ocupa dois bytes por pixel. O canal verde recebe um bit adicional porque a sensibilidade visual não é igual para as três componentes. Em comparação com representações de 24 ou 32 bits, o RGB565 reduz a memória e o volume transferido, com menor resolução de cor (GALAXYCORE, 2022).

O tamanho de um quadro sem compressão é dado pelo produto entre largura, altura e quantidade de bytes por pixel. Assim, a memória exigida cresce linearmente com a área da imagem e com a profundidade de cor. Em sistemas nos quais o quadro é enviado por uma interface serial, esses mesmos fatores também afetam o tempo mínimo de transferência.

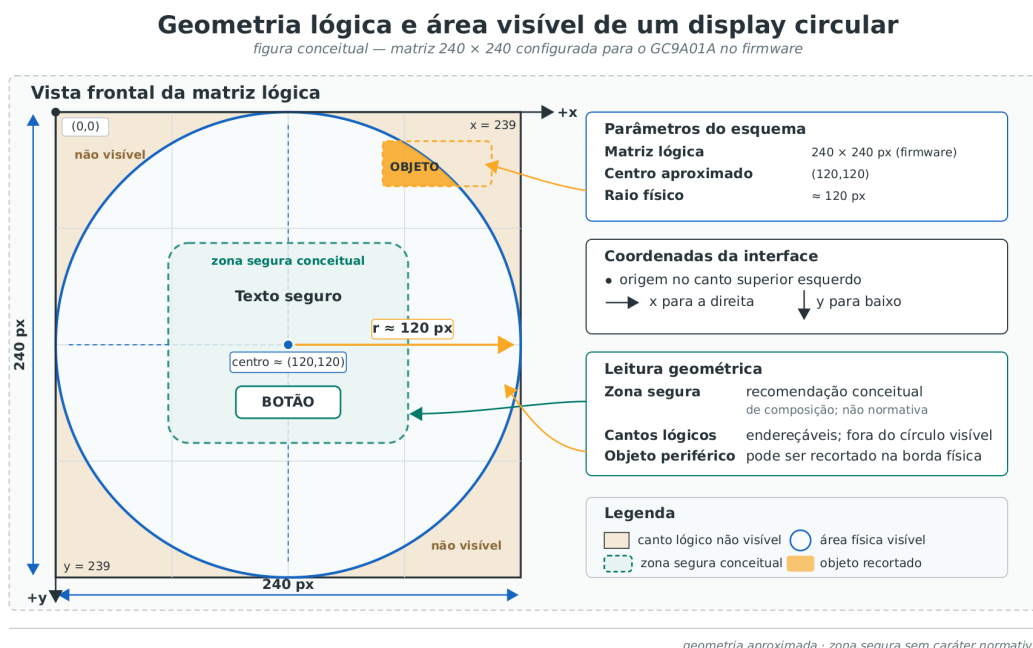
Quando o painel possui formato circular, ele ainda pode ser controlado por uma matriz de endereçamento retangular que contém pixels não visíveis nos cantos. A aplicação continua trabalhando com coordenadas cartesianas, mas somente os pontos internos à área circular aparecem ao usuário. Para um display de raio r e centro (x_c, y_c) , um ponto (x, y) pertence à região visível quando

$$(x - x_c)^2 + (y - y_c)^2 \leq r^2.$$

Essa geometria reduz a largura disponível nas partes superior e inferior da tela. Elementos adequados ao centro podem ser recortados quando deslocados para essas regiões, ainda que suas coordenadas pertençam ao retângulo lógico. O projeto da interface deve considerar a extensão completa de textos, imagens e áreas sensíveis ao toque, e não apenas a posição de seus centros.

O recorte circular também influencia a organização visual. Indicadores periféricos podem acompanhar a borda, enquanto informações que exigem maior largura tendem a ocupar a faixa central. A transformação entre coordenadas de toque e objetos gráficos deve manter a mesma orientação, origem e eventual rotação adotadas pelo controlador do display. A Figura 5 explicita a diferença entre a matriz lógica quadrada, a região circular visível e a zona central conceitualmente mais segura.

Figura 5 – Geometria de uma área circular inscrita em uma matriz lógica quadrada



Fonte: Elaborado pelo autor.

Sobre essa base geométrica, bibliotecas gráficas embarcadas, como a Light and Versatile Graphics Library (LVGL), representam a interface por objetos organizados hierarquicamente. Objetos podem conter outros objetos, herdar propriedades e responder a eventos. Essa organização separa a descrição da interface do mecanismo específico usado para transferir pixels ao display (LVGL, 2023a).

Quando uma propriedade visual muda, a biblioteca marca como inválidas as regiões afetadas. Durante o ciclo de atualização, essas regiões são combinadas quando conveniente, redesenhadas em um *buffer* e entregues ao *driver* por uma função de envio. Dessa forma, uma alteração localizada pode ser transmitida sem reconstruir e enviar necessariamente toda a tela (LVGL, 2023b).

O *buffer* de desenho pode corresponder ao quadro completo ou a uma fração dele. Um *buffer* parcial reduz a memória reservada, mas pode exigir várias etapas de desenho e transferência para concluir uma atualização. Buffers maiores diminuem a fragmentação da operação, enquanto dois *buffers* permitem que o processamento de uma região se sobreponha à transferência de outra quando o *hardware* e o *driver* oferecem esse suporte. A escolha depende da memória disponível, da taxa da interface e da complexidade da cena.

A atualização parcial reduz trabalho quando poucas regiões mudam, mas seu efeito depende do conteúdo. Animações extensas, rolagem e objetos translúcidos podem invalidar áreas maiores e elevar o custo de composição. A taxa de atualização observada resulta da combinação entre desenho, movimentação de memória, transferência ao controlador e execução das demais tarefas.

Os recursos usados pela interface também afetam a memória. Imagens armazenadas sem compressão consomem memória proporcionalmente às dimensões e à profundidade de cor. Uma imagem RGB565 de largura L e altura A , sem canal adicional, requer $2LA$ bytes. Metadados, alinhamento e máscaras de transparência podem acrescentar espaço. Formatos comprimidos reduzem o armazenamento, mas exigem decodificação e podem aumentar o tempo de processamento durante a exibição.

Fontes digitais armazenam a representação gráfica dos caracteres, denominada glifo, além de métricas de posicionamento. O consumo depende da quantidade de caracteres incluídos, do tamanho, da profundidade de bits e de informações como *kerneling*. Incluir todos os caracteres de uma família em vários tamanhos pode ocupar mais memória do que incluir apenas os conjuntos efetivamente utilizados.

Em microcontroladores, imagens e fontes constantes podem permanecer na memória de programa, enquanto *buffers*, estados dos objetos e dados temporários ocupam memória de acesso aleatório. Essa distinção deve ser observada ao avaliar o custo gráfico: reduzir o armazenamento de recursos não implica necessariamente reduzir o *buffer* de desenho, e vice-versa.

A interação pode ser fornecida por controladores de toque capacitivo, que estimam alterações de capacitância associadas à aproximação ou ao contato do dedo e entregam eventos ou coordenadas ao processador. O mapeamento para a interface depende da orientação, da rotação e da origem adotadas pelo display; por isso, o controlador de toque e a biblioteca gráfica precisam compartilhar a mesma transformação geométrica.

No modo de consulta periódica (*polling*), o processador verifica o controlador em intervalos definidos, mesmo quando não há novo toque. Uma linha de interrupção pode sinalizar a disponibilidade de um evento e reduzir consultas desnecessárias, mas ainda requer leitura posterior dos dados e tratamento de repiques ou eventos acumulados conforme o controlador. A escolha depende da interface elétrica, da latência desejada, da taxa de consultas e das capacidades do componente; interrupção não elimina a necessidade de coordenar o barramento compartilhado.

A base temporal pode permanecer independente da renderização por meio de um relógio de tempo real (Real-Time Clock — RTC), que mantém a contagem de segundos e o calendário sem depender da atualização da interface. O PCF8563 utiliza um cristal de 32,768 kHz, comunica-se por I²C e inclui detector de baixa tensão (NXP SEMICONDUCTORS, 2026). Uma fonte de backup pode preservar a oscilação e os registradores quando a alimentação principal é removida, desde que a placa implemente a comutação adequada.

O erro de frequência costuma ser expresso em partes por milhão (ppm). Se o erro relativo for aproximadamente constante, um desvio de e ppm produz, em primeira aproximação, erro temporal de $e \times 10^{-6}$ segundos por segundo. Temperatura, carga

capacitiva e envelhecimento do cristal alteram essa taxa; ajustar data e hora na interface não corrige automaticamente a frequência do oscilador. O bit de baixa tensão informa que a integridade do calendário pode ter sido comprometida e deve ser verificado antes de tratar a leitura como válida.

A alimentação portátil pode empregar uma célula de íons de lítio ou de polímero de lítio em configuração 1S, que fornece tensão variável ao longo da carga e da descarga. A tensão nominal identifica uma região de operação da química e não o valor constante nos terminais; os limites de carga, descarga e corrente devem ser obtidos para a célula e o circuito de proteção empregados. O carregador controla o processo de carga, enquanto o regulador adapta a tensão da célula aos circuitos do sistema.

Estado de carga (State of Charge — SoC) é a fração da carga remanescente em relação à capacidade considerada. A tensão terminal não determina univocamente o SoC: ela depende também da corrente, temperatura, histórico e relaxação da célula. Durante alimentação por USB ou sob carga dinâmica, a interpretação direta da tensão torna-se ainda mais dependente do circuito e do instante da amostra. Um indicador baseado somente em tensão deve, portanto, ser tratado como estimativa aproximada (ANALOG DEVICES, 2006).

Na contagem de coulombs, a corrente que entra e sai da bateria é integrada ao longo do tempo. Esse método mede fluxo de carga, mas acumula erros de *offset* e exige uma condição inicial ou correções periódicas. Medidores mais completos combinam tensão, corrente, temperatura e modelos da célula. Estimar autonomia apenas pela divisão entre capacidade nominal e corrente média pressupõe carga constante e capacidade integralmente utilizável; trata-se de cálculo simplificado, não de uma curva de descarga medida.

Para monitorar essa fonte, um divisor resistivo permite reduzir a tensão da bateria à faixa admissível do conversor analógico-digital (Analog-to-Digital Converter — ADC). Para resistores R_1 e R_2 , com R_1 entre a fonte e o nó medido e R_2 entre o nó e a referência,

$$V_{\text{ADC}} = V_{\text{bat}} \frac{R_2}{R_1 + R_2}.$$

Resistências menores oferecem menor impedância ao circuito de amostragem, mas drenam corrente continuamente. Resistências maiores reduzem esse consumo e tornam a leitura mais sensível à corrente de entrada, à capacitância de amostragem, a fugas e a interferências. O compromisso depende do ADC, da taxa de leitura e da possibilidade de comutar o divisor. O dimensionamento e a topologia efetivamente montada pertencem ao capítulo de Desenvolvimento.

O ADC SAR do ESP32-C6 apresenta variações e não idealidades que impedem converter contagens em tensão por uma relação ideal universal. A atenuação define

a faixa de entrada e integra a configuração da calibração. No ESP-IDF, o ESP32-C6 oferece o esquema de calibração por ajuste de curva (*curve fitting*), que usa parâmetros do dispositivo para converter a leitura bruta em tensão (ESPRESSIF SYSTEMS, 2025c). A média de amostras pode reduzir ruído aleatório não correlacionado, mas não corrige ganho, *offset* ou não linearidade sistemáticos.

Backlight, reguladores, carregador, sensores e interface consomem energia em regimes diferentes. A corrente quiescente dos circuitos permanece relevante quando as cargas principais estão reduzidas, e uma fonte de backup do RTC deve ser distinguida da bateria principal. A caracterização de autonomia exige declarar modo funcional, brilho, atividade dos sensores, comunicação e condição de carga.

2.5 MÓDULOS EMPREGADOS NA PLATAFORMA

A plataforma foi materializada com uma unidade de processamento, um conjunto de interface com o usuário, um relógio de tempo real e sensores biométricos, ambientais e inerciais. Esses componentes são apresentados em conjunto porque sua relevância para o trabalho decorre menos de sua operação isolada do que da coexistência de interfaces, cadências e formatos de dados distintos na mesma aplicação. O Quadro 1 explicita a função de cada dispositivo e evita que a identificação dos módulos fique dispersa entre os capítulos.

Quadro 1 – Componentes empregados e sua função na plataforma

Compo- nente	Grandeza ou função	Interface	Dado fornecido	Uso no protótipo
XIAO ESP32- C6	Processamento e coordenação	I ² C, SPI, GPIO e ADC	Estados da apli- cação	Execução do ESP- IDF, FreeRTOS e LVGL
GC9A01A	Exibição gráfica	SPI	Píxeis RGB565	Interface circular de 240 × 240 píx- eis
CHSC6X	Toque capaci- tivo	I ² C	Coordenadas e evento de toque	Navegação e co- mandos da inter- face
PCF8563	Relógio de tempo real	I ² C	Data e hora	Manutenção tem- poral com o sis- tema desligado
MAX30102	Fotopletismo- grafia	I ² C	Amostras vermel- has e infravermel- has	Frequência cardíaca e esti- mativa funcional de SpO ₂
DS18B20	Temperatura	1-Wire por RMT	Temperatura digi- tal	Indicação de tem- peratura
LTR390	Luz ambiente e ultravioleta	I ² C	Contagens ALS e UVS	Iluminância e índice UV
MPU-9250	Aceleração triax- ial	I ² C	Aceleração nos três eixos	Detecção de pas- sos
AK8963	Campo mag- nético triaxial	I ² C via <i>by- pass</i>	Campo nos três eixos	Calibração e cál- culo de rumo

Fonte: Elaborado pelo autor com base nas folhas de dados dos componentes.

O XIAO ESP32-C6 concentra o processamento e os serviços compartilhados. O GC9A01A e o CHSC6X formam a saída gráfica e a entrada por toque, enquanto o PCF8563 preserva data e hora. O MAX30102 fornece os canais ópticos, o DS18B20 realiza a conversão digital de temperatura, o LTR390 alterna entre os modos ALS e UVS e o conjunto MPU-9250/AK8963 fornece aceleração e campo magnético. A seleção combina componentes comerciais e uma placa desenvolvida para integrá-los; suas ligações e o procedimento de montagem são tratados na Metodologia.

As folhas de dados definem interfaces, registradores, formatos e condições elétricas, mas não determinam a arquitetura da aplicação. A separação entre o que cada

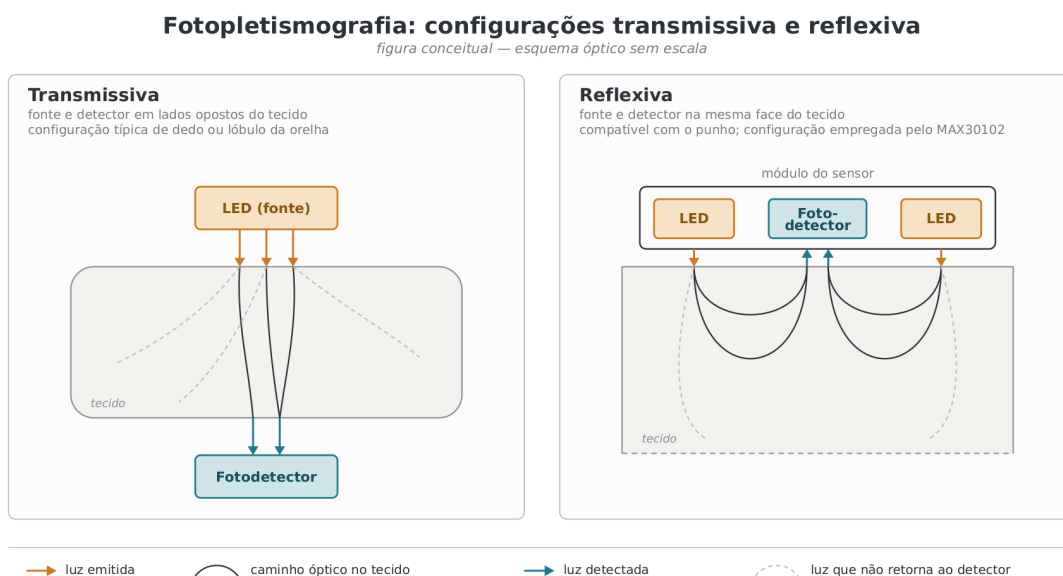
circuito mede, o que os componentes do ESP-IDF transportam e o que o código deste trabalho processa será retomada no Desenvolvimento. Essa distinção é necessária para atribuir corretamente os resultados aos sensores ou aos algoritmos implementados.

2.6 PROCESSAMENTO DOS SINAIS E ALGORITMOS

Entre os sinais processados na plataforma, a fotopletiografia (Photoplethysmography — PPG) é uma técnica óptica que registra variações associadas ao volume sanguíneo no tecido. Uma fonte luminosa ilumina a região medida e um fotodetector converte a luz recebida em sinal elétrico. As mudanças pulsáteis da perfusão alteram periodicamente a absorção e o espalhamento da luz, produzindo uma forma de onda relacionada ao ciclo cardíaco (ALLEN, 2007).

Na configuração transmissiva, fonte e detector ficam em lados opostos do tecido. Na configuração reflexiva, ambos ficam no mesmo lado e o detector recebe parte da luz que retorna após interagir com o tecido. A configuração reflexiva é compatível com regiões nas quais não é prático atravessar completamente o tecido, como o punho, mas sua resposta depende do contato e da geometria óptica (TAMURA et al., 2014). As duas geometrias são comparadas na Figura 6.

Figura 6 – Configurações transmissiva e reflexiva da fotopletiografia



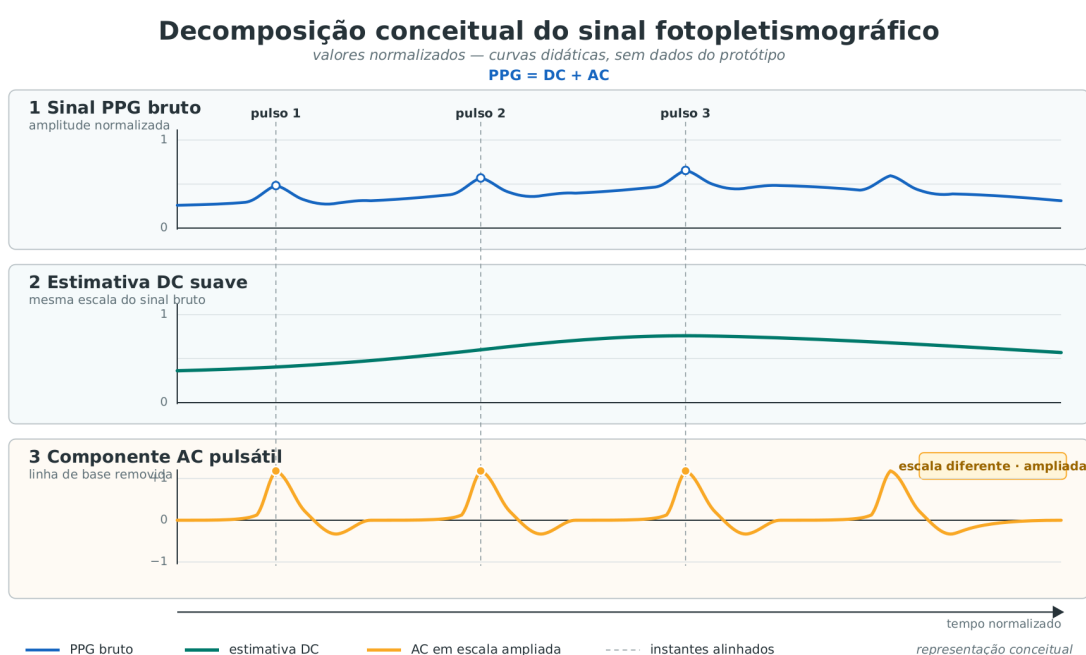
Fonte: Elaborado pelo autor com base em Allen (2007), Tamura et al. (2014) e Analog Devices (2018).

Nesse sinal, a forma de onda PPG pode ser interpretada como a combinação de uma componente de variação lenta, usualmente denominada DC, e uma componente pulsátil, denominada AC. A componente DC reúne a contribuição média dos tecidos,

do sangue não pulsátil, da geometria do contato e da intensidade óptica recebida. A componente AC corresponde às variações sincronizadas com a pulsação arterial.

A amplitude AC é normalmente muito menor que o nível DC. O processamento precisa preservar a variação pulsátil sem saturar a cadeia de aquisição e pode remover tendências lentas para facilitar a detecção dos pulsos. Como alterações de contato ou movimento também modificam o caminho óptico, a separação espectral não garante que toda variação na faixa cardíaca tenha origem fisiológica. A Figura 7 apresenta essa decomposição em valores normalizados e explicita a diferença de escala da componente pulsátil.

Figura 7 – Decomposição conceitual do sinal PPG em componentes DC e AC



Fonte: Elaborado pelo autor com base em Allen (2007).

A partir da componente pulsátil, a frequência cardíaca pode ser estimada pelo intervalo entre pulsos consecutivos identificados no sinal PPG. Se T representa o intervalo médio entre batimentos em segundos, a estimativa em batimentos por minuto é

$$FC = \frac{60}{T}.$$

O procedimento requer condicionamento do sinal e um critério para localizar pulsos válidos. Filtragem, limiar, distância mínima entre picos e rejeição de amplitudes incompatíveis podem reduzir detecções múltiplas e ruído. Esses parâmetros precisam respeitar a faixa fisiológica de interesse e a frequência de amostragem; valores inadequados podem suprimir batimentos reais ou aceitar artefatos como pulsos.

Uma média calculada sobre vários intervalos reduz a variação da leitura, mas responde mais lentamente a mudanças. A frequência cardíaca produzida pelo algoritmo é uma estimativa dependente da qualidade do sinal e do método de detecção, e não uma medição direta da atividade elétrica do coração.

O segundo resultado derivado dos canais ópticos é a estimativa de oxigenação. A oximetria de pulso explora a diferença entre os espectros de absorção da hemoglobina oxigenada e desoxigenada. A aquisição em dois comprimentos de onda permite comparar a modulação pulsátil observada em cada canal. Uma formulação comum utiliza a razão de razões

$$R = \frac{AC_{\text{vermelho}}/DC_{\text{vermelho}}}{AC_{\text{infravermelho}}/DC_{\text{infravermelho}}}.$$

Uma relação empírica converte R em estimativa da saturação periférica de oxigênio (SpO_2). Embora a lei de Beer–Lambert ajude a descrever a absorção em meios homogêneos, o tecido biológico apresenta espalhamento, múltiplos constituintes e caminhos ópticos variáveis. Por isso, os coeficientes da conversão não devem ser deduzidos apenas de um modelo ideal; dependem da geometria do sensor, do processamento e de calibração contra valores de referência (ALLEN, 2007).

A apresentação de uma estimativa de SpO_2 não caracteriza por si só um oxímetro validado. O uso clínico exige avaliação de exatidão e desempenho segundo procedimentos e faixas apropriados. Sem essa validação, o resultado deve permanecer identificado como estimativa experimental.

Essas estimativas permanecem sensíveis à aquisição: o movimento relativo entre sensor e pele altera a pressão de contato e o caminho percorrido pela luz. O artefato resultante pode ocupar a mesma faixa de frequências do sinal pulsátil, o que limita sua remoção apenas por filtragem. Luz ambiente que alcança o fotodetector, contato insuficiente, saturação, baixa perfusão e variações de pigmentação ou anatomia também podem modificar a relação sinal-ruído (TAMURA et al., 2014).

No punho, gestos e marcha produzem componentes periódicas que podem ser confundidas com batimentos. Sensores inerciais podem auxiliar a identificar períodos de movimento, mas não garantem a reconstrução do sinal fisiológico. A avaliação deve incluir o sinal bruto, o sinal processado e condições controladas de repouso e movimento, com registro de perdas de contato ou saturação.

As estimativas dependem ainda da taxa de amostragem, da estabilidade temporal, da corrente dos emissores, do ganho do receptor e dos filtros empregados. Comparações com um instrumento de referência precisam utilizar aquisição temporalmente alinhada e população compatível com o uso pretendido.

No componente empregado, o MAX30102 combina emissores vermelho e infravermelho, fotodetector, eletrônica de condicionamento, conversor e fila de primeiro

a entrar, primeiro a sair (First In, First Out — FIFO), comunicando-se por I²C (ANALOG DEVICES, 2018). A FIFO desacopla parcialmente o instante de amostragem do instante em que o processador atende o barramento, mas possui profundidade finita; se não for drenada a tempo, amostras podem ser sobrescritas ou perdidas conforme a configuração.

Taxa de amostragem, largura de pulso, faixa do conversor e corrente dos LEDs formam compromissos de resolução, relação sinal-ruído, consumo e risco de saturação. Corrente óptica maior não garante sinal utilizável se o receptor saturar, e faixa maior reduz a chance de saturação ao custo da resolução relativa. A ordem dos canais definida pelo componente deve ser confirmada para a placa e o caminho de leitura empregados; uma inversão observada em um módulo específico é evidência de integração, não propriedade universal do MAX30102.

Para a temperatura, sensores digitais integram o elemento sensível, a conversão e a interface de comunicação, fornecendo a medida em uma representação numérica. Essa integração reduz a necessidade de uma cadeia analógica externa, mas a exatidão permanece condicionada às especificações do sensor, à faixa de operação, à montagem e ao equilíbrio térmico com o meio observado.

Resolução e exatidão são características distintas. A resolução indica o menor incremento representável, enquanto a exatidão expressa o limite de erro em relação ao valor verdadeiro nas condições especificadas. Uma leitura com mais casas binárias não é necessariamente mais exata. Em sensores com resolução configurável, maior resolução também pode exigir maior tempo de conversão.

Quando a temperatura é lida por contato, o encapsulamento, a condução pelas conexões, o fluxo de ar e fontes de calor próximas influenciam a resposta. Em um dispositivo vestível, uma medida na região do punho não deve ser tratada automaticamente como temperatura corporal central. Códigos de verificação, como a verificação cíclica de redundância (Cyclic Redundancy Check — CRC), detectam determinadas corrupções na comunicação, mas não corrigem erros térmicos ou de posicionamento (ANALOG DEVICES, 2019).

O DS18B20 realiza conversão digital configurável de 9 a 12 bits e armazena o resultado em um *scratchpad* protegido por CRC. O tempo máximo de conversão cresce com a resolução selecionada, de modo que o controlador deve aguardar a conclusão ou consultá-la antes de interpretar o resultado. Após energização, o registrador de temperatura apresenta 85 °C até que uma conversão seja executada; esse valor deve ser interpretado com o estado da operação, e não aceito isoladamente como temperatura do meio (ANALOG DEVICES, 2019).

O tempo necessário para o encapsulamento aproximar-se da temperatura do objeto constitui resposta térmica e é diferente do tempo de conversão digital. Contato mecânico, massa térmica, condução pelos fios e convecção determinam uma constante

de tempo efetiva da montagem. Filtrar ou aumentar a resolução não acelera esse equilíbrio.

No sensoriamento ambiental, iluminância é a grandeza fotométrica que representa o fluxo luminoso incidente por unidade de área, ponderado pela sensibilidade espectral do sistema visual humano. Sua unidade no Sistema Internacional é o lux, equivalente a lúmen por metro quadrado. Por utilizar ponderação fotópica, a iluminância não corresponde simplesmente à potência óptica total recebida pelo sensor.

Sensores de luz ambiente aproximam essa resposta por meio de sua sensibilidade espectral e de uma conversão entre contagens digitais e lux. Ganho, tempo de integração e faixa de medida afetam resolução e saturação. A relação de conversão fornecida pelo fabricante pressupõe condições definidas e pode ser alterada pela janela óptica, pelo ângulo de incidência e pelo espectro da fonte (LITE-ON TECHNOLOGY, 2024).

No outro canal óptico ambiental, a radiação ultravioleta compreende comprimentos de onda menores que os da luz visível. Para comunicação do risco da exposição solar, o Índice Ultravioleta (Ultraviolet Index — UVI) representa a irradiância ultravioleta ponderada pela ação eritematosa sobre a pele e escalonada por uma constante. Trata-se de uma grandeza adimensional, não de uma leitura direta em lux (WORLD HEALTH ORGANIZATION et al., 2002).

Um sensor com canal ultravioleta produz contagens relacionadas à radiação que alcança sua área sensível. A conversão dessas contagens em UVI depende da resposta espectral, do ganho, do tempo de integração, da calibração e das condições ópticas de montagem. Barreiras transparentes no visível podem atenuar significativamente o ultravioleta (LITE-ON TECHNOLOGY, 2024).

Uma estimativa local produzida por um sensor de baixo custo não é automaticamente equivalente ao UVI meteorológico. Instrumentos de referência aplicam resposta espectral e calibração adequadas, enquanto o valor divulgado para uma região também pode envolver modelos e condições atmosféricas. A interpretação deve conservar essa distinção.

O LTR390 articula essas duas medições ao compartilhar a mesma cadeia de conversão entre um modo de luz ambiente (Ambient Light Sensing — ALS) e um modo de ultravioleta (Ultraviolet Sensing — UVS); os dois canais não são adquiridos simultaneamente. A alternância requer configurar o modo e aguardar uma amostra correspondente à nova condição. Uma leitura obtida durante a estabilização não deve ser misturada ao estado filtrado do outro canal (LITE-ON TECHNOLOGY, 2024).

O ganho multiplica a resposta antes da conversão e favorece sinais pequenos, mas reduz a margem até saturação. Maior tempo de integração e resolução podem melhorar a discriminação de contagens, ao custo de menor taxa de atualização e maior probabilidade de saturar em incidência intensa. Como ALS e UVS representam

grandezas e respostas espectrais distintas, cada canal deve manter seus próprios parâmetros de conversão e estado de filtragem quando as leituras são alternadas.

Para caracterizar movimento, um acelerômetro baseado em sistemas microeletromecânicos (Microelectromechanical Systems — MEMS) mede aceleração específica ao longo de um ou mais eixos. Em repouso sobre uma superfície, sua saída inclui a reação associada ao campo gravitacional; em movimento, combina essa componente com acelerações produzidas pela dinâmica do dispositivo. Sensores triaxiais fornecem as componentes a_x , a_y e a_z em um sistema de coordenadas fixado ao encapsulamento.

A magnitude

$$\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

reduz a dependência da orientação dos eixos para detectar variações de intensidade. Ela não elimina efeitos da posição no corpo, rotação do dispositivo, impacto ou movimento do braço. Faixa de medição, resolução, frequência de amostragem e filtragem também condicionam os eventos que podem ser distinguidos.

Desvios constantes, diferenças de escala entre eixos, ruído e desalinhamento introduzem erros. A calibração pode estimar parte desses termos a partir de orientações ou referências conhecidas, mas sua necessidade e seu procedimento dependem da aplicação.

Faixa de medição e resolução efetiva formam um compromisso: ampliar a faixa evita saturação em impactos, enquanto uma faixa menor atribui mais contagens por unidade de aceleração. A taxa de dados de saída (Output Data Rate — ODR) limita as variações temporais observáveis e deve ser compatível com a filtragem aplicada. Filtragem insuficiente pode preservar ruído e aliasing; filtragem excessiva pode atenuar eventos de passo.

O ADXL345 e o acelerômetro integrado ao MPU-9250 ilustram componentes capazes de fornecer aceleração triaxial, mas não são intercambiáveis sem considerar faixa, resolução, ODR e interface. Um componente avaliado em prototipagem não deve ser apresentado como parte da plataforma final se foi substituído. No MPU-9250, um mesmo encapsulamento reúne o giroscópio e o acelerômetro do núcleo MPU-6500 com o magnetômetro AK8963 em outro *die* (TDK INVENSENSE, 2016).

Para obter orientação, um magnetômetro triaxial mede as componentes do vetor densidade de fluxo magnético segundo três eixos ortogonais. Em condições ideais, a orientação do sensor em relação ao campo terrestre altera essas componentes sem alterar a magnitude do vetor. Na prática, a leitura inclui o campo de interesse, campos produzidos pelo próprio dispositivo, materiais próximos e erros do sensor.

O uso do magnetômetro como bússola requer conhecer a correspondência entre os eixos físicos e o sistema de coordenadas adotado pelo *software*. Trocas de eixo e inversões de sinal alteram o sentido angular calculado. A montagem e a orientação gráfica precisam, portanto, empregar uma convenção comum.

No MPU-9250, o AK8963 pode ser acessado pelo controlador auxiliar interno ou exposto ao barramento principal por um modo de *bypass*. Antes da medição, coeficientes de ajuste de sensibilidade gravados em memória de fusíveis devem ser lidos segundo a sequência definida pelo fabricante e aplicados por eixo. Esse ajuste de fábrica converte a sensibilidade do componente, mas não substitui a calibração das perturbações magnéticas introduzidas pela montagem e pelo ambiente (TDK IN-VESENSE, 2016; AKM, 2013).

A interpretação dessas componentes depende do campo de referência. O campo magnético terrestre pode ser representado por uma componente horizontal e uma componente vertical no local da medição. A direção da componente horizontal define o norte magnético. Ela não coincide necessariamente com o norte geográfico; o ângulo entre ambos é denominado declinação magnética e varia com posição e tempo.

O campo medido no interior de um equipamento pode diferir do campo ambiente. Correntes elétricas, ímãs, alto-falantes, motores, parafusos e estruturas ferromagnéticas produzem perturbações. A proximidade de objetos externos também pode tornar uma calibração previamente obtida inadequada para aquele ambiente.

Com os eixos definidos, o *heading* é o ângulo de orientação no plano horizontal em relação a uma direção de referência. Quando o sensor permanece nivelado e os eixos x e y correspondem ao plano horizontal, ele pode ser calculado por uma função de arco tangente com dois argumentos, por exemplo

$$\psi = \text{atan2}(m_y, m_x).$$

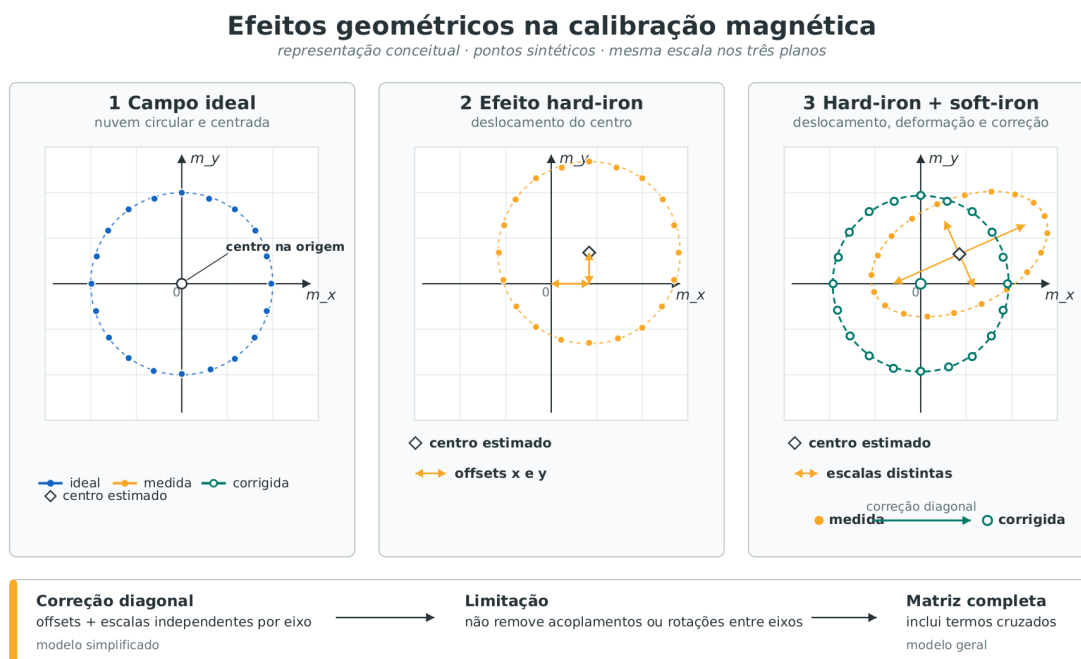
A ordem dos argumentos, os sinais e o deslocamento angular dependem da convenção dos eixos e do sentido positivo adotado. O resultado obtido do campo é referido ao norte magnético; a conversão para norte geográfico exige aplicar a declinação local.

Perturbações aproximadamente constantes no sistema de coordenadas do sensor deslocam o centro da nuvem de medidas e são associadas ao efeito *hard iron*. Materiais que deformam o campo podem alterar escalas e ortogonalidade, produzindo efeito *soft iron*. Em três dimensões, uma calibração geral estima um vetor de deslocamento e uma transformação matricial para aproximar a nuvem medida de uma esfera centrada na origem (RENAUDIN; AFZAL; LACHAPELLE, 2010).

Uma calibração baseada somente nos extremos de cada eixo corrige deslocamentos e escalas diagonais, mas não representa acoplamentos entre eixos. Além

disso, o cálculo bidimensional sem compensação de inclinação pressupõe que o sensor permaneça aproximadamente nivelado. Quando essa condição não é satisfeita, a componente vertical do campo é projetada nos eixos usados no cálculo e introduz erro angular. A Figura 8 compara as nuvens ideal, deslocada, deformada e corrigida, destacando o alcance limitado da correção diagonal.

Figura 8 – Efeitos de *hard iron* e *soft iron* e alcance da correção diagonal



Fonte: Elaborado pelo autor com base em Renaudin, Afzal e Lachapelle (2010).

A compensação de inclinação usa uma estimativa de atitude, normalmente obtida do acelerômetro e eventualmente do giroscópio, para projetar o vetor magnético no plano horizontal antes do cálculo angular. Ela reduz o erro geométrico associado à inclinação, mas introduz dependência da calibração e da dinâmica dos sensores inerciais. No escopo deste trabalho, esse recurso é fundamento para uma extensão futura: o algoritmo final avaliado calcula *heading* bidimensional e pressupõe o dispositivo aproximadamente nivelado.

Finalmente, a marcha produz padrões recorrentes de aceleração associados às fases do passo. Um pedômetro procura identificar esses eventos em meio a movimentos que não correspondem à caminhada. Métodos simples aplicam filtragem, limiares e busca de picos à magnitude da aceleração ou a um eixo selecionado.

Um limiar de amplitude rejeita variações pequenas, enquanto um intervalo mínimo entre detecções evita contar várias oscilações do mesmo passo. Limiares fixos podem falhar quando a intensidade da marcha, o usuário ou a posição do dispositivo mudam. Estratégias adaptativas e o uso de características temporais podem ampliar a faixa de

funcionamento, com maior custo e necessidade de parametrização (BRAJDIC; HARLE, 2013).

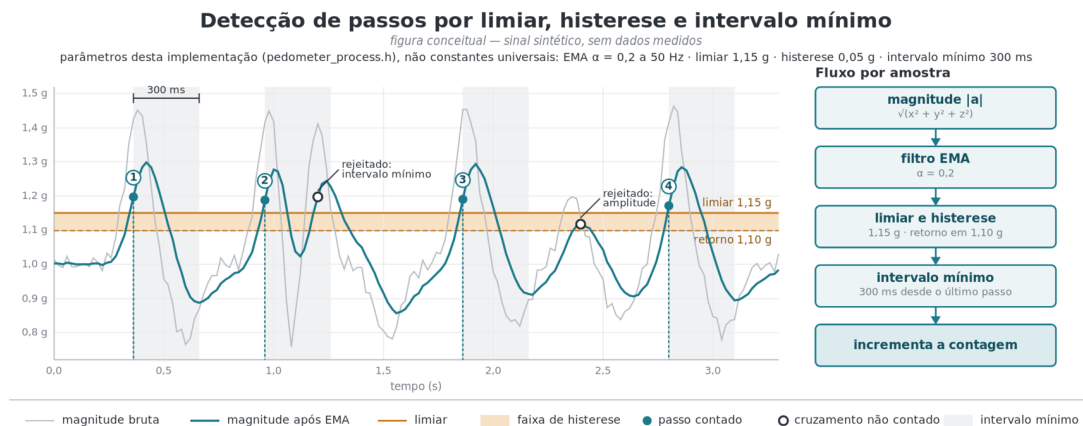
Uma média móvel exponencial (Exponential Moving Average — EMA) de primeira ordem atualiza o estado filtrado por

$$y_k = \alpha x_k + (1 - \alpha)y_{k-1}, \quad 0 < \alpha \leq 1,$$

em que x_k é a amostra corrente. Valores menores de α suavizam mais o sinal e aumentam o atraso. Em uma detecção por limiar, a histerese usa limites diferentes para armar e confirmar o evento, reduzindo comutações repetidas próximas ao limiar. Um período refratário rejeita novos eventos por um intervalo após um passo aceito; ele reduz contagens múltiplas, mas pode suprimir passos reais se for incompatível com a cadência. A Figura 9 relaciona o sinal filtrado, o limiar e o intervalo refratário na aceitação ou rejeição de picos.

Movimentos rítmicos do braço, transporte do dispositivo fora do punho e vibrações podem gerar falsos positivos; passos suaves podem resultar em falsos negativos. A validação exige trajetos ou sequências controladas, contagem de referência e condições declaradas de uso. A comparação entre passos detectados e passos reais pode ser expressa por erro absoluto, erro relativo e distribuição dos erros entre repetições.

Figura 9 – Detecção conceitual de passos por limiar e intervalo refratário



Fonte: Elaborado pelo autor com base em Brajdic e Harle (2013).

2.7 TRABALHOS RELACIONADOS E SÍNTESE TEÓRICA

Os trabalhos relacionados foram selecionados por apresentarem plataformas vestíveis programáveis, aquisição multissensor ou organização destinada a múltiplas aplicações. A comparação considera plataforma, sistema operacional, sensores, modularidade, extensibilidade, tolerância a falhas, forma de avaliação e disponibilidade de código. Os termos não são tratados como equivalentes quando descrevem níveis distin-

tos: modularidade física de sensores, modularidade de *software* e suporte a múltiplas aplicações são registrados separadamente.

Hester et al. (2016) apresentam o Amulet, uma plataforma vestível de baixo consumo destinada à execução simultânea de aplicações. O sistema combina *hardware* próprio, sistema operacional e cadeia de ferramentas, com isolamento de memória e controle de acesso a recursos. A avaliação enfatiza consumo de energia e execução de múltiplas aplicações. Inclusão de sensores por um contrato uniforme de módulos e continuidade diante da ausência de periféricos não são as unidades de análise adotadas nesse estudo.

Baldini et al. (2023) propõem uma plataforma vestível aberta para aquisição e processamento em tempo real de sinais fisiológicos. A organização distribui aquisição, fusão e armazenamento remoto entre blocos conectados e permite integrar diferentes dispositivos de medição. O foco da avaliação está na aquisição sincronizada e no monitoramento remoto, em uma arquitetura de sistema mais ampla que o *firmware* de um único smartwatch.

Van Dijk, Gawehns e Van Leeuwen (2023) apresentam o WEARDA, *software* aberto para registrar dados brutos de acelerômetro, giroscópio, barômetro e sistema de posicionamento em um smartwatch comercial. O trabalho privilegia transparência, reprodutibilidade da coleta e apoio a estudos de atividade humana. Extensibilidade da arquitetura embarcada e recuperação de falhas de barramento ficam fora da unidade de análise, pois o *software* utiliza sensores e interfaces oferecidos pelo sistema operacional do dispositivo. O Quadro 2 sintetiza as diferentes unidades de análise e formas de avaliação desses trabalhos.

Quadro 2 – Comparação entre plataformas vestíveis relacionadas

Trabalho	Ênfase	Organização e extensão	Avaliação reportada
Amulet (HESTER et al., 2016)	Plataforma vestível para múltiplas aplicações	Sistema operacional, isolamento e acesso controlado a recursos	Energia e execução de aplicações
WMP (BALDINI et al., 2023)	Aquisição fisiológica multissensor em tempo real	Blocos de aquisição, fusão e processamento remoto	Aquisição sincronizada e aplicação de monitoramento
WEARDA (VAN DIJK; GAWEHNS; VAN LEEUWEN, 2023)	Coleta aberta de dados brutos em smartwatch comercial	Funções de registro sobre sensores expostos pelo Tizen	Uso em coleta de atividade e reprodutibilidade

Fonte: Elaborado pelo autor com base em Hester et al. (2016), Baldini et al. (2023) e Van Dijk, Gawehns e Van Leeuwen (2023).

Os três trabalhos demonstram que plataformas vestíveis podem ser estudadas além de uma aplicação isolada, porém adotam unidades de análise diferentes. O Amulet concentra-se na coexistência de aplicações e no consumo; a WMP, na aquisição fisiológica conectada; e o WEARDA, na coleta reprodutível com *hardware* comercial. Nenhum deles apresenta conjuntamente, como objeto de avaliação, um contrato uniforme para módulos de sensores heterogêneos, coordenação explícita de um barramento compartilhado, inicialização não fatal e ensaios de operação com sensores ausentes. Essa delimitação estabelece os critérios de comparação do presente trabalho sem pressupor que propriedades não medidas estejam ausentes das plataformas relacionadas.

3 METODOLOGIA

Este capítulo apresenta os recursos e procedimentos empregados para transformar os testes isolados dos componentes em uma plataforma integrada e para produzir as evidências discutidas no Capítulo 5. A descrição concentra-se nas decisões metodológicas: os princípios de operação dos componentes foram reunidos na Fundamentação Teórica, enquanto os detalhes internos do código pertencem ao Desenvolvimento e o protótipo concluído pertence aos Resultados.

O trabalho combinou estudo documental, implementação incremental, análise estática do código e ensaios funcionais. O *firmware* corrente, os arquivos de dados, os scripts de tratamento, as fotografias e o caderno de ensaios foram tratados como fontes complementares de rastreabilidade. Essa distinção permitiu separar ensaio executado, teste piloto, confirmação qualitativa, análise estática e procedimento não realizado.

3.1 MATERIAIS E INFRAESTRUTURA EXPERIMENTAL

A base computacional foi o XIAO ESP32-C6 encaixado no Round Display for XIAO. A placa circular reúne o controlador gráfico GC9A01A, o controlador de toque CHSC6X, o RTC PCF8563, circuito de carga e conector para bateria. Foram integrados o MAX30102, o DS18B20, o LTR390 e o conjunto MPU-9250/AK8963, cujas funções e interfaces constam do Quadro 1. A alimentação foi fornecida por bateria recarregável ou USB, conforme o ensaio, e a tensão da bateria foi amostrada pelo ADC por meio de divisor resistivo externo.

O *firmware* foi desenvolvido em C com ESP-IDF 5.5.1, FreeRTOS em configuração de núcleo único e LVGL v8. O componente oficial do ESP-IDF foi empregado no transporte 1-Wire do DS18B20 por meio do periférico RMT; controladores próprios foram mantidos para os circuitos que exigiam acesso direto aos registradores ou adaptação ao I²C compartilhado. A compilação, a gravação e o monitoramento serial foram realizados pelas ferramentas do ESP-IDF, e scripts versionados foram usados no tratamento dos dados exportados.

As comparações funcionais utilizaram um multímetro digital, um oxímetro de dedo ECH de modelo não identificado, um Amazfit Active 2, um termo-higrômetro MFL Embutir e uma bússola magnética de consumo. Uma lanterna comum e uma lanterna UV genérica foram usadas somente como estímulos para os canais do LTR390; no pedômetro, a referência foi a contagem manual em percurso de 100 m. Os instrumentos de consumo não possuíam certificado de calibração disponível e, por isso, seus resultados não foram tratados como referências metrológicas ou clínicas rastreáveis.

3.2 MÉTODO INCREMENTAL DE DESENVOLVIMENTO

Os requisitos funcionais abrangeram data e hora, navegação por toque, aquisição dos sensores e apresentação dos estados em telas próprias. Como requisitos arquitetais, definiu-se que as funções sensoriais deveriam ser encapsuladas, que o acesso ao I²C deveria ser serializado e que a indisponibilidade de um sensor não poderia interromper deliberadamente a inicialização ou as funções independentes. Foram consideradas as restrições de memória, processamento em núcleo único, display circular e compartilhamento de barramento.

Para cada componente, o ciclo de desenvolvimento iniciou pelo estudo da folha de dados e da interface elétrica. Em seguida, foi implementado ou adaptado um teste isolado para verificar identificação, configuração, aquisição e tratamento. Com esse comportamento estabelecido, o código foi separado entre acesso ao dispositivo, processamento e apresentação, adaptado ao contrato da plataforma e integrado ao *firmware*. Depois de cada integração, foram repetidas verificações das funções existentes.

Quando a regressão revelava conflito de barramento, temporização, memória, alimentação ou interface, o diagnóstico retornava à etapa responsável, e o ciclo era repetido. Essa prática foi informada pela experiência profissional do autor com testes de sistemas embarcados, incluindo testes de fumaça, regressão e estresse, sem caracterizar a adoção de uma metodologia ágil específica. A Figura 10 sintetiza o procedimento efetivamente empregado.

3.3 CONSTRUÇÃO E INTEGRAÇÃO DO PROTÓTIPO

A integração física foi consolidada em uma placa protótipo autoral, e o acondicionamento foi produzido por impressão 3D. O XIAO ESP32-C6 permaneceu encaixado no Round Display, e os módulos externos foram distribuídos conforme o esquema e o leiaute desenvolvidos. O processo incluiu a verificação de continuidade, tensões de alimentação, endereços do I²C e correspondência entre pinos do projeto físico e definições do *firmware*.

O I²C foi compartilhado pelo toque, RTC, MAX30102, LTR390, MPU-9250 e AK8963, em SDA GPIO22 e SCL GPIO23. O GC9A01A utilizou o SPI2 somente para transmissão; o DS18B20 utilizou o GPIO20 com resistor de *pull-up* externo; e o divisor da bateria foi ligado ao GPIO0. O Quadro 3 consolida as conexões correspondentes à versão integrada.

Quadro 3 – Mapa consolidado de interfaces e ligações

Elemento	Interface	Endereço ou sinais	Conexão no ESP32-C6
GC9A01A	SPI2	SCLK, MOSI, CS e D/C	GPIO19, GPIO18, GPIO1 e GPIO21
CHSC6X	I ² C	0x2E e interrupção	GPIO22, GPIO23 e GPIO17
PCF8563	I ² C	0x51	GPIO22 e GPIO23
LTR390	I ² C	0x53	GPIO22 e GPIO23
MAX30102	I ² C	0x57	GPIO22 e GPIO23
MPU-9250	I ² C	0x68	GPIO22 e GPIO23
AK8963	I ² C via <i>bypass</i>	0x0C	Barramento exposto pelo MPU-9250
DS18B20	1-Wire RMT	por DQ e <i>pull-up</i> de 4,7 k Ω	GPIO20
Bateria	ADC	Divisor resistivo 1:2	GPIO0

Fonte: Elaborado pelo autor a partir do *firmware* integrado e do projeto da placa.

A Figura 11 apresenta a mesma montagem de forma gráfica. As imagens da placa concluída e as renderizações do projeto são apresentadas no Capítulo 5, pois documentam o artefato obtido, e não apenas o procedimento de construção.

3.4 PROCEDIMENTOS DE AVALIAÇÃO E RASTREABILIDADE

Cada coleta registrou, quando disponíveis, condições de alimentação, instrumentos, duração, operador, arquivos de dados e procedimento de tratamento. Os dados tabulares e scripts foram preservados no repositório; lacunas de metadados permaneceram indicadas, sem reconstrução retrospectiva. A Figura 12 diferencia conexões físicas das relações de comparação ou observação.

A Tabela 1 substitui a descrição fragmentada de um procedimento por subseção. O número de repetições é informado quando foi controlado; nos demais casos, a evidência é classificada segundo o registro disponível. Essa classificação impede que observações de depuração sejam apresentadas como validação quantitativa.

Tabela 1 – Procedimentos e alcance das evidências produzidas

Objeto	Procedimento e instrumento	Registro e repetições	Alcance
Extensibilidade e acoplamento	Inspeção de arquivos, chamadas e pontos de registro	Um estudo; relatório estático e matriz de dependências	Evidência estrutural
Memória	Ferramentas de tamanho do ESP-IDF	Uma compilação; síntese dos valores obtidos	Ocupação global, não por módulo
Sensores ausentes	Remoção individual e em diferentes arranjos; reinicialização e navegação	Arranjos registrados no caderno e nos registros de inicialização	Teste funcional qualitativo
MAX30102	Coleta PPG com oxímetro e Amazfit como comparadores	Uma sessão; CSV, script e gráficos	Comparação funcional, não clínica
DS18B20	Comparação em um ponto e estímulos térmicos	Uma sessão por condição; caderno e gráfico	Plausibilidade e resposta transitória
LTR390	Obstrução, luz ambiente, lanternas e exposição externa	Sequências funcionais; caderno, tabela e gráficos	Ordenação e saturação; sem referência calibrada
Bússola	Calibração no dispositivo e comparação no plano	Duas orientações; caderno e gráfico	Resposta direcional limitada
Pedômetro	Percurso de 100 m com contagem manual e Amazfit	Ida e volta; tabela e gráfico	Um participante e um ritmo
RTC	Comparação antes e depois de intervalo desligado	Uma execução; fotografias e anotação	Verificação funcional
Interface	Percurso por telas, comandos e retorno	Repetido durante a integração; capturas e caderno	Verificação funcional
Bateria e consumo	Tensão e corrente por cenário com multímetro	Nove cenários; tabela e gráfico	Caracterização descritiva

Fonte: Elaborado pelo autor a partir do caderno de ensaios.

Na avaliação da inicialização degradada, o protótipo foi desligado antes de cada alteração física. Cada sensor externo foi removido individualmente, e o procedimento também foi repetido com sensores ausentes em diferentes arranjos. Após religar o equipamento, foram percorridos a tela principal, os quatro menus e as telas dos módulos que continuavam conectados. Os critérios observados foram a conclusão da inicialização, o funcionamento do toque e do RTC, a indicação de indisponibilidade

na função dependente e a ausência de *panic*, reinicialização espontânea ou disparo do temporizador de vigilância. O procedimento foi qualitativo e não estabeleceu taxa estatística de disponibilidade.

O ensaio do MAX30102 combinou uma comparação funcional das indicações com a análise do sinal exportado. No teste isolado, o dedo permaneceu sobre o sensor por aproximadamente 65 s, enquanto um oxímetro de dedo e um Amazfit Active 2 eram observados simultaneamente. A aquisição interna operou a 100 Hz, e o CSV foi exportado a 50 Hz com amostras alinhadas dos canais vermelho e infravermelho, suas estimativas de linha de base e as componentes pulsáteis filtradas. O transiente inicial de contato foi separado, e a análise quantitativa utilizou o intervalo estável entre 20 e 73 s. Como os três equipamentos não compartilhavam referência eletrônica de tempo, a comparação das indicações foi feita por trechos observados e não por pareamento amostra a amostra.

Para a bússola, a calibração orientada nos 12 setores foi executada antes da comparação. O protótipo e a bússola de consumo permaneceram aproximadamente nivelados no mesmo plano, e duas orientações separadas por cerca de meia-volta foram registradas sem compensação de inclinação. O pedômetro foi avaliado em um percurso de 100 m delimitado com uma ferramenta cartográfica, percorrido na ida e na volta por um participante em ritmo habitual. A contagem manual foi adotada como referência, e o Amazfit Active 2 foi mantido no pulso como comparador secundário. Os erros foram calculados por

$$E_{\text{abs}} = N_{\text{detectado}} - N_{\text{referencia}}, \quad E_{\text{rel}} = \frac{|E_{\text{abs}}|}{N_{\text{referencia}}} \times 100\%.$$

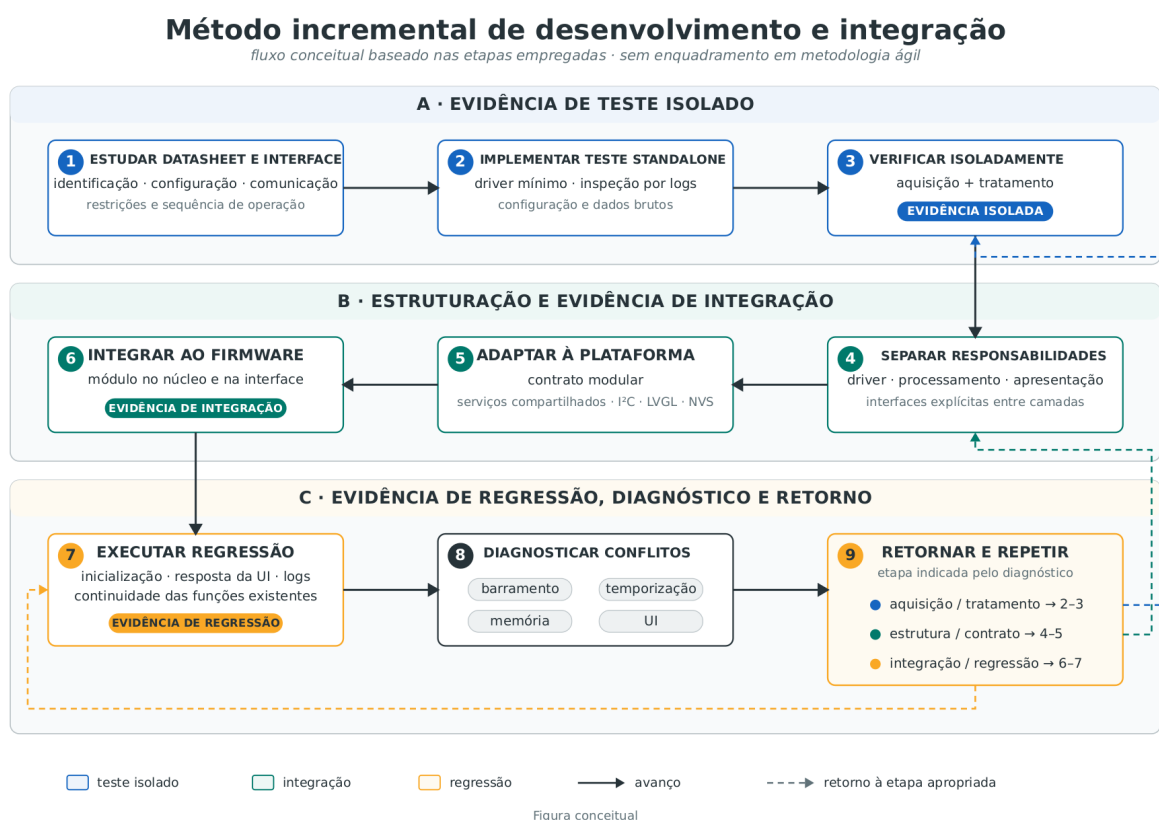
Nas verificações ambientais, o DS18B20 foi inicialmente mantido por cerca de 5 min ao lado do termo-higrômetro e, em ensaios separados, submetido a resfriamento em freezer e aquecimento próximo a uma fonte térmica. Esses transientes foram interrompidos após 2 min e ao atingir 70 °C, respectivamente, e não possuíam referência calibrada. O LTR390 foi exercitado em obstrução, ambiente interno, sombra externa e luz solar direta; os canais ALS e UVS foram avaliados separadamente, incluindo estímulos com lanterna comum e lanterna UV. O RTC foi ajustado na troca de minuto de uma referência horária e comparado após 1 h ligado e após 24 h com a alimentação principal removida.

Para caracterizar o consumo global, o multímetro foi ligado em série com o terminal positivo da bateria. O rádio permaneceu inativo e a iluminação de fundo permaneceu ligada. Cada um dos nove cenários foi selecionado pela interface, com aproximadamente 10 a 20 s para estabilização e observação posterior durante cerca de 30 a 60 s. O procedimento mediu somente a corrente total da plataforma; por isso, diferenças entre telas não foram usadas para atribuir consumo isolado ao processador,

ao sensor, à interface gráfica ou aos conversores. Também não foi realizada curva completa de descarga.

Não foram realizadas campanhas comparáveis de CPU e *heap* por cenário, compilações incrementais por módulo, série controlada de tempo de inicialização, injeção repetida de falhas no I²C ou ensaio prolongado com roteiro completo. O contador do gancho de ociosidade e a leitura de memória livre presentes no *firmware* servem como instrumentação, mas não substituem uma campanha com linha de base, janela fixa e repetições. Essas atividades são tratadas como continuidade do trabalho, e não como resultados desta versão.

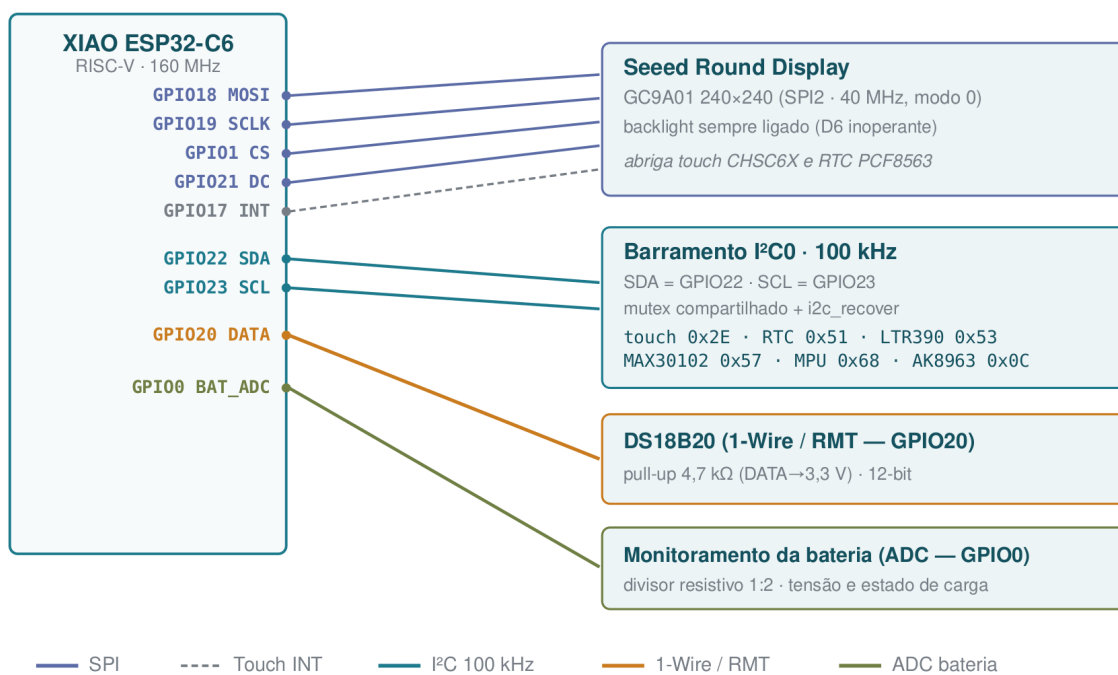
Figura 10 – Método incremental de desenvolvimento e integração



Fonte: Elaborado pelo autor a partir do procedimento empregado no projeto.

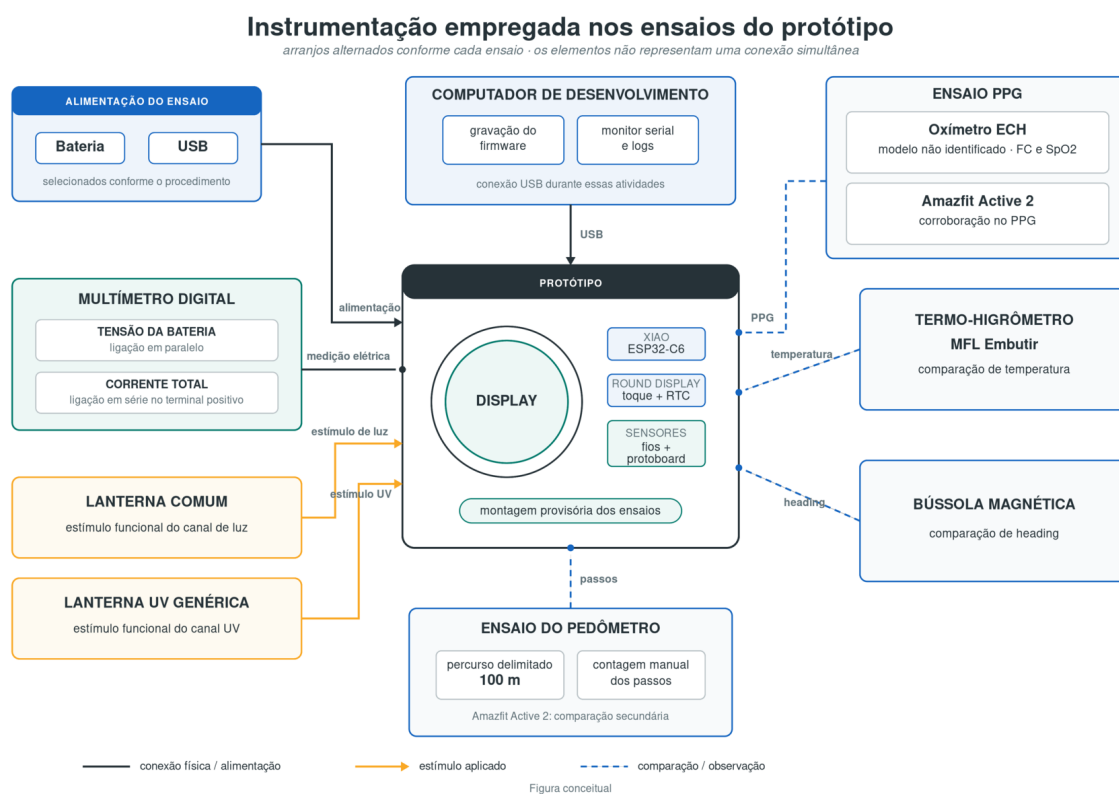
Figura 11 – Mapa de ligações da montagem final

Mapa de ligações — XIAO ESP32-C6 (confirmado no firmware)



Fonte: Elaborado pelo autor a partir do *firmware* integrado e do projeto da placa.

Figura 12 – Instrumentação empregada nos ensaios do protótipo



Fonte: Elaborado pelo autor a partir do caderno de ensaios.

4 DESENVOLVIMENTO E IMPLEMENTAÇÃO

Este capítulo apresenta a implementação da plataforma, desde a organização dos recursos físicos até a integração dos módulos funcionais. A descrição foi conferida diretamente na versão corrente do *firmware* em `Codigos/IDroid/main`; documentos históricos foram usados apenas para contextualizar decisões de desenvolvimento e não como autoridade sobre a versão integrada.

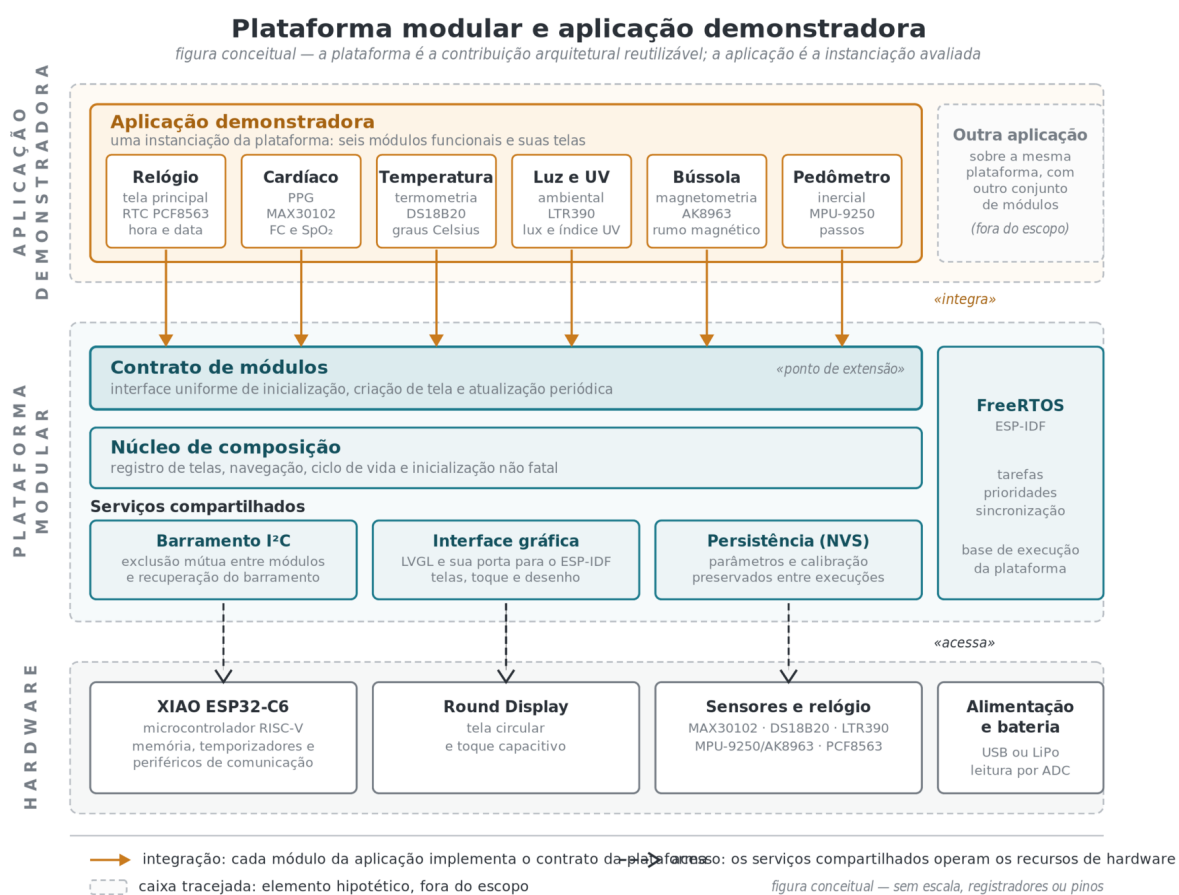
4.1 VISÃO GERAL E PLATAFORMA FÍSICA

A plataforma foi organizada em quatro níveis: *hardware*, *drivers*, serviços compartilhados e módulos funcionais. O *hardware* reúne o XIAO ESP32-C6, o Round Display e os sensores. Os *drivers* encapsulam os protocolos e registradores de cada dispositivo. Os serviços coordenam recursos utilizados por mais de um módulo, como I²C, LVGL e NVS. Os módulos associam aquisição, processamento, estado funcional e tela.

A Figura 13 diferencia a base arquitetural da aplicação que a instancia com os componentes deste trabalho. Essa distinção evita confundir o conjunto específico de sensores com o mecanismo de integração reutilizável.

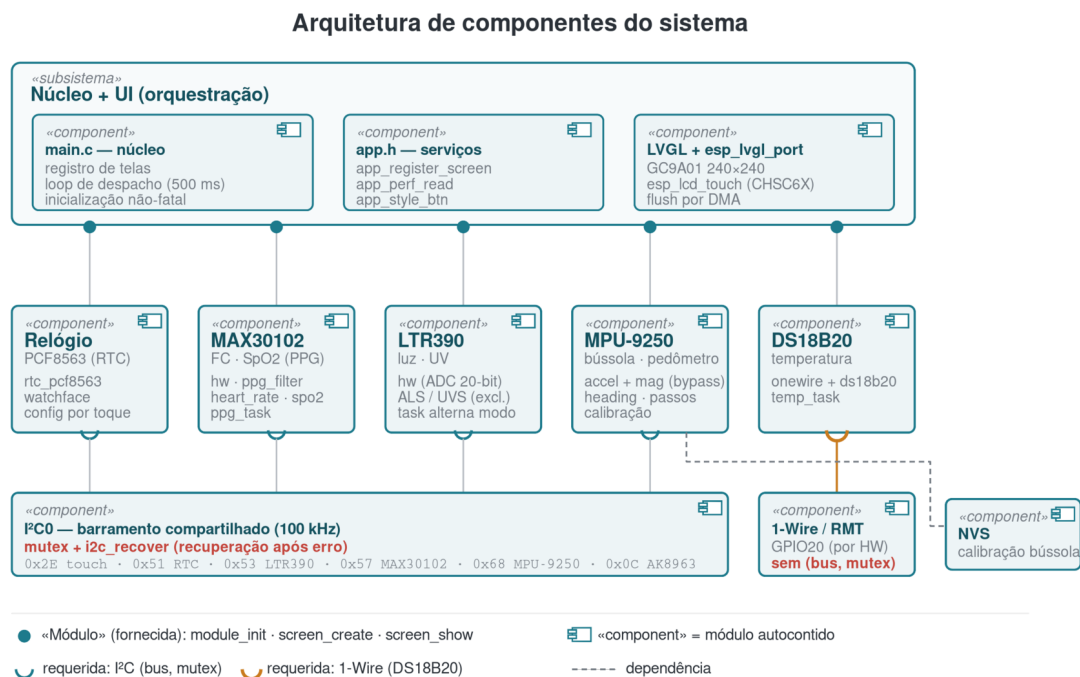
A Figura 14 sintetiza os componentes confirmados e as direções de controle, dados e acesso ao *hardware*.

Figura 13 – Relação entre a plataforma modular e a aplicação demonstradora



Fonte: Elaborado pelo autor.

Figura 14 – Arquitetura de componentes da plataforma implementada



Fonte: Elaborado pelo autor a partir do *firmware* integrado.

O núcleo não executa os algoritmos de cada sensor. Ele inicializa os serviços, solicita a inicialização dos módulos, cria as telas e despacha a atualização correspondente à tela ativa. A aquisição periódica permanece nas tarefas dos módulos, enquanto a LVGL é manipulada segundo o mecanismo de exclusão fornecido por sua porta para o ESP-IDF.

No barramento compartilhado, o I²C foi criado uma única vez pelo núcleo com a API mestre introduzida nas versões recentes do ESP-IDF. O barramento opera a 100 kHz em SDA GPIO22 e SCL GPIO23. Cada *driver* registra seu dispositivo lógico nessa instância, em vez de criar um controlador próprio.

Seis endereços coexistem no barramento: CHSC6X em 0x2E, PCF8563 em 0x51, LTR390 em 0x53, MAX30102 em 0x57, MPU-9250 em 0x68 e AK8963 em 0x0C. O último torna-se acessível externamente quando o MPU-9250 é colocado em modo *bypass*. Embora pertençam ao mesmo encapsulamento funcional, MPU-9250 e AK8963 constituem alvos I²C distintos.

A frequência de 100 kHz foi adotada na integração, substituindo configurações mais altas usadas em testes isolados. A redução aumenta a margem para os tempos de subida na montagem com fios e múltiplas placas. Como todos os dispositivos compartilham o controlador e as linhas, a coordenação por *software* e a recuperação do barramento foram tratadas como serviços sistêmicos.

Para a interface gráfica, o display GC9A01A utiliza o periférico SPI2 a 40 MHz. As linhas de clock e dados são compartilháveis com o slot microSD do Round Display,

enquanto sinais de seleção independentes delimitam as transações. O *firmware* integrado utiliza o controlador de LCD disponibilizado como componente do ecossistema ESP-IDF; o cartão não integra as funções avaliadas.

As transferências gráficas são predominantemente unidirecionais, do ESP32-C6 para o controlador. O uso de DMA pela porta gráfica permite transferir blocos de pixels sem movimentação individual pela CPU. O acesso ao display permanece encapsulado pelo *driver* e pela LVGL, de modo que módulos funcionais criam objetos gráficos sem iniciar transações SPI diretamente.

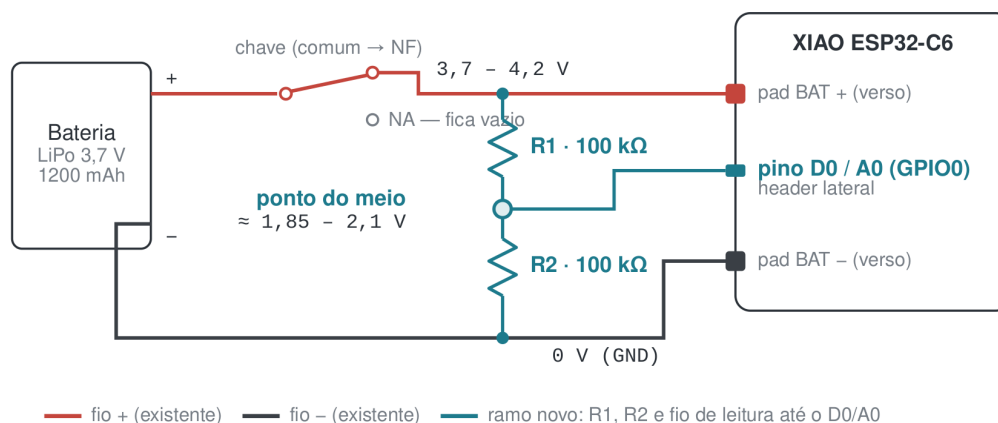
O DS18B20 utiliza uma interface separada no GPIO20, com resistor de *pull-up* externo. Os intervalos do protocolo 1-Wire são produzidos e amostrados por meio do periférico RMT. Essa solução reduz a dependência de atrasos ativos em *software* e evita que a temporização de cada bit dependa diretamente da execução contínua de uma tarefa.

Foram reutilizados os componentes oficiais `onewire_bus` e `ds18b20`. O módulo de temperatura recebe valores já convertidos pela API do componente e não conhece a representação dos símbolos do RMT. A separação demonstra que o contrato funcional não exige que todos os módulos utilizem o mesmo transporte.

Quanto à alimentação, o protótipo pode ser alimentado por USB por meio do XIAO ESP32-C6 ou por bateria conectada ao circuito do Round Display. A versão implementada não inclui gerenciamento avançado de energia, e o backlight permanece ligado durante a operação. Recursos de suspensão profunda, controle de brilho e otimização de autonomia não fazem parte do desenvolvimento descrito.

A leitura da tensão da bateria também integra a versão final. O positivo após a chave alimenta um divisor 1:2 formado por dois resistores de 100 k Ω ; o ponto médio segue para o GPIO0/A0 do XIAO ESP32-C6. O *firmware* aplica calibração do ADC, média de 16 amostras e suavização temporal antes de apresentar o estado estimado. A Figura 15 mostra o circuito efetivamente empregado. Como a estimativa se baseia em tensão, ela não equivale a um contador de carga.

Figura 15 – Divisor resistivo empregado na leitura da tensão da bateria



Fonte: Elaborado pelo autor a partir do circuito implementado e de bateria.c.

Durante a integração, a alimentação do MAX30102 exigiu atenção por causa dos pulsos de corrente dos emissores. Os testes A/B registrados no próprio código associaram a configuração anterior, de aproximadamente 14,2 mA por emissor, a afundamento da linha de 3,3 V e travamento do I²C na placa final. A configuração integrada passou a empregar o valor 0x24, correspondente a aproximadamente 7,2 mA por emissor.

Depois da validação funcional, a integração deixou a protoboard de desenvolvimento e foi transferida para uma placa protótipo autoral, cujo esquema, leiaute, renderizações e registro fotográfico foram preservados. Um invólucro também foi fabricado por impressão 3D para acomodar o conjunto. Esses elementos pertencem à implementação realizada; revisões futuras podem aprimorá-los, mas não correspondem a atividades pendentes de construção.

4.2 ARQUITETURA MODULAR E CONTRATO DE INTEGRAÇÃO

No *software*, o *firmware* foi dividido por responsabilidade. O núcleo contém o ponto de entrada, os registros da aplicação e os serviços comuns. Os *drivers* tratam comunicação e configuração dos circuitos integrados. Cada função apresentada ao usuário reúne os arquivos de seu módulo, seu processamento e sua tela. Recursos gráficos reutilizáveis permanecem em uma camada de interface.

No diretório `main`, o arquivo `main.c` concentra inicialização, menus e despacho; `app.h` expõe os serviços comuns da aplicação; `i2c_recover.c` encapsula a

recuperação do barramento; *relogio*/ contém RTC, configuração, bateria e tela principal; *sensores*/ separa os módulos por dispositivo; e *ui*/ preserva a base gráfica exportada. Dentro dos sensores, os sufixos *_hw*, *_process* e *_screen* materializam, quando aplicável, a separação entre acesso ao dispositivo, tratamento e apresentação.

O aspecto arquitetural relevante é a direção das dependências: os módulos utilizam serviços e *drivers*; o núcleo utiliza suas interfaces públicas; e um módulo funcional não acessa diretamente o estado interno de outro.

O percurso típico do dado também explicita as responsabilidades: a tarefa definida em **_screen.c* solicita a aquisição em **_hw.c*; o dado bruto retorna ao processamento em **_process.c*; o resultado atualiza o estado do módulo; e a função de atualização apresenta esse estado por objetos LVGL. Nem todos os módulos exigem os três arquivos, mas as dependências preservam essa direção conceitual.

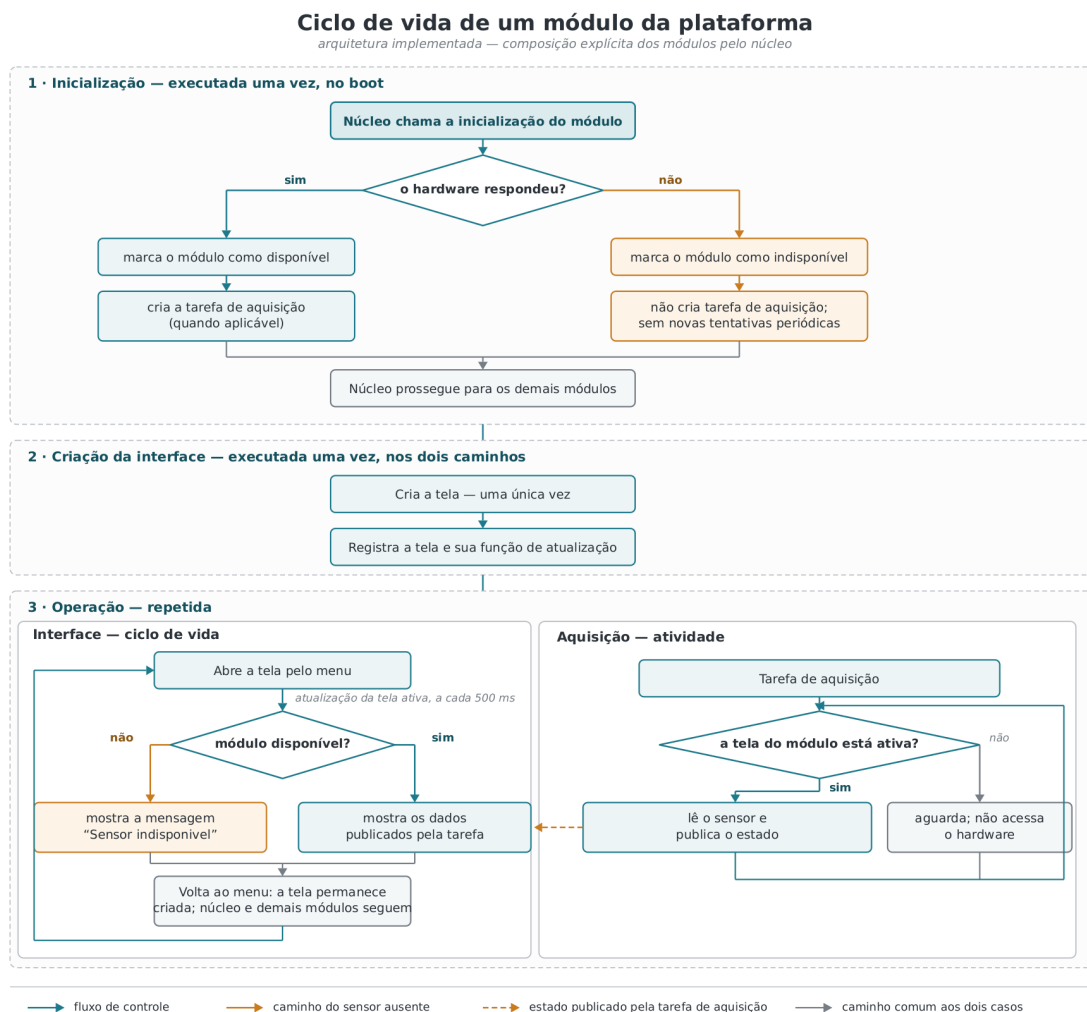
Esse arranjo é coordenado por um contrato que estabelece três etapas comuns: inicializar a função, criar sua tela e solicitar sua exibição. Em forma abstrata, essas operações correspondem a *modulo_init(dependencias)*, *modulo_screen_create(menu_origem)* e *modulo_screen_show()*.

A inicialização prepara o *driver*, o estado e, quando necessário, a tarefa de aquisição. A criação constrói uma única vez os objetos LVGL e registra a função que atualizará a tela. A exibição solicita a navegação para a tela já criada.

A uniformidade refere-se ao ciclo de vida e às responsabilidades observáveis, não à exigência de assinaturas literalmente idênticas. Módulos I²C recebem a referência do barramento e do *mutex*; o DS18B20 cria sua interface 1-Wire sem essas dependências. Essa variação conserva o transporte dentro do módulo sem obrigar o núcleo a simular uma dependência inexistente.

A Figura 16 mostra como o mesmo ciclo preserva uma tela informativa tanto no caminho disponível quanto no caminho de sensor ausente.

Figura 16 – Ciclo de vida dos módulos funcionais



Fonte: Elaborado pelo autor a partir do contrato observado no *firmware*.

Como serviço de integração, o registro de telas associa um objeto LVGL a uma função de atualização. A cada 500 ms, o despachante consulta a tela ativa e chama somente a função registrada para ela. Menus estáticos não precisam de atualização periódica, enquanto telas de sensores convertem o estado publicado pelas tarefas em texto, indicadores ou gráficos.

O núcleo também concentra a criação do barramento I²C, o scanner de endereços, a recuperação após falhas e os estilos compartilhados da interface. O serviço `app_perf_read()` fornece uma estimativa exploratória de ociosidade e o *heap* livre, mas, sem uma série controlada, esses valores não foram usados como caracterização de desempenho.

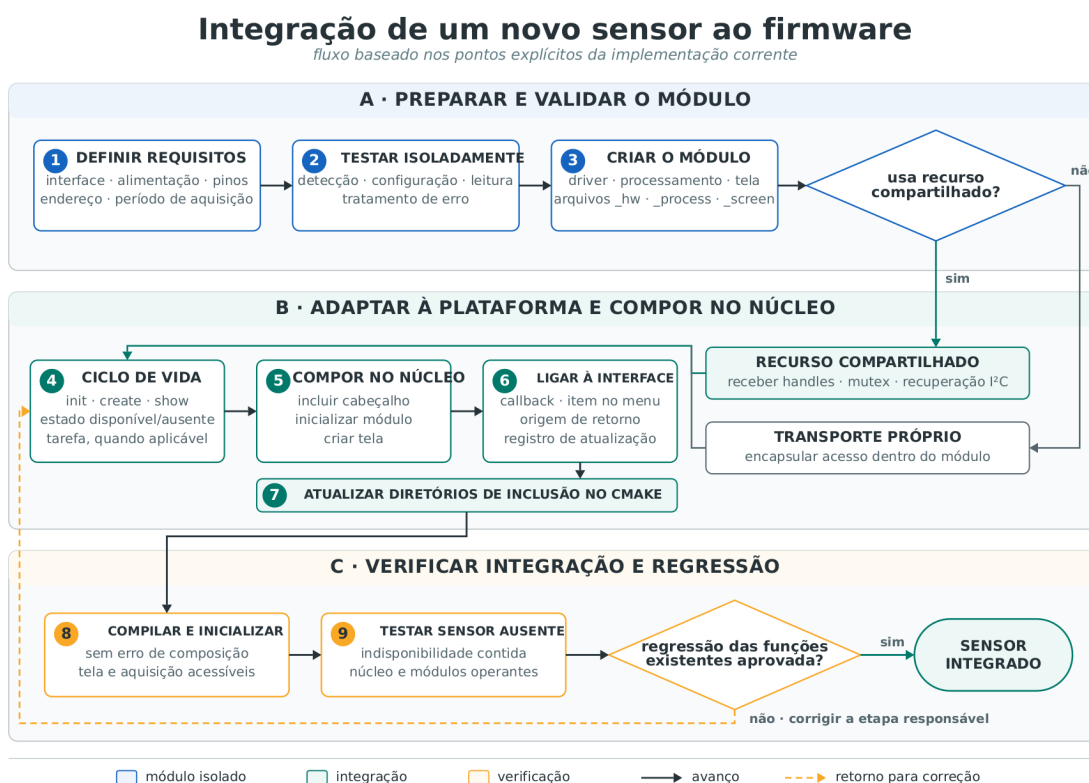
O ciclo de vida começa após a inicialização dos serviços básicos, quando o núcleo percorre os módulos conhecidos. Cada módulo tenta identificar e configurar seu *hardware* e registra internamente se ficou disponível. A criação das telas ocorre

mesmo quando um sensor falha, pois a interface precisa apresentar o estado de indisponibilidade sem depender de nova tentativa contínua.

As tarefas são criadas somente quando necessárias ao módulo. Durante a operação, elas alternam entre bloqueio temporizado, aquisição e publicação do estado. A tela consome esse estado sem executar diretamente a transação sensorial, evitando que uma atualização gráfica aguarde uma conversão lenta.

Como a composição dos módulos permanece explícita na implementação corrente, a inclusão de um novo sensor exige tanto a preparação do módulo quanto seu registro no núcleo e na interface. A Figura 17 organiza esse procedimento a partir dos pontos confirmados no *firmware*.

Figura 17 – Fluxo para integração de um novo sensor ao *firmware*



Fonte: Elaborado pelo autor a partir de `main.c`, `app.h`, `CMakeLists.txt` e da organização dos módulos sensoriais.

Se essa etapa falha, a inicialização do sensor retorna erro ao núcleo, mas esse retorno não aciona encerramento deliberado da aplicação. O módulo marca sua disponibilidade como falsa, preserva uma mensagem apropriada para a tela e não inicia aquisições que dependam do dispositivo ausente.

O núcleo prossegue com os módulos seguintes e com a criação da interface. Assim, relógio, toque e sensores presentes podem permanecer disponíveis quando um sensor externo não responde. A falha de recursos compartilhados, como o próprio

controlador I²C ou o display, possui alcance maior e precisa ser distinguida da ausência isolada de um módulo.

O Quadro 4 resume o comportamento comum sem impor que todos os módulos criem tarefas exatamente no mesmo ponto. No pedômetro, por exemplo, a tarefa associada à tela pode existir mesmo quando o MPU-9250 está ausente, mas permanece em bloqueio temporizado e não tenta adquirir dados. A propriedade comum é a ausência de aquisição dependente, e não necessariamente a ausência de qualquer tarefa auxiliar.

Quadro 4 – Pseudocódigo da inicialização não fatal e do despacho da tela ativa

```

PARA CADA módulo conhecido FAÇA
    resultado ← inicializar acesso ao dispositivo
    SE resultado indica sucesso ENTÃO
        marcar módulo como disponível
        criar tarefa própria, quando prevista pelo módulo
    SENÃO
        marcar módulo como indisponível e prosseguir
    FIM SE
FIM PARA
criar todas as telas e registrar suas funções de atualização
EM CADA tarefa sensorial criada
    SE função inativa ou dispositivo indisponível ENTÃO bloquear
    SENÃO ler dispositivo → processar → publicar estado
NO DESPACHANTE, enquanto a aplicação estiver ativa
    tela ← consultar tela ativa no LVGL
    atualizar somente essa tela a partir do estado publicado
    se o módulo estiver indisponível, apresentar essa condição
    bloquear por 500 ms

```

Fonte: Elaborado pelo autor a partir de `main.c` e das funções `*_module_init` e `*_screen_create`.

Para executar esses módulos, o FreeRTOS opera em configuração de núcleo único. A configuração padrão usada por `esp_lvgl_port` cria a tarefa gráfica com prioridade 4. As tarefas de MAX30102, DS18B20, LTR390, bússola e pedômetro são criadas explicitamente com prioridade 3; a tarefa auxiliar de captura usa prioridade 2; o laço de `app_main` permanece na tarefa principal do ESP-IDF; e a tarefa ociosa ocupa a prioridade 0.

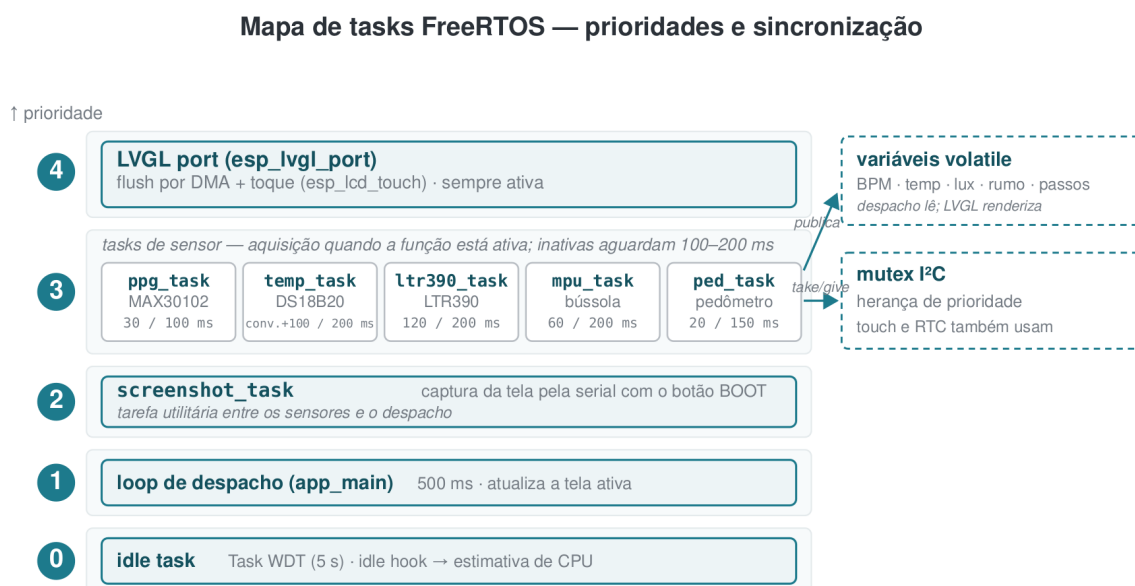
A frequência de *tick* foi configurada em 100 Hz, correspondendo a uma resolução de 10 ms. As tarefas aguardam com primitivas de bloqueio e não realizam espera ocupada entre aquisições. O *watchdog* de tarefas supervisiona a capacidade de a tarefa ociosa voltar a executar.

As tarefas sensoriais verificam se sua tela está ativa antes de acessar o *hardware*. Quando a função não está em uso, elas bloqueiam por um intervalo e voltam a verificar

o estado. Esse condicionamento reduz transações e processamento sem suspender ou destruir repetidamente as tarefas.

A Figura 18 organiza as tarefas confirmadas, suas prioridades e os recursos compartilhados.

Figura 18 – Tarefas FreeRTOS e recursos compartilhados no núcleo único



Fonte: Elaborado pelo autor a partir do *firmware* e da configuração do ESP-IDF.

Na comunicação com a interface, as tarefas publicam as últimas medidas válidas e indicadores de estado em variáveis pertencentes ao módulo. A função de atualização da tela lê uma fotografia desses valores sob a proteção adequada ao tipo de dado. Variáveis escalares alinhadas de até 32 bits podem ser lidas atomicamente pelo processador, mas conjuntos de valores correlacionados exigem cuidado para que a tela não combine amostras de instantes diferentes.

Apenas o contexto autorizado pela porta da LVGL altera objetos gráficos. Quando uma função externa ao contexto gráfico precisa atualizar a interface, ela obtém o bloqueio fornecido pela integração do LVGL. A aquisição permanece independente desse bloqueio para não reter o recurso gráfico durante transações ou processamento.

Como consequência do contrato, a inclusão de uma nova função segue uma sequência definida: implementar ou selecionar o *driver*; criar o estado e a tarefa, quando necessários; implementar as três operações do ciclo de vida; registrar a tela e sua atualização; adicionar o acesso à navegação. Serviços novos somente devem ser acrescentados ao núcleo se forem compartilhados por mais de um módulo ou representarem uma responsabilidade global.

Esse procedimento descreve o ponto de extensão implementado, mas não quantifica seu custo. O número de arquivos alterados, as mudanças no núcleo e o efeito sobre módulos existentes serão determinados pelo ensaio de extensibilidade.

4.3 SERVIÇOS COMPARTILHADOS, CONCORRÊNCIA E RECUPERAÇÃO

No barramento compartilhado, durante a integração, um NACK deixou de ser apenas um erro local de leitura. Como os dispositivos compartilham controlador e linhas, uma transação interrompida podia afetar a operação seguinte, ainda que destinada a outro endereço. Os documentos de desenvolvimento registram ocorrências em que o controlador retornou estado inválido depois da falha inicial.

Cada *driver* passou a tratar o retorno da API em todas as transações. A ausência de ACK não é interpretada como dado nulo nem ignorada. O módulo recebe o erro, evita publicar uma amostra inválida e aciona o procedimento comum de recuperação antes de uma tentativa posterior.

A exclusão mútua é realizada por um *mutex* único que protege as transações do barramento. A tarefa o obtém antes da sequência que deve permanecer indivisível, executa o acesso e o libera em todos os caminhos de saída. A proteção abrange a transação completa, incluindo seleção de registrador e leitura subsequente quando essas etapas formam uma operação lógica.

Foi utilizado *mutex*, e não semáforo binário, para dispor da herança de prioridade oferecida pelo FreeRTOS. Se uma tarefa prioritária aguarda o barramento retido por outra de menor prioridade, a tarefa proprietária pode herdar temporariamente a prioridade até concluir a região protegida.

Diante de um erro, o serviço de recuperação encapsula a chamada `i2c_master_bus_reset()` oferecida pela API mestre do ESP-IDF (ESPRESSIF SYSTEMS, 2025g). Quando uma transação falha, o *driver* solicita a recuperação ainda dentro do fluxo controlado e somente depois libera o *mutex*. Dessa forma, outra tarefa não inicia uma transação enquanto o controlador permanece no estado associado à falha anterior.

O reset não garante que um dispositivo ausente volte a responder. Sua finalidade é devolver o controlador e as linhas a uma condição na qual os demais dispositivos possam ser acessados. O número de tentativas permanece limitado para evitar um ciclo indefinido de falha e recuperação.

O Quadro 5 destaca a ordem implementada nos auxiliares dos *drivers*. A recuperação ocorre antes da liberação do *mutex*; portanto, outra tarefa somente recebe o barramento depois que a transação anterior terminou ou que o controlador foi reinicializado.

Quadro 5 – Pseudocódigo de uma transação I²C protegida e de sua recuperação

```

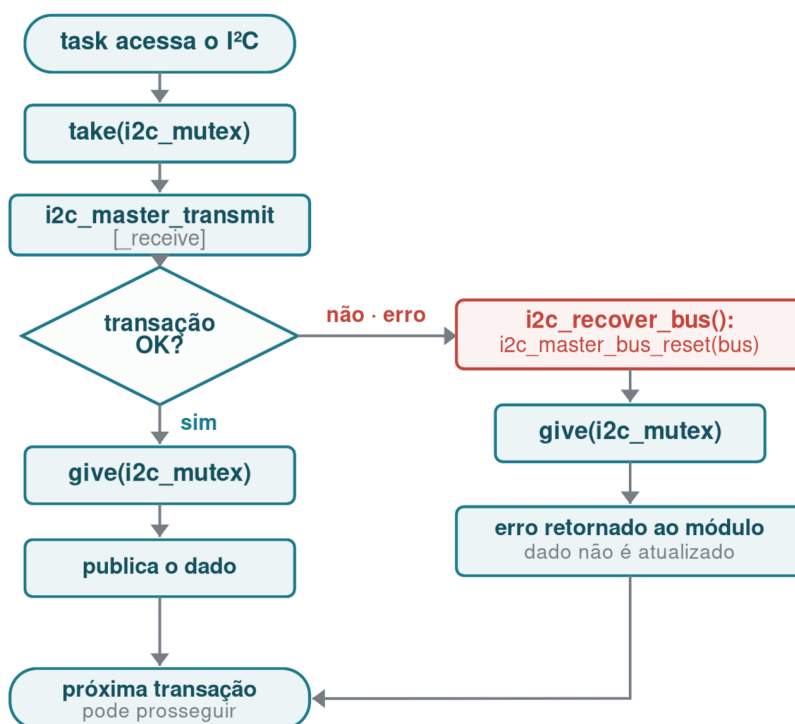
aguardar e obter o mutex compartilhado
resultado ← executar transação com limite de espera
SE resultado indica falha ENTÃO
    reinicializar o controlador do barramento
FIM SE
liberar o mutex compartilhado
retornar resultado ao módulo solicitante
  
```

Fonte: Elaborado pelo autor a partir de `i2c_recover.c` e dos auxiliares de leitura e escrita dos *drivers*.

A ordem do caminho normal e da recuperação pós-NACK é apresentada na Figura 19.

Figura 19 – Transação I²C e recuperação após falha

Recuperação do barramento I²C após erro de transação



Fonte: Elaborado pelo autor a partir de `i2c_recover.c` e dos *drivers* integrados.

Como diagnóstico de inicialização, após criar o barramento, o *firmware* percorre os endereços possíveis e registra os dispositivos que respondem. O scanner foi usado como diagnóstico da montagem e permitiu distinguir endereço incorreto, sensor ausente e problema geral das linhas.

O resultado da varredura não substitui a inicialização dos *drivers*. Responder ao endereço demonstra presença no barramento, mas não confirma identidade, configuração nem validade dos dados.

Para reduzir transações do toque, o controlador CHSC6X fornece um sinal de interrupção associado ao toque. A primeira integração consultava o dispositivo por I²C mesmo quando não havia interação. O *driver* foi alterado para verificar o GPIO17 e iniciar a leitura somente quando o sinal indica evento ativo.

Essa decisão reduz o uso do barramento por uma função que, na maior parte do tempo, permanece ociosa. A quantificação da redução pertence aos resultados; no desenvolvimento, o aspecto relevante é que a interface passou a utilizar uma indicação física de disponibilidade antes da transação.

Também foi necessário conter mensagens repetidas do *driver* I²C durante sequências de erro, pois elas aumentavam o trabalho de formatação e transmissão serial. Em um sistema de núcleo único, essa atividade concorria com a interface e com tarefas temporizadas. Os níveis de log dos componentes mais verbosos foram reduzidos na operação normal, preservando mensagens resumidas e contadores de falha.

Logs detalhados continuam úteis em versões de diagnóstico. Sua ativação deve ser tratada como instrumentação capaz de alterar o comportamento temporal, especialmente durante ensaios de CPU, *boot* e estabilidade.

Para acionar o display, o SPI2 é configurado antes da criação da interface gráfica. O componente do GC9A01A recebe os GPIOs, a frequência de 40 MHz, a resolução e os parâmetros de transformação do painel. Após a sequência de inicialização, a porta do LVGL associa o *buffer* de desenho à função de transferência do controlador.

O conteúdo foi limitado a uma região útil menor que o retângulo de 240 × 240 pixels, evitando cortar textos e controles próximos aos cantos invisíveis do painel circular.

Sobre esse transporte, o `esp_lvgl_port` coordena o temporizador da biblioteca, a entrada por toque e a transferência das regiões invalidadas. As telas foram construídas com widgets, estilos e fontes, em vez de imagens de quadro completo. Essa estratégia permite alterar valores e indicadores sem armazenar uma imagem integral para cada módulo.

Cada tela é criada uma vez e mantida para reutilização. A navegação troca o objeto exibido, enquanto o registro do núcleo seleciona a rotina de atualização apropriada. Operações que modificam objetos utilizam o bloqueio da porta gráfica.

A etapa de transferência em RGB565 exigiu alinhar a ordem dos bytes produzidos pelo LVGL, a ordem dos elementos de cor e a inversão configurada no painel. A configuração consolidada combina troca dos bytes de 16 bits, ordem BGR e inversão de cor do controlador.

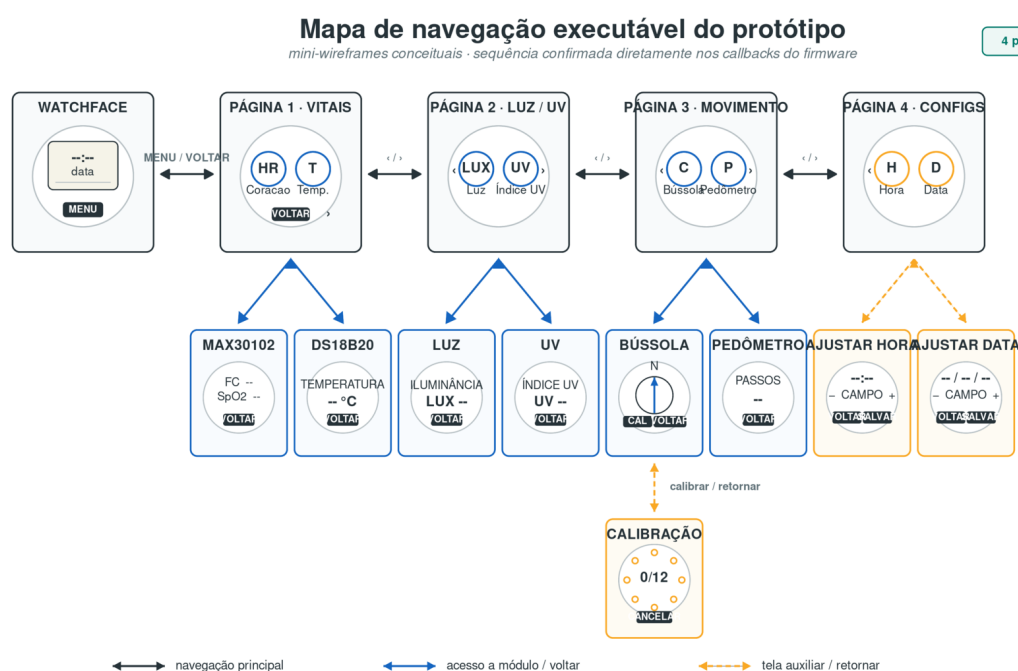
Esses parâmetros atuam em níveis diferentes e não devem ser atribuídos somente ao *endianness* do processador. No *firmware* corrente, `main.c` define ordem BGR e inversão no controlador, enquanto a opção de troca de bytes do RGB565 permanece na configuração da LVGL.

A navegação utiliza quatro páginas de menu na ordem executável: sinais vitais, com coração e temperatura; luz e UV; movimento, com bússola e pedômetro; e configurações de hora e data. A tela principal abre a primeira página, e as setas laterais percorrem as quatro páginas em sequência.

A navegação é independente dos detalhes do sensor. O item conhece a função pública usada para exibir a tela, mas não acessa dados, registradores nem tarefas do módulo.

A Figura 20 apresenta os acessos aos módulos, os caminhos de retorno e as telas auxiliares.

Figura 20 – Mapa de navegação da interface do protótipo



Fonte: Elaborado pelo autor a partir dos *callbacks* e arquivos de tela do *firmware*.

Como ponto de entrada da navegação, a tela principal apresenta hora e data obtidas do PCF8563. Um anel de 60 marcadores distribui a indicação dos segundos ao redor da circunferência. Os elementos centrais permanecem dentro do raio útil e são atualizados sem reconstruir a tela inteira.

O relógio também oferece o ponto de entrada para a configuração de data e hora. A atualização periódica lê os campos necessários do RTC e converte sua representação antes de alterar os rótulos.

Para reduzir o recurso gráfico, a fonte monoespaçada usada na hora foi gerada somente com os algarismos de zero a nove e o caractere de dois-pontos, totalizando 11 glifos. A redução evita incorporar caracteres que não aparecem naquele rótulo.

Como a fonte não contém espaço, o campo ativo na edição não pode ser ocultado por substituição textual. O *firmware* varia a opacidade do objeto, mantendo a indicação visual sem ampliar o conjunto de glifos.

Na configuração de data e hora, dois controles na tela permitem avançar o valor atual e confirmar a etapa. Uma máquina de estados percorre hora, minuto, dia, mês e ano. Ao confirmar o último campo, o *firmware* determina o dia da semana e grava o conjunto no PCF8563.

Durante a edição, o campo ativo alterna sua opacidade para indicar qual valor será alterado. A implementação não depende de botões físicos externos ao Round Display.

O PCF8563 é atendido por um *driver* próprio que lê sete bytes consecutivos a partir do registrador de segundos. Os valores armazenados em BCD são convertidos para inteiros e reunidos em uma estrutura de data e hora. Na escrita, o caminho inverso prepara os campos e preserva os bits de controle definidos pelo dispositivo.

O dia da semana é calculado a partir da data informada antes da gravação. A bateria de reserva do Round Display mantém o RTC durante a interrupção da alimentação principal. O RTC não participa da recuperação de tempo por rede, pois sincronização por NTP não foi implementada.

4.4 IMPLEMENTAÇÃO DOS MÓDULOS

Os módulos seguem o mesmo ciclo de vida, mas preservam particularidades de aquisição e processamento. A descrição a seguir foi agrupada por afinidade funcional para evidenciar o código implementado no projeto, sem reproduzir mapas de registradores já definidos pelos fabricantes.

No módulo MAX30102, o *driver* verifica a identidade do componente e programa o modo de aquisição dos canais vermelho e infravermelho. A configuração corrente utiliza taxa de 100 amostras por segundo, resolução de 18 bits, faixa do conversor de 16.384 nA e corrente aproximada de 7,2 mA em cada emissor. Esses valores correspondem, respectivamente, a 0x67 em SP02_CONFIG e 0x24 nos registradores de corrente dos LED.

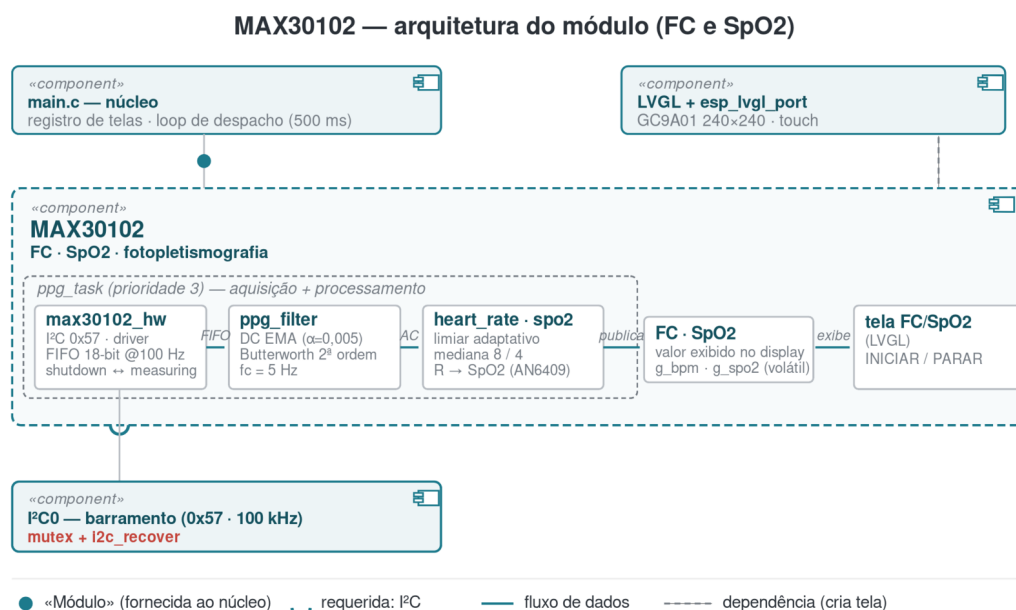
O componente permanece em *shutdown* quando não há medição solicitada. A tela oferece um comando de início, evitando manter os emissores ativos continuamente.

Durante a aquisição do MAX30102, a FIFO desacopla a produção das amostras do instante exato em que a tarefa obtém o barramento. A tarefa consulta a quantidade disponível, lê os bytes em bloco e recompõe cada valor de 18 bits. Os ponteiros da FIFO e condições de transbordamento são verificados para evitar processar dados incompletos como amostras válidas.

Cada amostra é encaminhada ao detector de contato e, quando aceita, às etapas de condicionamento. A interface consome apenas os valores publicados pelo processamento, sem drenar a FIFO no contexto gráfico.

O fluxo completo, incluindo as portas de aceitação que podem rejeitar uma amostra ou um pulso, é sintetizado na Figura 21.

Figura 21 – Aquisição e processamento implementados no módulo MAX30102



Fonte: Elaborado pelo autor a partir de max30102_hw.c, ppg_filter.c, heart_rate.c e spo2.c.

Antes do processamento pulsátil, a presença de contato é inferida do nível do sinal óptico bruto com limiares distintos para entrada e saída. A histerese reduz alternâncias quando o valor permanece próximo ao limite. Ao perder contato, o módulo invalida as estimativas e reinicializa estados que não devem atravessar duas aquisições independentes.

Essa detecção indica somente que o nível óptico é compatível com contato. Ela não garante qualidade suficiente para estimar frequência cardíaca ou SpO₂.

Na etapa de filtragem, uma média móvel exponencial acompanha a componente DC. Durante as primeiras 300 amostras, o coeficiente é 0,1 para acelerar a estabilização; depois, passa a $\alpha = 0,005$. A subtração produz a componente pulsátil, que passa por um filtro Butterworth de segunda ordem com frequência de corte de 5 Hz. A frequência de amostragem usada no cálculo dos coeficientes é 100 Hz.

Os estados dos filtros são preservados entre amostras consecutivas e reiniciados quando começa uma nova sessão ou quando o contato é perdido. O detector ignora as primeiras 500 amostras de cada sessão para que os transientes dos dois estágios decaiam antes de publicar batimentos.

Para estimar a frequência cardíaca, a detecção utiliza uma diferença com intervalo de quatro amostras, o cruzamento de sua derivada associado a um máximo e um limiar adaptativo de amplitude. Candidatos abaixo de 30% do limiar corrente ou com amplitude absoluta inferior ao limite implementado são rejeitados. Um período refratário de 400 ms impede que oscilações próximas sejam interpretadas como batimentos distintos.

A frequência cardíaca é calculada a partir dos intervalos entre batimentos. Uma mediana sobre oito intervalos reduz a influência de uma detecção isolada. Quando não há quantidade suficiente de eventos válidos, a tela mantém a estimativa como indisponível.

Para estimar SpO_2 , a cada batimento aceito, o módulo obtém componentes AC e DC dos canais vermelho e infravermelho e calcula a razão de razões. A conversão consolidada utiliza a relação empírica

$$\widehat{SpO_2} = -45,060R^2 + 30,354R + 94,845.$$

Crítérios de qualidade rejeitam denominadores inadequados, razões fora da faixa admitida e pulsos com baixa modulação. A saída é suavizada pela mediana de quatro estimativas aceitas. O valor permanece denominado estimativa de SpO_2 , pois a implementação não passou por validação clínica.

A integração física exigiu conferir a ordem dos canais, pois a ordem observada dos canais em determinadas placas de conexão não correspondeu à atribuição inicialmente presumida. Os três primeiros bytes de cada amostra foram tratados como infravermelho e os três seguintes como vermelho na montagem utilizada.

Essa correção pertence à associação entre o componente e a placa específica, não a uma inversão universal do MAX30102. O *driver* deve registrar ou encapsular a ordem adotada para impedir que o processamento receba canais trocados.

A tarefa do MAX30102 permanece bloqueada quando a tela não está ativa ou quando a medição não foi iniciada. Durante uma sessão, ela obtém o *mutex*, drena a FIFO, libera o barramento e executa o processamento. A tela apresenta estado de contato, frequência cardíaca e estimativa de SpO_2 .

Falhas de comunicação invalidam a aquisição corrente e acionam a recuperação comum do I²C. A interface continua navegável e diferencia sensor indisponível, ausência de contato e estimativa ainda não estabilizada.

No módulo DS18B20, são utilizados os componentes oficiais do barramento 1-Wire e do DS18B20. A criação do barramento informa o GPIO20 e seleciona o RMT como mecanismo de temporização. O sensor encontrado é associado ao componente de nível superior, que oferece operações de configuração e leitura.

Essa reutilização evita manter no projeto uma segunda implementação dos intervalos do protocolo e da verificação CRC. O módulo permanece responsável pelo ciclo funcional e pela apresentação.

Nesse módulo, o sensor foi configurado para 11 bits. A tarefa solicita uma conversão, bloqueia durante 375 ms e depois lê o *scratchpad*. Esse tempo corresponde à resolução selecionada na chamada efetiva de configuração.

A espera ocorre por bloqueio da tarefa, e não por laço ativo. Como a temperatura varia lentamente, uma nova conversão só é solicitada no período definido pelo módulo e quando sua função está ativa.

Antes de publicar a medida, o componente verifica o CRC e somente então entrega a temperatura. Leituras com erro ou fora da faixa funcional definida para a aplicação não substituem a última amostra válida. O processamento aplica uma média móvel exponencial com $\alpha = 0,5$ e a tela apresenta o resultado com uma casa decimal.

A configuração do LVGL adotada não oferece formatação direta de ponto flutuante pela função usada nos rótulos. O módulo converte o valor para décimos de grau e separa sinal, parte inteira e parte decimal antes de construir o texto.

A inicialização do DS18B20 não recebe o barramento nem o *mutex* I²C. Depois de encontrar e configurar o sensor, o módulo cria sua tarefa e publica temperatura e estado. A tela lê esses dados e apresenta indisponibilidade quando a inicialização ou a leitura falha.

No módulo LTR390, o *driver* adiciona o sensor ao barramento no endereço 0x53 e a 100 kHz, verifica sua identificação e programa resolução de 18 bits com integração e taxa de 100 ms. A configuração associa ganho 3× ao canal de luz ambiente e ganho 18× ao canal ultravioleta.

Os canais ALS e UVS utilizam o mesmo conversor e não operam simultaneamente. O estado do módulo registra o modo configurado, a disponibilidade de nova amostra e as médias mantidas separadamente para iluminância e UVI.

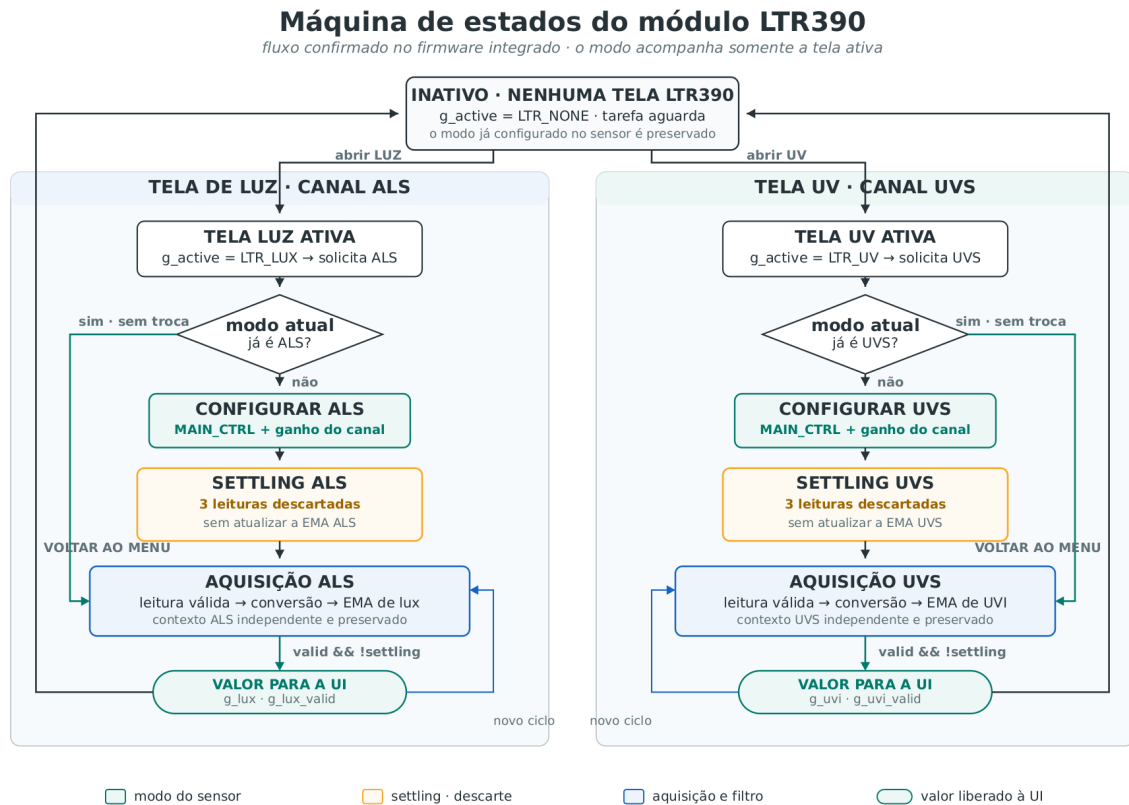
Uma solicitação da tela de luz seleciona ALS, enquanto a tela de ultravioleta seleciona UVS. Essa decisão permanece interna ao módulo e não modifica o núcleo nem o *driver* de outro sensor.

Quando o modo solicitado difere do atual, o *driver* reconfigura o sensor e o módulo descarta as três primeiras amostras. O descarte impede que valores produzidos durante a transição sejam publicados como pertencentes ao novo canal.

A tarefa só alterna quando a função ativa exige outro modo. Não há comutação contínua em segundo plano para manter simultaneamente dois valores que não estão sendo exibidos.

A Figura 22 explicita que o modo segue a tela ativa e que os valores somente se tornam válidos para a interface após o descarte das três amostras.

Figura 22 – Máquina de estados implementada para o LTR390



Fonte: Elaborado pelo autor a partir de `ltr390_screen.c` e `ltr390_process.c`.

Na configuração consolidada, a apresentação das grandezas começa pela iluminação, calculada por

$$E_v = 0,2 N_{ALS},$$

em que N_{ALS} é a contagem bruta. A estimativa de UVI é calculada por

$$\widehat{UVI} = \frac{N_{UVS}}{575}.$$

Os fatores dependem de ganho e tempo de integração. Por isso, as funções de conversão permanecem associadas à configuração usada pelo *driver* e deverão ser atualizadas em conjunto caso esses parâmetros mudem.

Um ajuste de robustez removeu a sequência que executava o comando de reinicialização por *software*. O dispositivo deixava de responder antes de confirmar a escrita, produzindo NACK em uma operação que, no teste isolado, podia parecer apenas uma particularidade local. No barramento compartilhado, esse erro afetava as transações seguintes.

Como todos os registradores necessários já eram programados explicitamente, o comando foi removido da inicialização. A recuperação do I²C continua disponível

para falhas reais, mas não é usada para conservar uma operação desnecessária que produz erro previsível.

Na execução, a tarefa observa qual das telas do módulo está ativa, configura o modo correspondente, aguarda a estabilização e publica o valor convertido. Cada canal mantém seu estado de filtragem por média móvel exponencial, com $\alpha = 0,3$.

Erros de comunicação não são convertidos em zero. A tela distingue indisponibilidade de uma leitura válida de baixa iluminância ou baixa resposta ultravioleta.

No conjunto inercial e magnético, o *driver* inicializa o MPU-9250 em 0x68 e habilita o *bypass* para expor o AK8963 em 0x0C ao controlador principal. Em seguida, verifica a identidade do magnetômetro e configura seu modo de aquisição. Todo o fluxo utiliza o mesmo *mutex* e o mesmo serviço de recuperação dos demais dispositivos.

Na leitura, os registradores de estado são consultados antes dos dados. O registrador ST2 é lido ao final para concluir a sequência e detectar transbordamento.

Na calibração de fábrica, os coeficientes ASA armazenados de fábrica são lidos no modo apropriado do AK8963. Eles corrigem diferenças de sensibilidade entre os eixos e são aplicados à conversão das contagens para microteslas:

$$m_i = \text{raw}_i \frac{4912}{32760} ASA_i.$$

O ajuste ASA não substitui a calibração contra perturbações da montagem. Após essa conversão, o módulo aplica deslocamento e escala obtidos pelo procedimento executado no dispositivo.

Para a calibração pelo usuário, a tela divide o movimento necessário em 12 setores. Durante a rotação do dispositivo, a tarefa acumula mínimos e máximos de cada eixo e atualiza a indicação de cobertura. Ao concluir os setores, calcula para cada eixo o centro

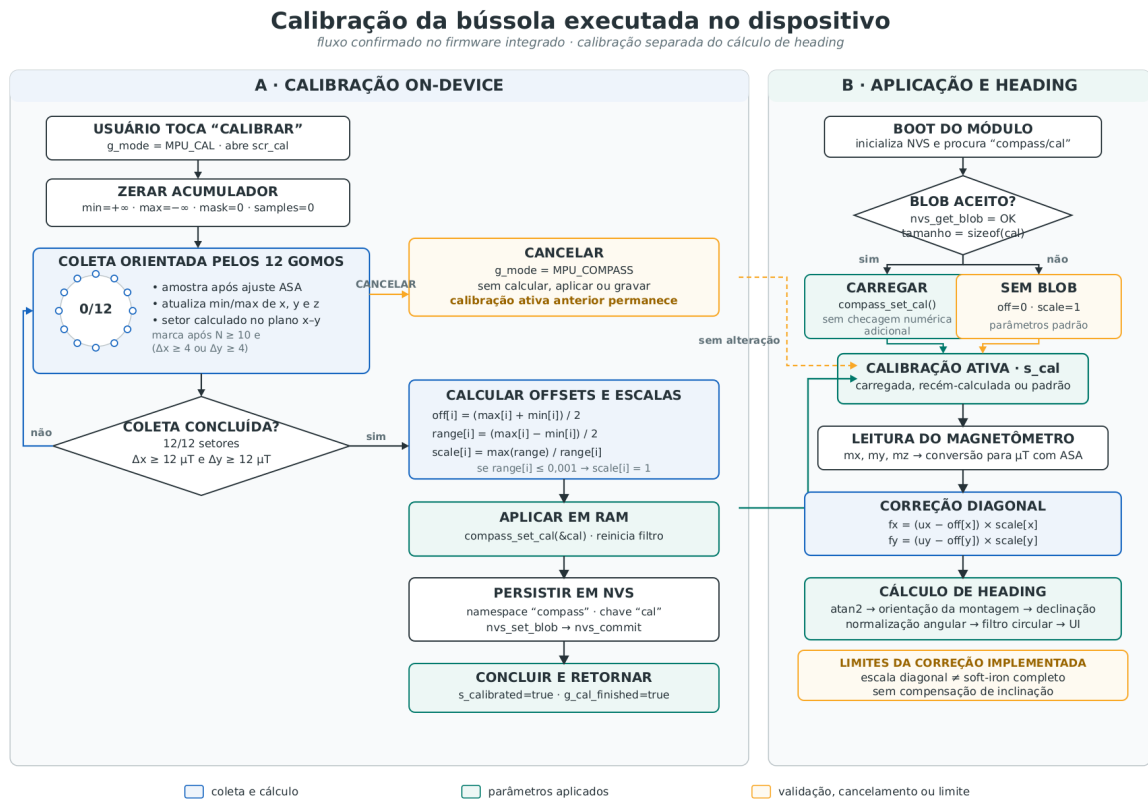
$$b_i = \frac{m_{i,\max} + m_{i,\min}}{2}.$$

e uma escala relativa baseada nas meias-amplitudes observadas. Essa correção representa deslocamento e escala por eixo; não implementa uma transformação elipsoidal completa com termos cruzados.

O procedimento anterior baseado em exportação CSV foi substituído pela calibração executada na própria interface. A versão final não exige recompilar o *firmware* para aplicar novos parâmetros.

A Figura 23 separa o fluxo acionado pelo usuário do cálculo posterior de *heading* e mostra o caminho de cancelamento que preserva a calibração válida.

Figura 23 – Calibração magnética executada no protótipo



Fonte: Elaborado pelo autor a partir do processamento da bússola, da tela de calibração e da persistência em NVS.

Após o cálculo, offsets, escalas e um indicador de validade são gravados na Non-Volatile Storage (NVS), mecanismo de armazenamento de pares chave-valor em flash fornecido pelo ESP-IDF (ESPRESSIF SYSTEMS, 2025h). Na inicialização, o módulo tenta carregar os valores e utiliza padrões somente quando não há conjunto válido. Uma calibração concluída substitui os dados anteriores de forma controlada.

A persistência é restrita à bússola. O contador do pedômetro não é restaurado após reinicialização.

Para obter o rumo, após aplicar ASA e calibração, o módulo ajusta o mapeamento dos eixos à orientação física da placa e calcula

$$\psi = \text{atan2}(m_y, -m_x).$$

O resultado é normalizado para o intervalo de 0° a 360° . Um filtro com $\alpha = 0,15$ trata a diferença angular pelo caminho mais curto, evitando uma média incorreta entre valores próximos de 359° e 1° .

A implementação inverte o sentido de rotação devido à montagem, acrescenta 180° de orientação física e aplica declinação fixa de -21° , conforme os parâmetros compilados. A tela converte o ângulo nas oito direções cardeais e intercardiais. Não

há compensação de inclinação, de modo que a indicação pressupõe o dispositivo aproximadamente nivelado; a declinação fixa também deve ser atualizada para outro local ou época.

Em execução, a tarefa da bússola e a tarefa do pedômetro são distintas, embora ambas acessem o mesmo MPU-9250 por meio do *mutex* compartilhado. Quando a bússola está ativa, sua tarefa lê o magnetômetro a cada 60 ms, atualiza o *heading* e responde a comandos de calibração. A tela apresenta ângulo, direção e estado dos parâmetros persistidos.

Na implementação do pedômetro, a primeira abordagem procurou utilizar o Digital Motion Processor (DMP) do MPU-9250. Ela exigia carregar *firmware* no dispositivo por registradores de memória. Na combinação de módulo e implementação testada, esses acessos produziram NACK e impediram concluir a inicialização.

O ocorrido não demonstra incompatibilidade geral do DMP com o ESP32-C6. Diante do custo de investigar uma dependência adicional e da necessidade de controlar o algoritmo, optou-se por um pedômetro implementado no *firmware* principal.

A solução adotada utiliza apenas o acelerômetro do MPU-9250. O giroscópio permanece desligado para essa função. As amostras dos três eixos são convertidas para unidades de *g*, combinadas pela magnitude e processadas por um detector de eventos.

O acesso ao acelerômetro fica na camada de *hardware* do módulo, enquanto filtragem e contagem permanecem na camada de processamento. A tarefa coordena as duas sem transferir essas responsabilidades ao núcleo.

No modo configurado, o acelerômetro opera na faixa de $\pm 2g$, com fator de conversão de 16.384 contagens por *g*, e é amostrado a 50 Hz. O período de 20 ms é compatível com a resolução de 10 ms do *tick* configurado.

A detecção começa pela magnitude, calculada por

$$a_m = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

Uma média móvel exponencial com $\alpha = 0,2$ reduz componentes rápidas. Um passo é reconhecido quando o sinal cruza o limiar de 1,15 *g*, respeitando uma histerese de 0,05 *g*. Depois de uma detecção, novos eventos são ignorados por 300 ms, limitando contagens múltiplas do mesmo movimento.

Os parâmetros são fixos e foram escolhidos durante o desenvolvimento. Sua adequação a ritmos e usuários diferentes permanece dependente do ensaio controlado de pedometria.

O Quadro 6 evidencia quais operações pertencem ao algoritmo implementado pelo autor. O MPU-9250 fornece as três contagens brutas de aceleração; conversão,

magnitude, filtragem, decisão por limiar, histerese e intervalo mínimo são executadas no ESP32-C6.

Quadro 6 – Pseudocódigo da aquisição e da detecção de passos

```

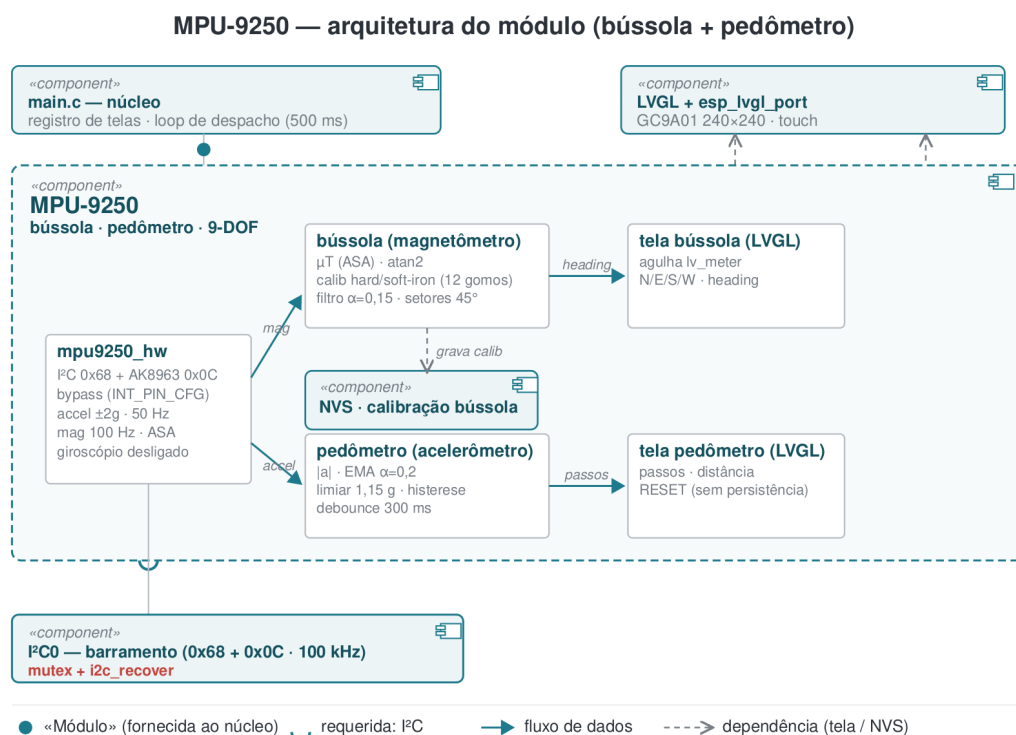
ENQUANTO a tarefa estiver ativa FAÇA
  SE tela inativa ou MPU-9250 indisponível ENTÃO
    bloquear por 150 ms e reiniciar o laço
  FIM SE
  amostra bruta  $\leftarrow$  ler acelerômetro
  SE a leitura for válida ENTÃO
    converter os três eixos para  $g$ 
    magnitude  $\leftarrow \sqrt{a_x^2 + a_y^2 + a_z^2}$ 
    filtrado  $\leftarrow 0,2 \cdot \text{magnitude} + 0,8 \cdot \text{filtrado anterior}$ 
    SE houver cruzamento ascendente de  $1,15\ g$  E intervalo  $> 300$ 
ms ENTÃO
      incrementar contagem e registrar instante
    FIM SE
    SE filtrado  $< 1,10\ g$  ENTÃO rearmar o detector
  FIM SE
  bloquear por 20 ms
FIM ENQUANTO

```

Fonte: Elaborado pelo autor a partir de pedometer_process.c e pedometer_screen.c.

A arquitetura compartilhada entre bússola e pedômetro é mostrada na Figura 24. O detector implementado segue a relação conceitual entre limiar, histerese e intervalo refratário apresentada na Figura 9, na fundamentação teórica.

Figura 24 – Arquitetura do módulo MPU-9250



Fonte: Elaborado pelo autor a partir de `pedometer_process.c` e `pedometer_screen.c`.

Além da contagem, a distância apresentada é o produto do número de passos por um comprimento fixo de 0,70 m:

$$\hat{d} = 0,70 N_{\text{passos}}.$$

Esse valor não é individualizado por estatura ou padrão de marcha. A interface o identifica como estimativa derivada do contador.

Durante a exibição, a tarefa coleta aceleração quando a tela do pedômetro está ativa, atualiza o detector e publica passos e distância. A tela oferece comando para zerar a sessão. O estado permanece em memória volátil e retorna a zero após reinicialização.

4.5 SÍNTESE DA INTEGRAÇÃO

O Quadro 7 resume a fronteira entre comportamento do circuito, infraestrutura fornecida pelo ESP-IDF e processamento escrito para esta plataforma. Essa separação é importante porque uma amostra produzida pelo sensor não equivale ao valor apresentado na interface: transporte, conversão, filtragem e publicação pertencem a camadas diferentes.

Quadro 7 – Responsabilidades na aquisição e no tratamento dos dados

Função	Circuito sensor	ESP-IDF ou componente	Código do autor
PPG	Conversão óptica e FIFO vermelha/infravermelha	Transações I ² C e tarefa	Leitura da FIFO, DC/AC, filtragem, picos, FC e estimativa de SpO ₂
Temperatura	Conversão e memória digital	1-Wire por RMT, CRC e API DS18B20	Cadência, suavização, estado e tela
Luz e UV	Conversão ALS ou UVS	Transações I ² C	Troca de modo, estabilização, descarte, conversão e filtros independentes
Bússola	Campo magnético por eixo	Transporte I ² C e NVS	Calibração, correção diagonal, rumo e interface
Pedômetro	Aceleração por eixo	Transporte I ² C e temporização de tarefa	Magnitude, EMA, limiar, histerese, intervalo refratário e contagem
Interface	Memória de pixels e toque	Controladores LCD, DMA e porta LVGL	Objetos, navegação, atualização e estados de indisponibilidade

Fonte: Elaborado pelo autor a partir do *firmware* integrado.

O contrato uniforme liga essas responsabilidades por inicialização, disponibilidade, criação e apresentação de tela, tarefa opcional e atualização. A inicialização de um sensor pode falhar sem cancelar o restante da sequência; nesse caso, a tarefa dependente não é criada e a tela correspondente apresenta indisponibilidade. O *mutex* do I²C somente serializa transações de dispositivos presentes: ele não substitui o sensor ausente nem bloqueia indefinidamente, pois esse caminho deixa de iniciar a aquisição.

A partição de aplicação foi mantida em 1 MiB. A compilação limpa da versão final do binário produziu um arquivo de 784.240 bytes (765,86 KiB), equivalente a 74,79% da partição. A contribuição incremental de cada módulo ainda não foi medida por compilações controladas.

O pool interno do LVGL foi configurado para 96 KiB. A configuração anterior de 32 KiB não comportava a criação dos objetos da bússola, especialmente o medidor gráfico.

Esse ajuste aumenta a memória reservada à interface e precisa ser considerado junto ao *heap* disponível.

Uma imagem RGB565 de 240×240 pixels requer ao menos 115.200 bytes sem transparência; recursos convertidos podem acrescentar cabeçalho, alinhamento ou outro formato. Para evitar repetir esse custo em cada tela, a interface utiliza widgets, formas e fontes com subconjuntos de glifos.

Quanto à persistência, a NVS armazena os parâmetros de calibração da bússola. O RTC mantém data e hora em seu próprio domínio de alimentação, com bateria de reserva. Esses dois mecanismos possuem ciclos de vida distintos: a NVS conserva parâmetros do *firmware*, enquanto o PCF8563 mantém a base de calendário.

O pedômetro não possui persistência, e valores instantâneos dos sensores não são gravados. O projeto também não implementa histórico de medições, armazenamento em cartão microSD ou sincronização remota. Essas ausências limitam o escopo do estado persistente e evitam atribuir ao protótipo funções de registro que não foram implementadas.

5 RESULTADOS E DISCUSSÃO

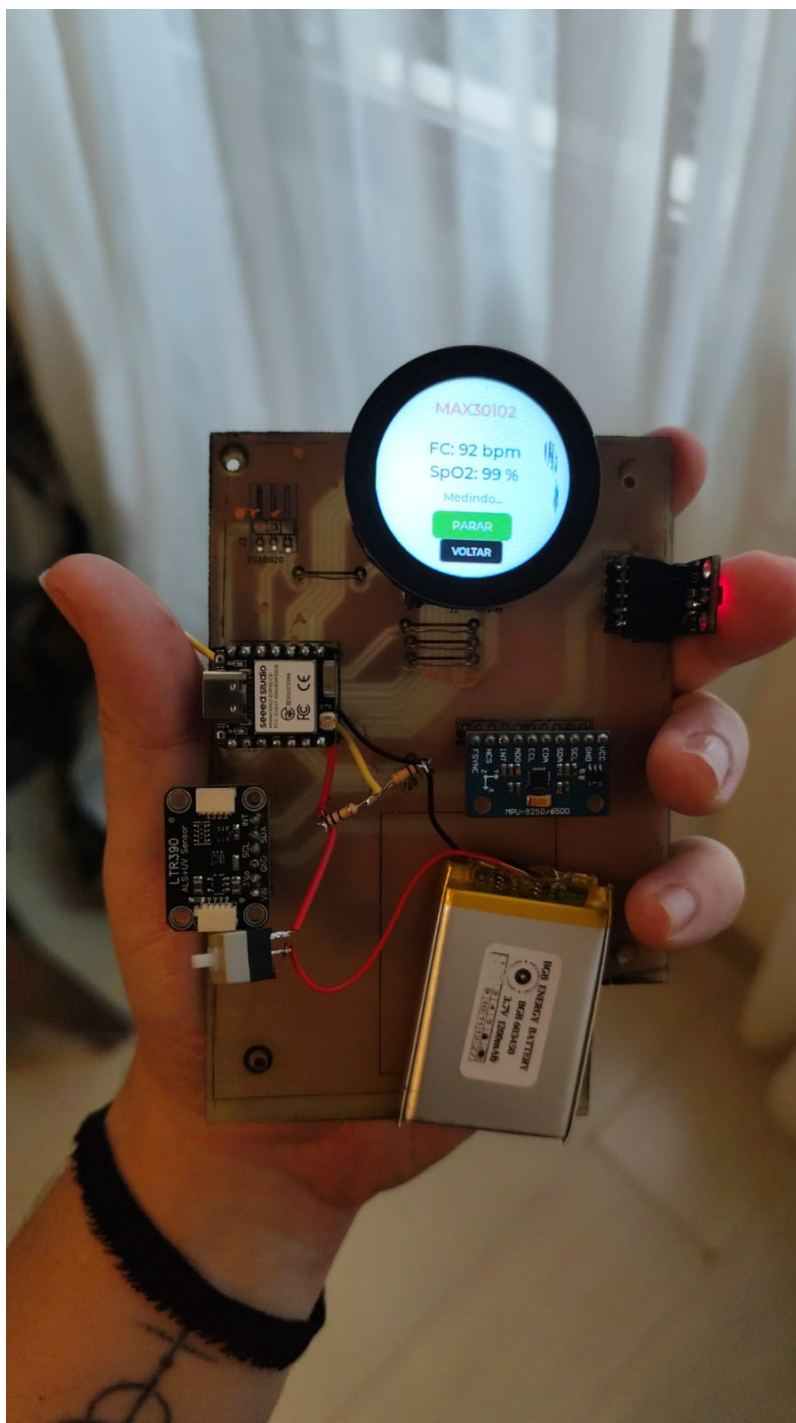
Este capítulo apresenta o artefato integrado e as evidências produzidas pelos procedimentos do Capítulo 3. A discussão privilegia o resultado central do trabalho — a integração modular e a contenção da ausência de sensores — e diferencia medidas quantitativas, análises estáticas e verificações funcionais. Resultados dos algoritmos implementados pelo autor são discutidos separadamente das conversões realizadas pelos circuitos sensores.

5.1 PROTÓTIPO INTEGRADO E ATENDIMENTO AOS REQUISITOS

O protótipo concluído reuniu o XIAO ESP32-C6, o Round Display, a placa autoral, o invólucro impresso em 3D e quatro unidades sensoriais externas: MAX30102, DS18B20, LTR390 e MPU-9250/AK8963. A aplicação executável apresentou hora e data, quatro páginas de menu, navegação por toque e telas para sinais vitais, temperatura, luz, UV, bússola, calibração, pedômetro e configuração temporal. A Figura 25 registra a placa em operação, e a Figura 26 documenta as faces projetadas.

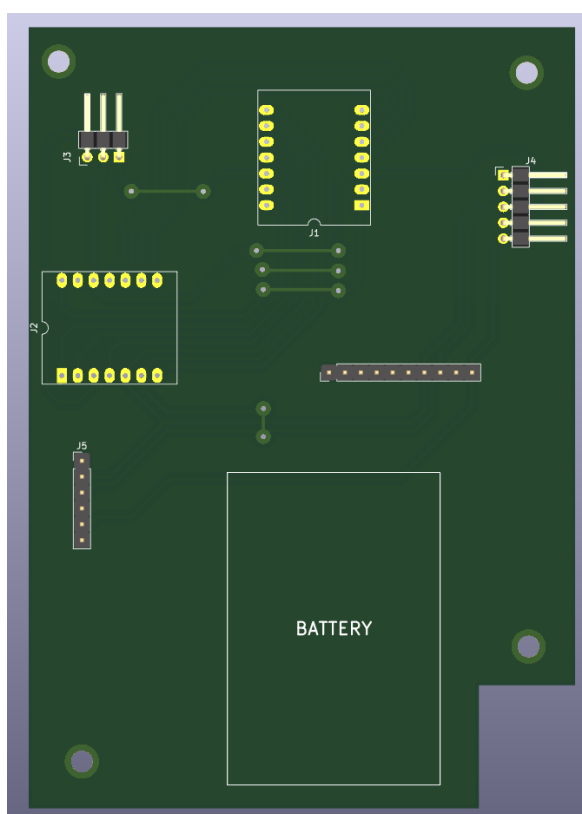
O Quadro 8 relaciona os requisitos aos registros disponíveis. A arquitetura e as funções principais foram implementadas; a avaliação de recursos permaneceu parcial porque não houve campanha comparável de CPU e *heap*, memória incremental ou tempo de inicialização. Essa limitação não impede verificar a execução integrada, mas restringe afirmações sobre desempenho e escalabilidade quantitativa.

Figura 25 – Placa protótipo com display e módulos durante a operação

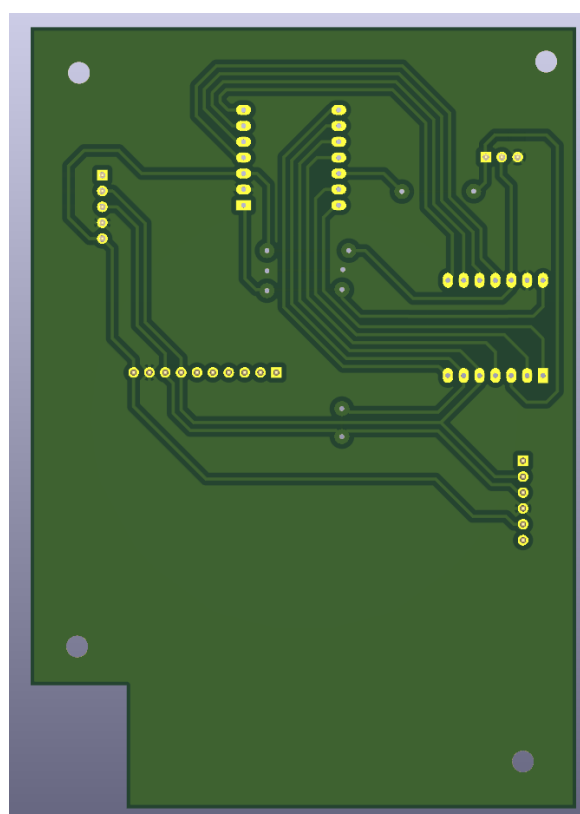


Fonte: Acervo do autor.

Figura 26 – Renderizações das faces da placa desenvolvida



(a) Face superior



(b) Face inferior

Fonte: Elaborado pelo autor a partir do projeto da placa.

Quadro 8 – Atendimento aos requisitos da plataforma

Requisito	Estado	Evidência
Contrato e separação dos módulos	Implementado	Interfaces públicas, tarefas, estados e registro de telas
Integração dos sensores	Verificada funcionalmente	Caderno, dados, tabelas e figuras dos ensaios
Navegação, toque e RTC	Verificados funcionalmente	Capturas, sequência de telas e ensaio temporal
I ² C compartilhado	Implementado	Mutex, limites de espera, varredura e recuperação
Inicialização não fatal	Verificada	Remoções individuais e diferentes arranjos de sensores
Extensibilidade e acoplamento	Avaliados estruturalmente	Estudo retrospectivo e grafo de dependências
Uso de recursos	Parcialmente medido	Tamanho integrado e mapa estático de memória

Fonte: Elaborado pelo autor a partir do *firmware* e do caderno de ensaios.

A compilação limpa gerou arquivo gravável de 784.240 bytes em uma partição de 1.048.576 bytes, correspondendo a 74,79% de ocupação. O mapa registrou 179.360 bytes de DIRAM utilizados entre os 452.112 bytes considerados pela ferramenta, mas esse valor remanescente não equivale ao *heap* disponível em execução. O *pool* interno do LVGL foi configurado para 96 KiB. Como não houve compilações incrementais, esses números caracterizam a configuração completa e não permitem atribuir custo isolado a cada módulo.

5.2 MODULARIDADE E CONTENÇÃO DA AUSÊNCIA DE SENSORES

O *firmware* final preservou em cada função sensorial o acesso ao dispositivo, o processamento, o estado e a apresentação. O núcleo conhece os pontos públicos necessários para compor a aplicação, mas não executa os algoritmos dos sensores nem acessa seus estados internos. A análise de cabeçalhos não encontrou dependências diretas entre MAX30102, DS18B20 e LTR390; a dependência do pedômetro em `mpu9250_hw` é intencional, pois ambas as funções compartilham o acelerômetro. A Figura 27 apresenta as relações identificadas.

No estudo retrospectivo do MAX30102, os seis arquivos de processamento `heart_rate.c/.h`, `ppg_filter.c/.h` e `spo2.c/.h`, que totalizam 678 linhas, permaneceram idênticos entre o teste isolado e o módulo integrado. A adaptação

concentrou-se no acesso ao barramento compartilhado, na ativação da aquisição, na tarefa e na tela. Fora da pasta do módulo foram encontrados seis pontos explícitos de composição: inclusão do cabeçalho, função de exibição, item de menu, inicialização, criação da tela e inclusão do diretório na compilação. O caso demonstra localização das mudanças, embora não meça o tempo necessário para integrar qualquer novo componente.

A contenção de falhas foi verificada desligando o protótipo, removendo individualmente cada sensor externo e repetindo o procedimento com sensores ausentes em diferentes arranjos. Em todas as condições verificadas, a inicialização foi concluída; tela principal, toque, RTC, menus e módulos conectados continuaram acessíveis. A função dependente do componente removido apresentou indisponibilidade, sem executar a aquisição correspondente.

Nenhuma remoção ensaiada produziu *panic*, disparo do temporizador de vigiância, reinicialização espontânea ou impedimento de uso do sistema principal. O *mutex* não mantém uma espera por dispositivo inexistente: quando a inicialização falha, o módulo é marcado como indisponível e a aquisição dependente não é iniciada. O resultado sustenta a contenção funcional nos arranjos testados, mas, pela ausência de protocolo estatístico, não define taxa numérica de disponibilidade.

O Quadro 9 explicita a relação entre o componente removido e o alcance da degradação observada. A remoção do MPU-9250/AK8963 afeta conjuntamente bússola, calibração e pedômetro porque essas funções compartilham o mesmo módulo físico; isso não constitui dependência lógica do pedômetro em relação à bússola. Nos arranjos combinados, todas as funções correspondentes aos componentes ausentes foram desativadas, enquanto as funções sustentadas pelo *hardware* remanescente continuaram acessíveis.

Quadro 9 – Comportamento observado durante a inicialização com sensores ausentes

Condição ensaiada	Função marcada como indisponível	Funções cuja continuidade foi verificada
MAX30102 removido	Aquisição de frequência cardíaca e SpO ₂	Tela principal, toque, RTC, menus e demais sensores conectados
DS18B20 removido	Aquisição de temperatura	Tela principal, toque, RTC, menus e demais sensores conectados
LTR390 removido	Aquisições de luz ambiente e ultravioleta	Tela principal, toque, RTC, menus e demais sensores conectados
MPU-9250/AK8963 removido	Bússola, calibração magnética e pedômetro	Tela principal, toque, RTC, menus e demais sensores conectados
Arranjos com mais de um sensor removido	Funções associadas aos componentes ausentes	Inicialização, navegação e módulos ainda conectados

Fonte: Elaborado pelo autor a partir do caderno de ensaios.

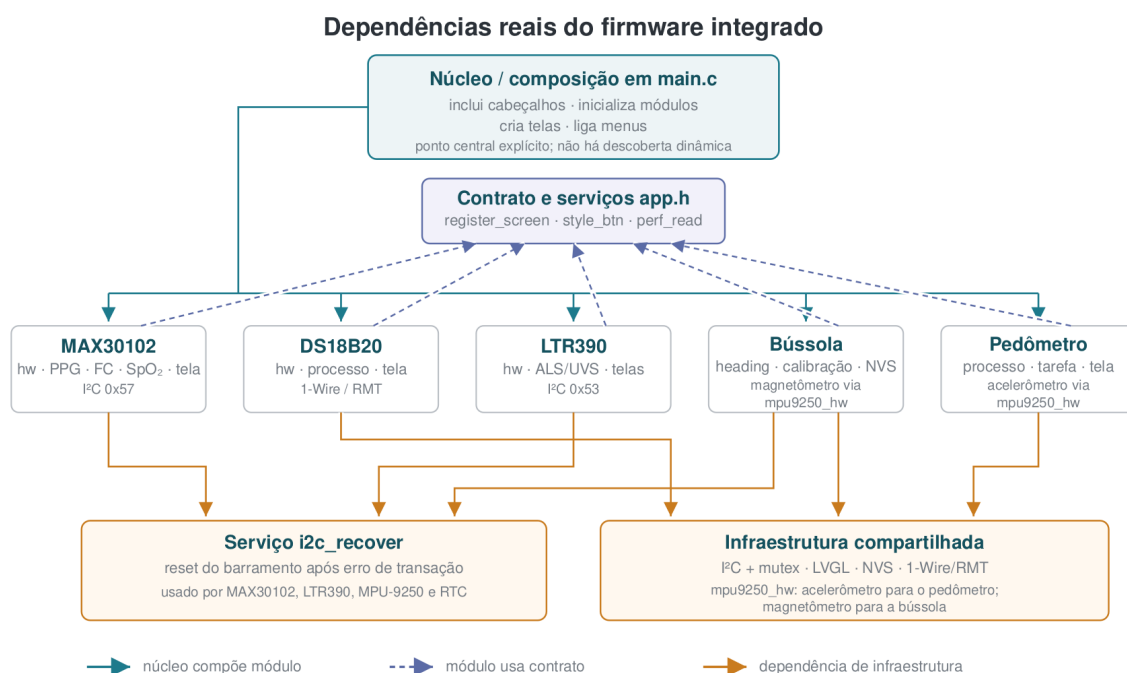
O serviço de recuperação do I²C foi confirmado por análise do código: os caminhos de erro liberam ou reinicializam o controlador antes de retornar, e a varredura inicial pode repetir a busca após uma recuperação. Não foi executada campanha de falhas induzidas durante a operação; por isso, a presença do mecanismo está demonstrada, mas sua latência e taxa de sucesso permanecem por quantificar.

5.3 COMPORTAMENTO DOS ALGORITMOS IMPLEMENTADOS

No MAX30102, a análise do intervalo estável entre 20 e 73 s utilizou 2.651 amostras exportadas. Os níveis médios foram 83.181 contagens no infravermelho e 72.152 no vermelho, grandezas que representam a linha de base óptica e não batimentos por minuto. Os canais pulsáteis filtrados apresentaram correlação de Pearson de 0,955, e a maior componente espectral do infravermelho entre 0,5 e 3,3 Hz ocorreu em aproximadamente 1,40 Hz, equivalente a cerca de 84 BPM.

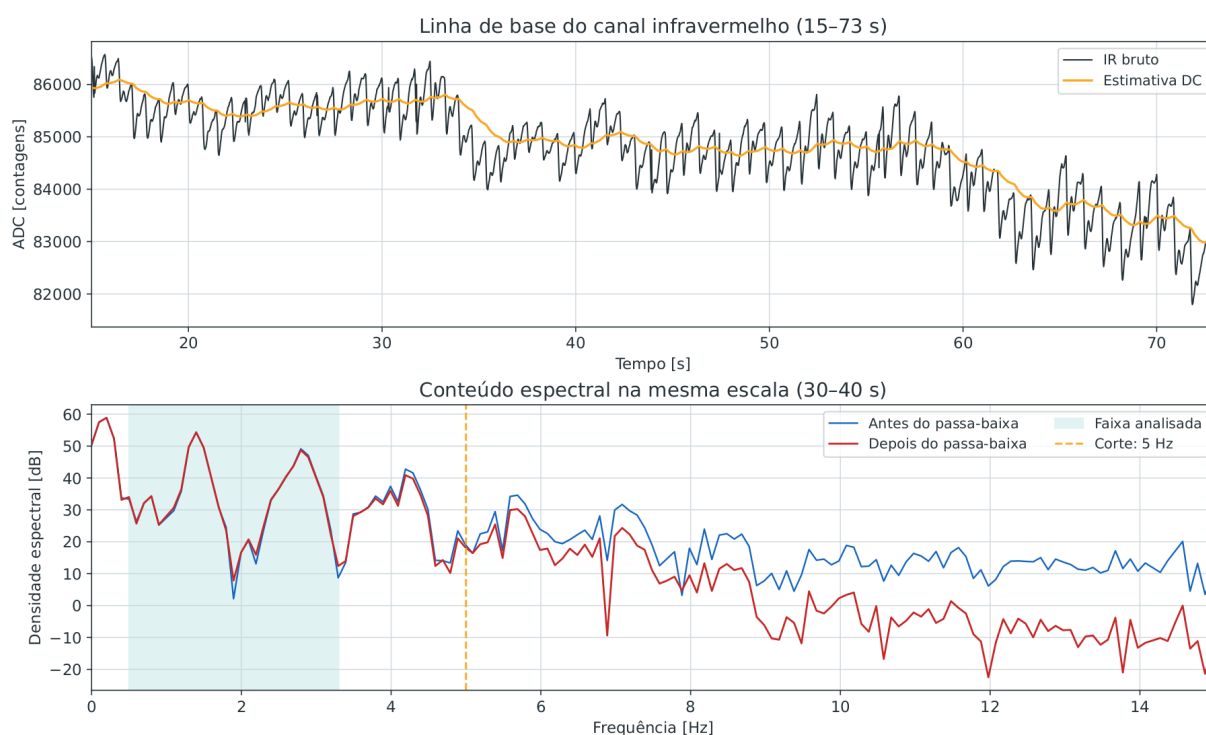
O passa-baixa reduziu em aproximadamente 5,7 dB a potência entre 5 e 10 Hz, mas somente 2,2% da energia do infravermelho e 2,7% da energia do vermelho já estavam acima de 5 Hz. Por isso, as curvas antes e depois do filtro permanecem próximas: o resultado demonstra limitação de banda, não melhora comprovada na exatidão da frequência cardíaca ou da SpO₂. A Figura 28 combina a estimativa de linha de base com a comparação espectral na mesma escala pertinente a cada painel.

Figura 27 – Dependências identificadas por análise estática do *firmware* integrado



Fonte: Elaborado pelo autor a partir das inclusões e chamadas do *firmware*.

Figura 28 – Linha de base do canal infravermelho e comparação espectral antes e depois do passa-baixa



Fonte: Elaborado pelo autor a partir do CSV do ensaio isolado.

Na comparação funcional, MAX30102 e oxímetro indicaram inicialmente 81 BPM. O módulo acompanhou a direção das cinco variações observadas, com subestimação em torno de 5 BPM nos patamares superiores e estados transitórios de cálculo. A SpO_2 indicada pelo oxímetro permaneceu em 97%, enquanto o módulo apresentou 98% a 99%. A Tabela 2 preserva os trechos observados. Como não houve sincronização eletrônica entre os equipamentos, seus limites representam intervalos aproximados de observação, e não janelas metrologicamente alinhadas. O ensaio envolveu um participante e instrumentos de consumo; portanto, verifica o funcionamento do processamento, sem estabelecer desempenho clínico.

Tabela 2 – Comparação funcional das indicações do MAX30102 e do oxímetro

Trecho aproxi- mado	Oxímetro	MAX30102	Comportamento observado
0–15 s	81 BPM	81 BPM	Coincidência no patamar inicial
16–20 s	71–73 BPM	62–68 BPM	Acompanhou a redução, com indicação inferior
21–39 s	79–81 BPM	Pico de 86 BPM, seguido de cerca de 74 BPM	Acompanhou a elevação, com transiente e subestimação posterior
40–46 s	Cerca de 76 BPM	Cerca de 70 BPM	Acompanhou nova redução
47–60 s	Cerca de 81 BPM	74–80 BPM	Acompanhou a elevação gradual
SpO_2	97%	98–99%	Indicações estáveis, com diferença de 1 a 2 pontos percentuais

Fonte: Elaborado pelo autor a partir do caderno de ensaios.

Na bússola, os 12 setores orientaram a atualização de mínimos e máximos dos eixos. Os parâmetros aceitos foram convertidos em deslocamentos e escalas diagonais, persistidos em NVS e aplicados às leituras seguintes. O método corrige *hard iron* e diferenças de escala, mas não termos cruzados de *soft iron* nem inclinação. Em duas orientações no mesmo plano, a bússola de consumo indicou 320° e 130°, enquanto o protótipo indicou 336° e 152°, diferenças de +16° e +22°. As duas observações demonstram resposta direcional, não precisão angular generalizável.

O pedômetro aplicou magnitude da aceleração, média móvel exponencial, limiar, histerese e intervalo refratário. No percurso de 100 m, a contagem manual foi de 140 passos tanto na ida quanto na volta; o protótipo registrou 131 e 133. A Tabela 3

apresenta os erros observados. O resultado é restrito a um participante, um ritmo e duas repetições, e a distância exibida continua baseada no comprimento fixo de 0,70 m por passo.

Tabela 3 – Contagens e erros observados no percurso de 100 m

Percurso	Manual	Protótipo	Erro (%)	Amazfit	Erro (%)
Ida	140	131	-6,4	175	+25,0
Volta	140	133	-5,0	167	+19,3

Fonte: Elaborado pelo autor a partir do caderno de ensaios.

A média do protótipo foi de 132 passos, correspondente a erro relativo médio de -5,7%, com diferença de dois passos entre ida e volta. A contagem manual implica comprimento médio de 0,714 m por passo no percurso, valor próximo do fator fixo de 0,70 m empregado para estimar distância. Essa proximidade não valida o fator para outras pessoas ou ritmos, mas ajuda a explicar sua adequação ao único participante ensaiado. O Amazfit apresentou 175 e 167 passos, com maior diferença entre as duas passagens; a comparação é descritiva e não estabelece superioridade geral entre dispositivos.

5.4 VERIFICAÇÕES FUNCIONAIS DOS MÓDULOS

As verificações complementares foram reunidas na Tabela 4 porque seu propósito foi confirmar aquisição, resposta ao estímulo ou integração com a interface, e não caracterizar separadamente o desempenho de cada sensor. O agrupamento também explicita a diferença entre dado produzido pelo circuito e tratamento realizado pelo *firmware*.

Tabela 4 – Síntese das verificações funcionais complementares

Função	Resultado observado	Tratamento na plataforma	Alcance
Temperatura	22,5 °C no DS18B20 e 23,4 °C no termohigrômetro; resposta de 23 a 3,3 °C no resfriamento e até 70 °C no aquecimento	Conversão pelo componente 1-Wire, suavização e publicação na tela	Um ponto de comparação e estímulos não calibrados

Função	Resultado observado	Tratamento na plataforma	Alcance
Iluminância	Resposta entre obstrução e cerca de 52 klux; teto coerente com ganho $3\times$ e 18 bits	Conversão ALS, filtro independente e indicação de saturação da configuração	Ordenação qualitativa, sem luxímetro
Ultravioleta	UVI entre 3 e 6 ao ar livre diante de referência meteorológica 5; aumento com lanterna UV	Conversão UVS após estabilização e descarte de três amostras	Plausibilidade; fonte e referência não rastreáveis
RTC	Contagem preservada durante 24 h desligado, com atraso observado de aproximadamente 2 s	Leitura do PCF8563 e ajuste por toque	Uma execução; não caracteriza deriva de longo prazo
Display e toque	Navegação pelas quatro páginas, telas funcionais e configurações	Atualização parcial por LVGL, transferência por DMA e leitura condicionada à interrupção do CHSC6X	Verificação funcional; sem latência medida
Bateria	Aproximadamente 4,04 V com USB e 3,84 V somente com bateria	ADC, média de 16 amostras e suavização; indicação baseada em tensão	Não mede capacidade nem autonomia
Consumo	Corrente total entre 90 e 210 mA nos nove cenários registrados	Leitura externa em série; nenhuma decomposição por tarefa	Caracterização descritiva por cenário

Fonte: Elaborado pelo autor a partir do caderno de ensaios.

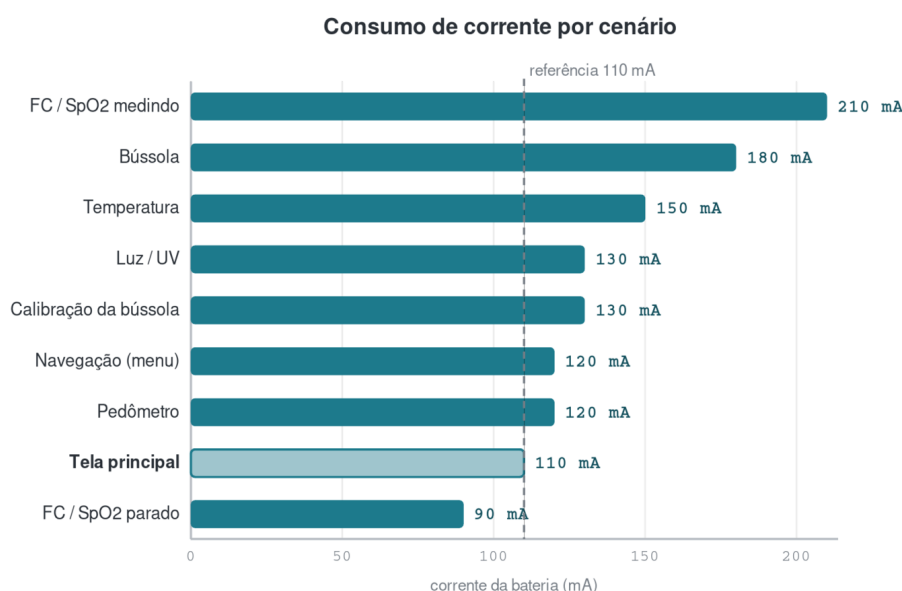
O LTR390 confirmou a necessidade de tratar ALS e UVS como modos exclusivos. A troca conforme a tela ativa, a estabilização e o descarte de três amostras evitaram apresentar imediatamente uma amostra do modo anterior; filtros independentes impediram que o estado de um canal contaminasse o outro. A saturação próxima de 52 klux pertence à configuração empregada e não deve ser apresentada como limite absoluto do componente.

No RTC, nenhuma diferença foi distinguida após 1 h na resolução do procedimento. Após 24 h com a alimentação principal removida, a bateria de reserva manteve a contagem, e foi observado atraso de aproximadamente 2 s. A razão correspondente, próxima de 23 ppm, descreve somente esse intervalo; uma caracterização de deriva exigiria vários dias, repetições e controle da referência e da temperatura.

Na interface, o CHSC6X foi identificado no endereço 0x2E e sua linha de interrupção passou a condicionar as leituras do toque. O LVGL atualiza regiões alteradas do quadro lógico, e a porta gráfica transfere os blocos de pixels por DMA sem exigir que a CPU envie cada pixel individualmente. O funcionamento foi confirmado por navegação e comandos, mas não foram coletadas métricas de latência ou taxa de falha.

A corrente total variou conforme a tela e o módulo ativo. Foram registrados 110 mA na tela principal, 120 mA na navegação, 150 mA na tela de temperatura, 130 mA em luz/UV, 180 mA na bússola, 130 mA na calibração, 120 mA no pedômetro, 90 mA com a tela de PPG parada e 210 mA durante a aquisição de PPG. A Figura 29 apresenta os nove cenários sobre a referência visual da tela principal.

Figura 29 – Corrente total medida nos cenários funcionais da plataforma



Fonte: Elaborado pelo autor a partir do caderno de ensaios.

O menor valor ocorreu com a tela estática do PPG, e o maior durante sua aquisição. A tela principal não foi o estado de menor corrente, resultado compatível com sua atualização visual periódica, mas o ensaio não separou o custo da interface daquele dos demais subsistemas. Da mesma forma, os 210 mA observados durante o PPG reúnem iluminação dos emissores, processamento e atualização gráfica, sem permitir decomposição individual. Como não houve descarga completa, esses valores caracterizam cenários de operação e não constituem medição de autonomia.

5.5 DISCUSSÃO INTEGRADA E LIMITAÇÕES

O resultado mais consistente é a execução de sensores, interface, RTC e serviços compartilhados em um mesmo núcleo, com tratamento localizado da indisponibilidade.

A análise estática mostrou que a modularidade não elimina pontos explícitos de composição no núcleo, mas restringe o processamento e o estado aos módulos. Os ensaios de remoção complementaram essa evidência ao mostrar que o contrato de disponibilidade tem efeito observável: a função dependente é desativada sem impedir a navegação e os componentes independentes.

O I²C concentrou os principais efeitos sistêmicos. Acesso excessivo do toque, NACK durante operações de configuração e mensagens seriais em excesso deixaram de ser questões locais quando passaram a disputar barramento e tempo de processador. Mutex, limites de espera, leitura condicionada pela interrupção e recuperação do controlador formam uma resposta coerente a esse compartilhamento, embora a eficácia dinâmica da recuperação ainda exija falhas controladas e repetidas.

Em comparação com os trabalhos relacionados, o Amulet enfatiza múltiplas aplicações, energia e isolamento; a WMP enfatiza aquisição fisiológica sincronizada e processamento remoto; e o WEARDA enfatiza coleta reprodutível sobre dispositivo comercial (HESTER et al., 2016; BALDINI et al., 2023; VAN DIJK; GAWEHNS; VAN LEEUWEN, 2023). A contribuição deste trabalho situa-se no *firmware* de uma plataforma construída com sensores heterogêneos, no contrato de integração, na coordenação do barramento e na contenção funcional da ausência de componentes. Os escopos e métricas distintos não sustentam uma classificação de superioridade geral.

As principais limitações quantitativas são a ausência de medições comparáveis de CPU e *heap*, custo incremental por módulo, série de tempo de inicialização, injeção de falhas I²C e ensaio prolongado com roteiro e duração fixos. Nos sensores, faltaram ampliação da amostra de participantes e condições, referências calibradas, compensação de inclinação da bússola e caracterização da bateria por descarga. A placa e o invólucro foram realizados, mas não passaram por ensaios mecânicos ou ambientais.

Essas limitações definem a fronteira das conclusões, sem descaracterizar o resultado de integração. O protótipo é uma plataforma funcional e documentada para evolução; não é um instrumento clínico ou metrológico, nem uma demonstração de confiabilidade estatística. A rastreabilidade dos arquivos permite que as lacunas sejam tratadas em campanhas futuras sem confundi-las com atividades já executadas.

6 CONCLUSÕES

Este trabalho desenvolveu uma plataforma vestível modular multissensor baseada no XIAO ESP32-C6, no ESP-IDF e no FreeRTOS. O problema abordado foi organizar sensores heterogêneos, interface gráfica e serviços compartilhados sem concentrar o processamento específico no núcleo e sem comprometer as funções independentes quando um componente sensorial não estivesse disponível.

A implementação separou núcleo, serviços, acesso ao *hardware*, processamento, estado e apresentação. MAX30102, DS18B20, LTR390, bússola e pedômetro foram incorporados por uma convenção comum de inicialização, disponibilidade, tarefa opcional, criação de tela, apresentação e atualização. As diferenças de transporte permaneceram encapsuladas: dispositivos I²C receberam barramento e *mutex* compartilhados, enquanto o DS18B20 utilizou 1-Wire por RMT. O fluxo entre tarefa, aquisição, processamento e LVGL tornou explícito quando o dado provinha do sensor e quando era transformado pelo código do autor.

O serviço I²C reuniu exclusão mútua com herança de prioridade, limites de espera, varredura inicial e recuperação após erro. A análise do código confirmou esses caminhos, e a remoção individual de cada sensor, além de diferentes arranjos de remoção, mostrou que o sistema concluiu a inicialização e preservou tela principal, toque, RTC, menus e funções conectadas. A tela dependente indicou indisponibilidade, e nenhuma aquisição foi executada para o dispositivo ausente. Desse modo, o trabalho demonstrou a contenção funcional da ausência de componentes nas condições ensaiadas, embora não tenha quantificado confiabilidade nem recuperação sob falhas dinâmicas induzidas.

A extensibilidade foi examinada estruturalmente no caso do MAX30102. Seis arquivos de processamento, com 678 linhas, foram reutilizados sem alteração, e a integração exigiu seis pontos explícitos fora da pasta do módulo, sem modificar os outros módulos sensoriais. O resultado demonstra localização das mudanças no caso estudado; ele não elimina a composição explícita no núcleo nem permite estimar o tempo de integração de qualquer dispositivo futuro.

O protótipo físico, a placa autoral e o invólucro impresso em 3D foram concluídos. A aplicação integrou quatro páginas de menu, sensores biométricos, ambientais e inerciais, relógio de tempo real, leitura de bateria e navegação por toque. A compilação final ocupou 784.240 bytes, ou 74,79% da partição de 1 MiB. Nos algoritmos, o PPG apresentou canais pulsáteis coerentes e componente espectral próxima de 84 BPM; a calibração magnética executada no dispositivo produziu parâmetros diagonais persistentes; e o pedômetro registrou 131 e 133 passos diante de 140 passos contados manualmente. As verificações dos demais módulos confirmaram aquisição, resposta aos estímulos e apresentação na interface dentro do alcance experimental declarado.

As limitações concentram-se na caracterização quantitativa. Não foram realizadas campanhas comparáveis de CPU e *heap*, custo incremental de memória, tempo de inicialização, recuperação I²C sob falhas induzidas, ensaio prolongado com roteiro completo ou descarga integral da bateria. Os comparadores sensoriais não constituíam referências rastreáveis, a amostra experimental foi reduzida, a bússola não compensa inclinação nem aplica calibração matricial completa, e o pedômetro utiliza parâmetros fixos e estado volátil. Essas restrições delimitam o alcance das conclusões, sem retirar o resultado principal de integração arquitetural e funcional.

Como continuidade imediata, recomenda-se caracterizar CPU, memória dinâmica e custo incremental por cenário, induzir falhas controladas no I²C, executar ensaio prolongado e obter curva completa de descarga. Em energia, devem ser avaliados controle efetivo da iluminação de fundo, desligamento seletivo de sensores, modos de baixo consumo, despertar pelo RTC e um circuito dedicado de contagem de carga.

Na expansão da plataforma, o cartão microSD pode receber históricos, dados de ensaio e recursos gráficos tecnicamente adequados, com seleção independente e exclusão mútua no SPI compartilhado com o display. Wi-Fi e Bluetooth do ESP32-C6 podem apoiar transferência, configuração e sincronização, desde que consumo, segurança e concorrência sejam medidos. Um módulo GNSS com suporte a RTK também pode ser investigado para trajetórias, considerando antena, consumo, fonte de correções, operação a céu aberto e comparação com referência apropriada, sem antecipar precisão antes dos ensaios.

Os algoritmos podem evoluir com compensação de inclinação e calibração magnética matricial, persistência e reinicialização diária do pedômetro, parâmetros adaptativos, amostra experimental ampliada e índice de qualidade do PPG com detecção de movimento e rejeição de artefatos. Na arquitetura, um registro declarativo ou dinâmico pode reduzir os pontos de alteração no núcleo. Também é pertinente avaliar o Zephyr RTOS como alternativa tecnológica, comparando seu modelo de dispositivos, árvore de dispositivos e mecanismos de configuração modular com a solução implementada em ESP-IDF; essa avaliação deve medir benefícios, custos de migração e impacto nos serviços existentes, sem pressupor superioridade antes do estudo.

Conclui-se que o objetivo de desenvolver uma plataforma vestível modular multisensor foi atingido. O artefato executável, a separação de responsabilidades, a integração dos módulos e a operação diante de sensores ausentes constituem uma base técnica documentada para novas medições e extensões do projeto.

A REPOSITÓRIO E MAPA DOS ARTEFATOS

Os arquivos que permitem estudar e reproduzir o trabalho estão disponíveis no repositório público <https://github.com/DaFrancoGui/TCC>, no ramo `master`. O caderno usado para rastrear procedimentos, instrumentos, dados, figuras e limitações pode ser consultado em https://github.com/DaFrancoGui/TCC/blob/master/Ensaaios/caderno_de_ensaaios.md. Não se indica um *commit* específico porque a entrega final será mantida nesse ramo.

Os diagnósticos de integração foram publicados em https://github.com/DaFrancoGui/TCC/tree/master/docs/problemas_solucoes. Esses registros preservam sintomas, hipóteses, causas e soluções, inclusive estados históricos; em caso de divergência, o código em <https://github.com/DaFrancoGui/TCC/tree/master/Codigos/IDroid/main> é a autoridade para a implementação corrente.

Quadro 10 – Mapa dos principais artefatos publicados

Artefato	Localização no repositório
Firmware integrado	Codigos/IDroid/
Testes isolados e dados	Codigos/Teste_de_componentes/
Caderno de ensaios	Ensaaios/
Diagramas, gráficos e fotografias	Imagens/
Problemas e soluções	docs/problemas_solucoes/
Fonte consolidada e PDF	Rascunho/TCC_COMPLETO.tex e .pdf

Fonte: Elaborado pelo autor.

REFERÊNCIAS

- AKM. **AK8963: 3-axis electronic compass**. Tokyo: Asahi Kasei Microdevices, 2013. Disponível em: <https://www.akm.com/content/dam/documents/products/electronic-compass/ak8963c/ak8963c-en-datasheet.pdf>. Acesso em: 1 jul. 2026.
- ALLEN, John. Photoplethysmography and its application in clinical physiological measurement. *Physiological Measurement*, Bristol, v. 28, n. 3, p. R1-R39, 2007. DOI: 10.1088/0967-3334/28/3/R01.
- ANALOG DEVICES. **Battery fuel gauges: accurately measuring charge level**. Wilmington, 22 dez. 2006. Disponível em: <https://www.analog.com/en/resources/technical-articles/2022/07/16/09/50/battery-fuel-gauges-accurately-measuring-charge-level.html>. Acesso em: 1 jul. 2026.
- ANALOG DEVICES. **How to power the extended features of 1-Wire devices**. Wilmington, 17 jun. 2008. Disponível em: <https://www.analog.com/en/resources/technical-articles/how-to-power-the-extended-features-of-1wire-devices.html>. Acesso em: 1 jul. 2026.
- ANALOG DEVICES. **MAX30102: high-sensitivity pulse oximeter and heart-rate sensor for wearable health**. Rev. 1. Wilmington, 2018. Disponível em: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30102.pdf>. Acesso em: 1 jul. 2026.
- ANALOG DEVICES. **DS18B20: programmable resolution 1-Wire digital thermometer**. Rev. 6. Wilmington, 2019. Disponível em: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>. Acesso em: 1 jul. 2026.
- ANALOG DEVICES. **Guidelines for SpO2 measurement using the MAX30102**. Application Note 6409. Wilmington, 2018. Disponível em: <https://www.analog.com/en/resources/app-notes/an6409.html>. Acesso em: 1 jul. 2026.
- ANALOG DEVICES. Enhancing the performance of pedometers using a single accelerometer. *Analog Dialogue*, Wilmington, v. 44, 2010. Disponível em: <https://www.analog.com/en/resources/analog-dialogue/articles/pedometer-design-3-axis-digital-acceler.html>. Acesso em: 1 jul. 2026.
- AVIZIENIS, Algirdas et al. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, Piscataway, v. 1, n. 1, p. 11-33, 2004. DOI: 10.1109/TDSC.2004.2.
- BALDINI, Andrea et al. A real-time, open-source, IoT-like, wearable monitoring platform. *Electronics*, Basel, v. 12, n. 6, art. 1498, 2023. DOI: 10.3390/electronics12061498.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.
- BRAJDIC, Agata; HARLE, Robert. Walk detection and step counting on unconstrained smartphones. In: ACM INTERNATIONAL JOINT CONFERENCE ON PERVASIVE AND UBIQUITOUS COMPUTING, 2013, Zurich. *Proceedings [...]*. New York: ACM, 2013. p. 225-234. DOI: 10.1145/2493432.2493449.

CARUSO, Michael J. **Applications of magnetic sensors for low cost compass systems**. Plymouth: Honeywell, 2000.

CIE. **CIE 018:2019: the basis of physical photometry**. Vienna: International Commission on Illumination, 2019.

ESPRESSIF SYSTEMS. **ESP32-C6 series datasheet**. Version 1.5. Shanghai, 2025a. Disponível em: https://documentation.espressif.com/esp32-c6_datasheet_en.pdf. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **ESP32-C6 technical reference manual**. Version 1.2. Shanghai, 2025b. Disponível em: https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **ADC calibration driver: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025c. Disponível em: https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/peripherals/adc_calibration.html. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **Watchdogs: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025d. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/system/wdts.html>. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **SPI master driver: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025e. Disponível em: https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/peripherals/spi_master.html. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **Remote control transceiver: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025f. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/peripherals/rmt.html>. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **I2C driver: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025g. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/peripherals/i2c.html>. Acesso em: 1 jul. 2026.

ESPRESSIF SYSTEMS. **Non-volatile storage library: ESP32-C6: ESP-IDF Programming Guide v5.5.1**. Shanghai, 2025h. Disponível em: https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32c6/api-reference/storage/nvs_flash.html. Acesso em: 1 jul. 2026.

FREERTOS. **vTaskDelayUntil()**. [S. l.], 2026a. Disponível em: <https://www.freertos.org/vtaskdelayuntil.html>. Acesso em: 1 jul. 2026.

FREERTOS. **FreeRTOS binary semaphores**. [S. l.], 2026b. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/02-Binary-semaphores>. Acesso em: 1 jul. 2026.

FREERTOS. **The FreeRTOS kernel book**. [S. l.], 2026c. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-Mastering-the-FreeRTOS-real-time-kernel>. Acesso em: 1 jul. 2026.

GALAXYCORE. **GC9A01A TFT LCD driver IC datasheet**. Version 1.0. Shenzhen, 2022.

HESTER, Josiah et al. Amulet: an energy-efficient, multi-application wearable platform. In: ACM CONFERENCE ON EMBEDDED NETWORKED SENSOR SYSTEMS, 14., 2016, Stanford. *Proceedings [...]*. New York: ACM, 2016. p. 216-229. DOI: 10.1145/2994551.2994554.

ISO. **ISO 80601-2-61:2017: medical electrical equipment: particular requirements for basic safety and essential performance of pulse oximeter equipment**. Geneva: International Organization for Standardization, 2017.

LITE-ON TECHNOLOGY. **LTR-390UV-01: UV/ambient light sensor with I2C interface**. Rev. C. New Taipei City, 2024. Disponível em: https://optoelectronics.liteon.com/upload/download/DS86-2015-0004/LTR-390UV-01_Final_%20DS_V1.7.PDF. Acesso em: 1 jul. 2026.

LIU, Chung Laung; LAYLAND, James W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, New York, v. 20, n. 1, p. 46-61, 1973. DOI: 10.1145/321738.321743.

LVGL. **Objects: LVGL documentation v8.3**. 2023a. Disponível em: <https://docs.lvgl.io/8.3/overview/object.html>. Acesso em: 1 jul. 2026.

LVGL. **Drawing: LVGL documentation v8.3**. 2023b. Disponível em: <https://docs.lvgl.io/8.3/overview/drawing.html>. Acesso em: 1 jul. 2026.

MEYER, Bertrand. Applying design by contract. *Computer*, Los Alamitos, v. 25, n. 10, p. 40-51, 1992. DOI: 10.1109/2.161279.

NITZAN, Meir; ROMEM, Ayal; KOPPEL, Robert. Pulse oximetry: fundamentals and technology update. *Medical Devices: Evidence and Research*, Auckland, v. 7, p. 231-239, 2014. DOI: 10.2147/MDER.S47319.

NXP SEMICONDUCTORS. **UM10204: I2C-bus specification and user manual**. Rev. 7.0. Eindhoven, 2021. Disponível em: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. Acesso em: 1 jul. 2026.

NXP SEMICONDUCTORS. **PCF8563: real-time clock/calendar**. Rev. 11.1. Eindhoven, 2026. Disponível em: <https://www.nxp.com/docs/en/data-sheet/PCF8563.pdf>. Acesso em: 1 jul. 2026.

PARNAS, David L. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, New York, v. 15, n. 12, p. 1053-1058, 1972. DOI: 10.1145/361598.361623.

RENAUDIN, Valérie; AFZAL, Muhammad Haris; LACHAPELLE, Gérard. Complete triaxis magnetometer calibration in the magnetic domain. *Journal of Sensors*, London, v. 2010, art. 967245, 2010. DOI: 10.1155/2010/967245.

SENEVIRATNE, Suranga et al. A survey of wearable devices and challenges. *IEEE Communications Surveys & Tutorials*, Piscataway, v. 19, n. 4, p. 2573-2620, 2017. DOI: 10.1109/COMST.2017.2731979.

SEED STUDIO. **Getting started with Seeed Studio XIAO ESP32C6**. Shenzhen, 2024. Disponível em: https://wiki.seeedstudio.com/xiao_esp32c6_getting_started/. Acesso em: 1 jul. 2026.

SEED STUDIO. **Round Display for XIAO**. Shenzhen, [s.d.]. Disponível em: https://wiki.seeedstudio.com/get_start_round_display/. Acesso em: 1 jul. 2026.

SHA, Lui; RAJKUMAR, Ragunathan; LEHOCZKY, John P. Priority inheritance protocols: an approach to real-time synchronization. *IEEE Transactions on Computers*, Piscataway, v. 39, n. 9, p. 1175-1185, 1990. DOI: 10.1109/12.57058.

SPRAGER, Sebastijan; JURIC, Matjaz B. Inertial sensor-based gait recognition: a review. *Sensors*, Basel, v. 15, n. 9, p. 22089-22127, 2015. DOI: 10.3390/s150922089.

STEVENS, Wayne P.; MYERS, Glenford J.; CONSTANTINE, Larry L. Structured design. *IBM Systems Journal*, Armonk, v. 13, n. 2, p. 115-139, 1974. DOI: 10.1147/sj.132.0115.

TAMURA, Toshiyo et al. Wearable photoplethysmographic sensors: past and present. *Electronics*, Basel, v. 3, n. 2, p. 282-302, 2014. DOI: 10.3390/electronics3020282.

TDK INVENSENSE. **MPU-9250 product specification**. Rev. 1.1. San Jose, 2016. Disponível em: <https://invensense.tdk.com/wp-content/uploads/2015/02/PS-MPU-9250A-01-v1.1.pdf>. Acesso em: 1 jul. 2026.

TDK INVENSENSE. **MPU-9250 register map and descriptions**. Rev. 1.6. San Jose, 2017. Disponível em: <https://invensense.tdk.com/wp-content/uploads/2017/11/RM-MPU-9250A-00-v1.6.pdf>. Acesso em: 1 jul. 2026.

VAN DIJK, Richard M. K.; GAWEHNS, Daniela; VAN LEEUWEN, Matthijs. WEARDA: recording wearable sensor data for human activity monitoring. *Journal of Open Research Software*, London, v. 11, art. 13, 2023. DOI: 10.5334/jors.454.

WEBSTER, John G. **Design of pulse oximeters**. Boca Raton: CRC Press, 1997.

WHITTLE, Michael W. **Gait analysis: an introduction**. 4. ed. Edinburgh: Butterworth-Heinemann, 2007.

WORLD HEALTH ORGANIZATION; WORLD METEOROLOGICAL ORGANIZATION; UNITED NATIONS ENVIRONMENT PROGRAMME; INTERNATIONAL COMMISSION ON NON-IONIZING RADIATION PROTECTION. **Global solar UV index: a practical guide**. Geneva: WHO, 2002.